pybind11——无缝连接C+11和Python

pybind11是一个只有头文件的轻量级库，它在导出C++类型到Python的同时，也导出Python类型到C++中，其主要目的是建立现有C++代码的Python绑定。它与David Abrahams的Boost.Python库目的和语法相似，都是通过编译期内省来推断类型信息，以最大程度地降低传统扩展模块中的重复样板代码。

Boost.Python的问题主要在于Boost本身，这也是我创建一个类似项目的原因。Boost是一套庞大且复杂的工具库，它几乎兼容所有的C++编译器。但这种兼容性是有成本的：为了支持那些极其古老且充满BUG的编译器版本，Boost不得不使用各种晦涩难懂的模板技巧与变通方法。现在，支持C++11的编译器已经被广泛使用，这种沉重结构已成为一种过大且不必要的依赖。

你可以把pybind11库想象成Boost.Python的一个小型独立版本，其中所有与python绑定生成无关的内容都被删除了。不算注释，pybind11核心头文件大约只有4K行代码，并且它只依赖于Python（2.7或3.5+，或PyPy）和C ++标准库。由于C++11语言的新特性(特别是元组、lambda函数和可变参数模板)，这种紧凑的实现才成为可能。自创建以来，这个库已经在很多方面超越了Boost.Python，多数常见情况下pybind11使得python绑定代码变得非常简单。

1.1 核心特性

pybind11可以将以下C++核心特性映射到Python：

- 函数入参和返回值可以是自定义数据结构的值、引用或者指针；
- 类成员方法和静态方法；
- 重载函数；
- 类成员变量和静态变量；
- 任意异常类型；
- 枚举；
- 回调函数；
- 迭代器和ranges；

- 自定义操作符；
- 单继承和多重继承；
- STL数据结构；
- 智能指针；
- Internal references with correct reference counting；
- 可以在Python中扩展带虚函数（和纯虚函数）的C++类；

1.2 好用的功能

除了上述核心功能外，pybind11还提供了一些好用的功能：

- 支持Python2.7, 3.5+, PyPy/PyPy3 7.3与实现无关的接口。
- 可以绑定带捕获参数的lambda函数，lambda捕获的数据存储生成的Python函数对象中。
- pybind11使用C++11移动构造函数和移动运算符，尽可能有效的转换自定义数据类型。(pybind11 uses C++11 move constructors and move assignment operators whenever possible to efficiently transfer custom data types.)
- 通过Python的buffer协议，可以很轻松地获取自定义类型的内存指针。这样，我们可以很方便地在C++矩阵类型（如Eigen）和NumPy之间快速转换，而无需昂贵的拷贝操作。
- pybind11可以自动将函数矢量化，以便它们透明地应用于以NumPy数组为参数的所有条目。
- 只需几行代码就可以支持Python基于切片的访问和赋值操作。
- 使用时只需要包含几个头文件即可，不用链接任何其他的库。
- 相比Boost.Python，生成的库文件更小，编译更快。
- 使用 `constexpr` 在编译器与计算函数签名，进一步减小了库文件大小。
- 可以轻松地让C++类型支持Python pickle和unpickle操作。

1.3 支持的编译器

1. Clang/LLVM 3.3以上 (Apple Xcode’s clang需要5.0.0以上版本)
2. GCC 4.8以上
3. Microsoft Visual Studio 2015 Update 3以上

4. Intel classic C++ compiler 18 or newer (ICC 20.2 tested in CI)
5. Cygwin/GCC (previously tested on 2.5.1)
6. NVCC (CUDA 11.0 tested in CI)
7. NVIDIA PGI (20.9 tested in CI)

1.4 关于

This project was created by Wenzel Jakob. Significant features and/or improvements to the code were contributed by Jonas Adler, Lori A. Burns, Sylvain Corlay, Eric Cousineau, Aaron Gokaslan, Ralf Grosse-Kunstleve, Trent Houliston, Axel Huebl, @hulucc, Yannick Jadoul, Sergey Lyskov Johan Mabille, Tomasz Miąsko, Dean Moldovan, Ben Pritchard, Jason Rhinelander, Boris Schäling, Pim Schellart, Henry Schreiner, Ivan Smirnov, Boris Staletic, and Patrick Stewart.

We thank Google for a generous financial contribution to the continuous integration infrastructure used by this project.

1.5 贡献

See the contributing guide for information on building and contributing to pybind11.

1.6 License

pybind11 is provided under a BSD-style license that can be found in the LICENSE file. By using, distributing, or contributing to this project, you agree to the terms and conditions of this license.

改动日志

主要介绍了各个发布版本增加的功能、改进点，以及修复的BUG。暂时不翻译吧，有兴趣的可以看官方文档。

更新指南

3. 安装说明

我们可以在pybind/pybind11 on GitHub获取到pybind11的源码。推荐pybind11开发者使用下面介绍的前三种方法之一，来获取pybind11。

3.1 以子模块的形式集成

当你的项目使用Git管理时，你可以将pybind11当做一个子模块嵌入到你的项目中。在你的git仓库，使用以下命令即可包含pybind11：

```
git submodule add -b stable ../../pybind/pybind11 extern/pybind11
git submodule update --init
```

这里假设你将项目的依赖放在了`extern`目录下，并且使用GitHub。如果你没有使用GitHub，可以使用完整的https或ssh URL来代替上面的相对URL`../../pybind/pybind11`。一些服务器可能需要`.git`扩展（GitHub不用）。

到这一步后，你可以直接include `extern/pybind11/include`目录即可。或者，你可以使用各种集成工具（见Build System一章)来包含pybind11。

3.2 通过PyPI来集成

你可以使用pip，通过PyPI来下载Pybind11的Python包，里面包含了源码已经CMake文件。像这样：

```
pip install pybind11
```

这样pybind11将以标准的Python包的形式提供。如果你想在root环境下直接使用pybind11，可以这样做：

```
pip install "pybind11[global]"
```

如果你使用系统自带的Python来安装，我们推荐在root环境下安装。这样会在`/usr/local/include/pybind11`和`/usr/local/share/cmake/pybind11`添加文件，除非你想这样。还是推荐你只在虚拟环境或你的`pyproject.toml`中使用。

3.3 通过conda-forge集成

You can use pybind11 with conda packaging via conda-forge:

```
conda install -c conda-forge pybind11
```

3.4 通过vcpkg集成

你可以通过Microsoft vcpkg依赖管理工具来下载和安装pybind11：

```
git clone https://github.com/Microsoft/vcpkg.git
cd vcpkg
./bootstrap-vcpkg.sh
./vcpkg integrate install
vcpkg install pybind11
```

3.5 通过brew全局安装

brew包管理（Homebrew on macOS, or Linuxbrew on Linux）有pybind11包。这样安装：

```
brew install pybind11
```

3.6 其他方法

Other locations you can find pybind11 are listed here; these are maintained by various packagers and the community.

pybind11使用指南

1. 基础用法

1.1 安装与编译

在安装python3-dev和下载了pybind11源码的前提下，可以直接include pybind11头文件目录和python3头文件目录即可。cmake示例如下：

```
set(PYTHON_TARGET_VER 3.6)
find_package(PythonInterp ${PYTHON_TARGET_VER} EXACT)
find_package(PythonLibs ${PYTHON_TARGET_VER} EXACT REQUIRED)

include_directories(pybind11_include_path)
include_directories(${PYTHON_INCLUDE_DIRS})
```

1.2 绑定函数

```
#include <pybind11/pybind11.h>

int add(int i, int j) {
    return i + j;
}

PYBIND11_MODULE(example, m) {
    m.doc() = "pybind11 example plugin"; // optional module docstring
    m.def("add", &add, "A function which adds two numbers");
}
```

宏 `PYBIND11_MODULE` 会创建模块初始化函数，它在Python中 `import` 模块时被调用。其参数分别是模块名，类型为 `py::module_` 的变量（m），是创建绑定的主要接口。`module_::def()` 方法，可以生成函数的绑定。

1.2.1 关键字参数

使用 `py::arg` 可以指定函数的参数名，Python侧调用函数时可以使用关键字参数，以增加代码的可读性。

```
m.def("add", &add, "A function which adds two numbers",
      py::arg("i"), py::arg("j"));
```

更简洁的写法：

```
// regular notation
m.def("add1", &add, py::arg("i"), py::arg("j"));
// shorthand
using namespace pybind11::literals;
m.def("add2", &add, "i"_a, "j"_a);
```

Python使用示例：

```
import example
example.add(i=1, j=2)  #3L
```

1.2.2 参数默认值

pybind11不能自动地提取默认参数，因为它不属于函数类型信息的一部分。我们需要借助 `arg` 在绑定时指定参数默认值：

```
m.def("add", &add, "A function which adds two numbers",
      py::arg("i") = 1, py::arg("j") = 2);
```

更简短的声明方式：

```
// regular notation
m.def("add1", &add, py::arg("i") = 1, py::arg("j") = 2);
// shorthand
m.def("add2", &add, "i"_a=1, "j"_a=2);
```

1.2.3 重载函数

重载方法即拥有相同的函数名，但入参不一样的函数。

在绑定重载函数时，我们需要增加函数签名相关的信息以消除歧义。绑定多个函数到同一个Python名称，将会自动创建函数重载链。Python将会依次匹配，找到最合适的重载函数。

```
m.def("add", static_cast<int(*)(int, int)>(&add), "A function which adds
two int numbers");
m.def("add", static_cast<double(*)(double, double)>(&add), "A function
which adds two double numbers");
```

如果你的编译器支持C++14，也可以使用下面的语法来转换重载函数：

```
m.def("add", py::overload_cast<int, int>(&add), "A function which adds two
int numbers");
m.def("add", py::overload_cast<double, double>(&add), "A function which
adds two double numbers");
```

这里，`py::overload_cast`仅需指定函数类型，不用给出返回值类型，以避免原语法带来的不必要的干扰(`void (Pet::*)`)。如果是基于const的重载，需要使用`py::const`标识。

1.3 导出变量

我们可以使用`attr`函数来注册需要导出到Python模块中的C++变量。内建类型和常规对象会在指定attriutes时自动转换，也可以使用`py::cast`来显式转换。

```
PYBIND11_MODULE(example, m) {
    m.attr("the_answer") = 42;
    py::object world = py::cast("World");
    m.attr("what") = world;
}
```

Python中使用如下：

```pyhton
>>> import example
>>> example.the_answer
42
>>> example.what
'World'
```

1.4 绑定类或结构体

现在我们来绑定一个C++自定义数据结构`Pet`。定义如下：

```
struct Pet {
    Pet(const std::string &name) : name(name) { }
    void setName(const std::string &name_) { name = name_; }
    const std::string &getName() const { return name; }

    std::string name;
};
```

绑定代码如下所示：

```cpp
#include <pybind11/pybind11.h>
namespace py = pybind11;

PYBIND11_MODULE(example, m) {
    py::class_<Pet>(m, "Pet")
        .def(py::init<const std::string &>())
        .def("setName", &Pet::setName)
        .def("getName", &Pet::getName)
        .def("__repr__",
            [](const Pet &a) {
                return "<example.Pet named '" + a.name + "'>";
            });
}
```

`class_`会创建C++ class或 struct的绑定。`init()`方法用于创建绑定类的构造函数，它使用类构造函数的参数类型作为模板参数，并包装相应的构造函数。

使用`print(p)`打印对象信息时，默认会得到一些没用的信息。我们可以绑定一个工具函数到`__repr__`方法，来返回可读性好的摘要信息。在不改变Pet类的基础上，使用一个匿名函数来完成这个功能是一个不错的选择。

Python使用示例如下；

```
>>> import example
>>> p = example.Pet("Molly")
>>> print(p)
<example.Pet named 'Molly'>
>>> p.getName()
u'Molly'
>>> p.setName("Charly")
>>> p.getName()
u'Charly'
```

静态成员函数需要使用`class_::def_static`来绑定。

1.4.1 成员函数

使用`class_::def_readwrite`方法可以导出公有成员变量，使用`class_::def_readonly`方法则可以导出只读成员。

```
py::class_<Pet>(m, "Pet")
    .def(py::init<const std::string &>())
    .def_readwrite("name", &Pet::name)
    // ... remainder ...
```

Python中使用示例如下：

```
>>> p = example.Pet("Molly")
>>> p.name
u'Molly'
>>> p.name = "Charly"
>>> p.name
u'Charly'
```

假设`Pet::name`是一个私有成员变量，向外提供setter和getters方法。

```
class Pet {
public:
    Pet(const std::string &name) : name(name) { }
    void setName(const std::string &name_) { name = name_; }
    const std::string &getName() const { return name; }
private:
    std::string name;
};
```

可以使用`class_::def_property()`(只读成员使用`class_::def_property_readonly()`)来定义并私有成员，并生成相应的setter和geter方法：

```
py::class_<Pet>(m, "Pet")
    .def(py::init<const std::string &>())
    .def_property("name", &Pet::getName, &Pet::setName)
    // ... remainder ...
```

只写属性通过将read函数定义为nullptr来实现。

相似的方法`class_::def_readwrite_static()`，`class_::def_readonly_static()` `class_::def_property_static()`，`class_::def_property_readonly_static()`用于绑定静态变量和属性。

1.4.2 动态属性

原生的Pyhton类可以动态地获取新属性：

```
>>> class Pet:
...     name = "Molly"
...
>>> p = Pet()
>>> p.name = "Charly"  # overwrite existing
>>> p.age = 2  # dynamically add a new attribute
```

默认情况下，从C++导出的类不支持动态属性，其可写属性必须是通过`class_::def_readwrite`或`class_::def_property`定义的。试图设置其他属性将产生错误：

```
>>> p = example.Pet()
>>> p.name = "Charly"  # OK, attribute defined in C++
>>> p.age = 2  # fail
AttributeError: 'Pet' object has no attribute 'age'
```

要让C++类也支持动态属性，我们需要在`py::class_`的构造函数添加`py::dynamic_attr`标识：

```
py::class_<Pet>(m, "Pet", py::dynamic_attr())
    .def(py::init<>())
    .def_readwrite("name", &Pet::name);
```

这样，之前报错的代码就能够正常运行了。

```
>>> p = example.Pet()
>>> p.name = "Charly"  # OK, overwrite value in C++
>>> p.age = 2  # OK, dynamically add a new attribute
>>> p.__dict__  # just like a native Python class
{'age': 2}
```

需要提醒一下，支持动态属性会带来小小的运行时开销。不仅仅因为增加了额外的`__dict__`属性，还因为处理循环引用时需要花费更多的垃圾收集跟踪花销。但是不必担心这个问题，因为原生Python类也有同样的开销。默认情况下，pybind11导出的类比原生Python类效率更高，使能动态属性也只是让它们处于同等水平而已。

1.4.3 重载方法

重载类的方法同上一节的普通函数重载，这里举个实例仅供参考：

```
struct Pet {
    Pet(const std::string &name, int age) : name(name), age(age) { }

    void set(int age_) { age = age_; }
    void set(const std::string &name_) { name = name_; }

    std::string name;
    int age;
};

// method 1
py::class_<Pet>(m, "Pet")
   .def(py::init<const std::string &, int>())
   .def("set", static_cast<void (Pet::*)(int)>(&Pet::set), "Set the pet's
age")
   .def("set", static_cast<void (Pet::*)(const std::string &)>(&Pet::set),
"Set the pet's name");

// method 2
py::class_<Pet>(m, "Pet")
    .def("set", py::overload_cast<int>(&Pet::set), "Set the pet's age")
    .def("set", py::overload_cast<const std::string &>(&Pet::set), "Set
the pet's name");
```

1.5 绑定枚举类型

对于C风格的枚举类型，绑定示例如下：

```
enum Flags {
    Read = 4,
    Write = 2,
    Execute = 1
};

py::enum_<Flags>(m, "Flags", py::arithmetic())
    .value("Read", Flags::Read)
    .value("Write", Flags::Write)
    .value("Execute", Flags::Execute)
    .export_values();
```

`enum_::export_values()` 用来导出枚举项到父作用域，C++11的强枚举类型需要跳过这点。

枚举类型的枚举项会被导出到类 `__members__` 属性中， `name` 属性可以返回枚举值的名称的unicode字符串， `str(enum)` 也可以做到，但两者的实现目标不同。

1.6 接收*args和**kwargs参数

Python的函数可以接收任意数量的参数和关键字参数：

```
def generic(*args, **kwargs):
    ...  # do something with args and kwargs
```

我们也可以通过pybind11来创建这样的函数：

```
void generic(py::args args, const py::kwargs& kwargs) {
    /// .. do something with args
    if (kwargs)
        /// .. do something with kwargs
}

/// Binding code
m.def("generic", &generic);
```

`py::args` 继承自 `py::tuple` ， `py::kwargs` 继承自 `py::dict` 。

2. 函数绑定进阶

2.1 返回值策略

Python和C++在管理内存和对象生命周期管理上存在本质的区别。这导致我们在创建返回no-trivial类型的函数绑定时会出问题。仅通过类型信息，我们无法明确是Python侧需要接管返回值并负责释放资源，还是应该由C++侧来处理。因此，pybind11提供了一些返回值策略来确定由哪方管理资源。这些策略通过`model::def()`和`class_def()`来指定，默认策略为`return_value_policy::automatic`。

返回值策略难以捉摸，正确地选择它们则显得尤为重要。下面我们通过一个简单的例子来阐释选择错误的情形：

```
/* Function declaration */
Data *get_data() { return _data; /* (pointer to a static data structure)
*/ }
...

/* Binding code */
m.def("get_data", &get_data); // <-- KABOOM, will cause crash when called
from Python
```

当Python侧调用`get_data()`方法时，返回值（原生C++类型）必须被转换为合适的Python类型。在这个例子中，默认的返回值策略（`return_value_policy::automatic`）使得pybind11获取到了静态变量`_data`的所有权。

当Python垃圾收集器最终删除`_data`的Python封装时，pybind11将尝试删除C++实例（通过operator delete()）。这时，这个程序将以某种隐蔽的错误并涉及静默数据破坏的方式崩溃。

对于上面的例子，我们应该指定返回值策略为`return_value_policy::reference`，这样全局变量的实例仅仅被引用，而不涉及到所有权的转移：

```
m.def("get_data", &get_data, py::return_value_policy::reference);
```

另一方面，引用策略在多数其他场合并不是正确的策略，忽略所有权的归属可能导致资源泄漏。作为一个使用pybind11的开发者，熟悉不同的返回值策略及其适用场合尤为重要。下面的表格将提供所有策略的概览：

返回值策略	描述
`return_value_policy::take_ownership`	引用现有对象（不创建一个新对象所有权。在引用计数为0时，Pyht构函数和delete操作销毁对象。
`return_value_policy::copy`	拷贝返回值，这样Python将拥有拷象。该策略相对来说比较安全，因例的生命周期是分离的。
`return_value_policy::move`	使用 `std::move` 来移动返回值的内例，新实例的所有权在Python。该来说比较安全，因为两个实例的生分离的。
`return_value_policy::reference`	引用现有对象，但不拥有所有权。该对象的生命周期管理，并在对象用时负责析构它。注意：当Pytho用引用的对象时，C++侧删除对象义行为。
`return_value_policy::reference_internal`	返回值的生命周期与父对象的生命定，即被调用函数或属性的 `this` 象。这种策略与reference策略类似了 `keep_alive<0, 1>` 调用策略保还被Python引用时，其父对象就不回收掉。这是由 `def_property` 、`def_readwrite` 创建的属性gette认返回值策略。
`return_value_policy::automatic`	当返回值是指针时，该策略使用 `return_value_policy::take_ow` 反之对左值和右值引用使用 `return_value_policy::copy` 。

返回值策略	描述
	的描述，了解所有这些不同的策略 这是 py::class_ 封装类型的默认
return_value_policy::automatic_reference	和上面一样，但是当返回值是指针 return_value_policy::referer 这是在C++代码手动调用Python函 pybind11/stl.h 中的casters时的 策略。你可能不需要显式地使用该

返回值策略也可以应用于属性：

```
class_<MyClass>(m, "MyClass")
    .def_property("data", &MyClass::getData, &MyClass::setData,
                  py::return_value_policy::copy);
```

在技术层面，上述代码会将策略同时应用于getter和setter函数，但是setter函数并不关心返回值策略，这样做仅仅出于语法简洁的考虑。或者，你可以通过 cpp_function 构造函数来传递目标参数：

```
class_<MyClass>(m, "MyClass")
    .def_property("data"
        py::cpp_function(&MyClass::getData,
py::return_value_policy::copy),
        py::cpp_function(&MyClass::setData)
    );
```

注意：代码使用无效的返回值策略将导致未初始化内存或多次free数据结构，这将导致难以调试的、不确定的问题和段错误。因此，花点时间来理解上面表格的各个选项是值得的。

提示：

1. 上述策略的另一个重点是，他们仅可以应用于pybind11还不知晓的实例，这时策略将澄清返回值的生命周期和所有权问题。当pybind11已经知晓参数（通过其在内存中的类型和地址来识别），它将返回已存在的Python对象封装，而不是创建一份拷贝。
2. 下一节将讨论上面表格之外的调用策略，他涉及到返回值和函数参数的引用关系。

3. 可以考虑使用智能指针来代替复杂的调用策略和生命周期管理逻辑。智能指针会告诉你一个对象是否仍被C++或Python引用，这样就可以消除各种可能引发crash或未定义行为的矛盾。对于返回智能指针的函数，没必要指定返回值策略。

2.2 附加的调用策略

除了以上的返回值策略外，进一步指定调用策略可以表明参数间的依赖关系，确保函数调用的稳定性。

保活（keep alive）

当一个C++容器对象包含另一个C++对象时，我们需要使用该策略。`keep_alive<Nurse, Patient>` 表明至少在索引Nurse被回收前，索引Patient应该被保活。0表示返回值，1及以上表示参数索引。1表示隐含的参数this指针，而常规参数索引从2开始。当Nurse的值在运行前被检测到为None时，调用策略将什么都不做。

当nurse不是一个pybind11注册类型时，实现依赖于创建对nurse对象弱引用的能力。如果nurse对象不是pybind11注册类型，也不支持弱引用，程序将会抛出异常。

如果你使用一个错误的参数索引，程序将会抛出“Could not cativate keep_alive!“警告的运行时异常。这时，你应该review你代码中使用的索引。

参见下面的例子：一个list append操作，将新添加元素的生命周期绑定到添加的容器对象上：

```
py::class_<List>(m, "List").def("append", &List::append, py::keep_alive<1, 2>());
```

为了一致性，构造函数的实参索引也是相同的。索引1仍表示this指针，索引0表示返回值（构造函数的返回值被认为是void）。下面的示例将构造函数入参的生命周期绑定到被构造对象上。

```
py::class_<Nurse>(m, "Nurse").def(py::init<Patient &>(), py::keep_alive<1, 2>());
```

> Note: `keep_alive` 与Boost.Python中的 `with_custodian_and_ward` 和 `with_custodian_and_ward_postcall` 相似。

Call guard

`call_guard<T>` 策略允许任意T类型的scope guard应用于整个函数调用。示例如下：

```
m.def("foo", foo, py::call_guard<T>());
```

上面的代码等价于：

```
m.def("foo", [](args...) {
    T scope_guard;
    return foo(args...); // forwarded arguments
});
```

仅要求模板参数T是可构造的，如 `gil_scoped_release` 就是一个非常有用的类型。

`call_guard` 支持同时制定多个模板参数， `call_guard<T1, T2, T3 ...>` 。构造顺序是从左至右，析构顺序则相反。

> See also: `test/test_call_policies.cpp` 含有更丰富的示例来展示 `keep_alive` 和 `call_guard` 的用法。

2.3 Keyword-only参数

Python3提供了keyword-only参数（在函数定义中使用 `*` 作为匿名参数）：

```
def f(a, *, b):  # a can be positional or via keyword; b must be via
keyword
    pass

f(a=1, b=2)  # good
f(b=2, a=1)  # good
f(1, b=2)  # good
f(1, 2)  # TypeError: f() takes 1 positional argument but 2 were given
```

pybind11提供了`py::kw_only`对象来实现相同的功能：

```
m.def("f", [](int a, int b) { /* ... */ },
      py::arg("a"), py::kw_only(), py::arg("b"));
```

注意，该特性不能与`py::args`一起使用。

2.4 Positional-only参数

python3.8引入了Positional-only参数语法，pybind11通过`py::pos_only()`来提供相同的功能：

```
m.def("f", [](int a, int b) { /* ... */ },
       py::arg("a"), py::pos_only(), py::arg("b"));
```

现在，你不能通过关键字来给定`a`参数。该特性可以和keyword-only参数一起使用。

2.5 Non-converting参数

有些参数可能支持类型转换，如：

- 通过`py::implicitly_convertible<A,B>()`进行隐式转换
- 将整形变量传给入参为浮点类型的函数
- 将非复数类型（如float）传给入参为`std::complex<float>`类型的函数

- Calling a function taking an Eigen matrix reference with a numpy array of the wrong type or of an incompatible data layout.

有时这种转换并不是我们期望的，我们可能更希望绑定代码抛出错误，而不是转换参数。通过`py::arg`来调用`.noconvert()`方法可以实现这个事情。

```
m.def("floats_only", [](double f) { return 0.5 * f; },
py::arg("f").noconvert());
m.def("floats_preferred", [](double f) { return 0.5 * f; }, py::arg("f"));
```

尝试进行转换时，将抛出`TypeError`异常：

```
>>> floats_preferred(4)
2.0
>>> floats_only(4)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: floats_only(): incompatible function arguments. The following
argument types are supported:
    1. (f: float) -> float

Invoked with: 4
```

该方法可以与缩写符号`_a`和默认参数配合使用，像这样`py::arg().noconvert()`。

3. 类绑定进阶

3.1 继承与多态

现在有两个具有继承关系的类：

```cpp
struct Pet {
    Pet(const std::string &name) : name(name) { }
    std::string name;
};

struct Dog : Pet {
    Dog(const std::string &name) : Pet(name) { }
    std::string bark() const { return "woof!"; }
};
```

pybind11提供了两种方法来指明继承关系：1）将C++基类作为派生类`class_`的模板参数；2）将基类名作为`class_`的参数绑定到派生类。两种方法是等效的。

```cpp
py::class_<Pet>(m, "Pet")
   .def(py::init<const std::string &>())
   .def_readwrite("name", &Pet::name);

// Method 1: template parameter:
py::class_<Dog, Pet /* <- specify C++ parent type */>(m, "Dog")
    .def(py::init<const std::string &>())
    .def("bark", &Dog::bark);

// Method 2: pass parent class_ object:
py::class_<Dog>(m, "Dog", pet /* <- specify Python parent type */)
    .def(py::init<const std::string &>())
    .def("bark", &Dog::bark);
```

指明继承关系后，派生类实例将获得两者的字段和方法：

```
>>> p = example.Dog("Molly")
>>> p.name
u'Molly'
>>> p.bark()
u'woof!'
```

上面的例子是一个常规非多态的继承关系，表现在Python就是：

```
// 返回一个指向派生类的基类指针
m.def("pet_store", []() { return std::unique_ptr<Pet>(new Dog("Molly"));
});
```

```
>>> p = example.pet_store()
>>> type(p)  # `Dog` instance behind `Pet` pointer
Pet          # no pointer downcasting for regular non-polymorphic types
>>> p.bark()
AttributeError: 'Pet' object has no attribute 'bark'
```

`pet_store`函数返回了一个Dog实例，但由于基类并非多态类型，Python只识别到了Pet。在C++中，一个类至少有一个虚函数才会被视为多态类型。pybind11会自动识别这种多态机制。

```
struct PolymorphicPet {
    virtual ~PolymorphicPet() = default;
};

struct PolymorphicDog : PolymorphicPet {
    std::string bark() const { return "woof!"; }
};

// Same binding code
py::class_<PolymorphicPet>(m, "PolymorphicPet");
py::class_<PolymorphicDog, PolymorphicPet>(m, "PolymorphicDog")
    .def(py::init<>())
    .def("bark", &PolymorphicDog::bark);

// Again, return a base pointer to a derived instance
m.def("pet_store2", []() { return std::unique_ptr<PolymorphicPet>(new
PolymorphicDog); });
```

```
>>> p = example.pet_store2()
>>> type(p)
PolymorphicDog  # automatically downcast
>>> p.bark()
u'woof!'
```

pybind11会自动地将一个指向多态基类的指针，向下转型为实际的派生类类型。这和C++常见的情况不同，我们不仅可以访问基类的虚函数，还能获取到通过基类看不到的，具体的派生类的方法和属性。

3.2 Python继承C++类

对于一个拥有纯虚函数的类，使用常规的绑定方法并在Python中直接继承它会报错，因为纯虚基类是不可构造的。

```
class Animal {
public:
    virtual ~Animal() { }
    virtual std::string go(int n_times) = 0;
};

class Dog : public Animal {
public:
    std::string go(int n_times) override {
        std::string result;
        for (int i=0; i<n_times; ++i)
            result += "woof! ";
        return result;
    }
};

std::string call_go(Animal *animal) {
    return animal->go(3);
}

PYBIND11_MODULE(example, m) {
    py::class_<Animal>(m, "Animal")
        .def("go", &Animal::go);

    py::class_<Dog, Animal>(m, "Dog")
        .def(py::init<>());

    m.def("call_go", &call_go);
}
```

这样绑定之后，用户在Python中继承实现Animal会报错，提示“No constructor defined! “。

这时，我们需要定义一个新的Animal类作为辅助跳板：

```
class PyAnimal : public Animal {
public:
    /* Inherit the constructors */
    using Animal::Animal;

    /* Trampoline (need one for each virtual function) */
    std::string go(int n_times) override {
        PYBIND11_OVERRIDE_PURE(
            std::string, /* Return type */
            Animal,      /* Parent class */
            go,          /* Name of function in C++ (must match Python
name) */
            n_times      /* Argument(s) */
        );
    }
};
```

定义纯虚函数时需要使用`PYBIND11_OVERRIDE_PURE`宏，而有默认实现的虚函数则使用`PYBIND11_OVERRIDE`。`PYBIND11_OVERRIDE_PURE_NAME`和`PYBIND11_OVERRIDE_NAME`宏的功能类似，主要用于C函数名和Python函数名不一致的时候。以`__str__`为例：

```
std::string toString() override {
  PYBIND11_OVERRIDE_NAME(
      std::string, // Return type (ret_type)
      Animal,      // Parent class (cname)
      "__str__",   // Name of method in Python (name)
      toString,    // Name of function in C++ (fn)
  );
}
```

Animal类的绑定代码也需要一些微调：

```
PYBIND11_MODULE(example, m) {
    py::class_<Animal, PyAnimal /* <--- trampoline*/>(m, "Animal")
        .def(py::init<>())
        .def("go", &Animal::go);

    py::class_<Dog, Animal>(m, "Dog")
        .def(py::init<>());

    m.def("call_go", &call_go);
}
```

pybind11通过向 `class_` 指定额外的模板参数PyAnimal，让我们可以在Python中继承Animal类。

接下来，我们可以像往常一样定义构造函数。绑定时我们需要使用真实类，而不是辅助类。

```
py::class_<Animal, PyAnimal /* <--- trampoline*/>(m, "Animal");
    .def(py::init<>())
    .def("go", &PyAnimal::go); /* <--- THIS IS WRONG, use &Animal::go */
```

下面的Python代码展示了我们继承并重载了 `Animal::go` 方法，并通过虚函数来调用它：

```
from example import *
d = Dog()
call_go(d)      # u'woof! woof! woof! '
class Cat(Animal):
    def go(self, n_times):
        return "meow! " * n_times

c = Cat()
call_go(c)    # u'meow! meow! meow! '
```

如果你在派生的Python类中自定义了一个构造函数，你必须保证显示调用C++构造函数(通过 `__init__`)，不管它是否为默认构造函数。否则，实例属于C++那部分的内存就未初始化，可能导致未定义行为。在pybind11 2.6版本中，这种错误将会抛出 `TypeError` 异常。

```
class Dachshund(Dog):
    def __init__(self, name):
        Dog.__init__(self)  # Without this, a TypeError is raised.
        self.name = name

    def bark(self):
        return "yap!"
```

注意必须显式地调用`__init__`，而不应该使用`supper()`。在一些简单的线性继承中，`supper()`或许可以正常工作；一旦你混合Python和C++类使用多重继承，由于Python MRO和C++的机制，一切都将崩溃。

3.3 虚函数与继承

综合考虑虚函数与继承时，你需要为每个你允许在Python派生类中重载的方法提供重载方式。下面我们扩展Animal和Dog来举例：

```
class Animal {
public:
    virtual std::string go(int n_times) = 0;
    virtual std::string name() { return "unknown"; }
};
class Dog : public Animal {
public:
    std::string go(int n_times) override {
        std::string result;
        for (int i=0; i<n_times; ++i)
            result += bark() + " ";
        return result;
    }
    virtual std::string bark() { return "woof!"; }
};
```

上节涉及到的Animal辅助类仍是必须的，为了让Python代码能够继承`Dog`类，我们也需要为`Dog`类增加一个跳板类，来实现`bark()`和继承自Animal的`go()`、`name()`等重载方法（即便Dog类并不直接重载name方法）。

```
class PyAnimal : public Animal {
public:
    using Animal::Animal; // Inherit constructors
    std::string go(int n_times) override {
PYBIND11_OVERRIDE_PURE(std::string, Animal, go, n_times); }
    std::string name() override { PYBIND11_OVERRIDE(std::string, Animal,
name, ); }
};
class PyDog : public Dog {
public:
    using Dog::Dog; // Inherit constructors
    std::string go(int n_times) override { PYBIND11_OVERRIDE(std::string,
Dog, go, n_times); }
    std::string name() override { PYBIND11_OVERRIDE(std::string, Dog,
name, ); }
    std::string bark() override { PYBIND11_OVERRIDE(std::string, Dog,
bark, ); }
};
```

> 注意到 `name()` 和 `bark()` 尾部的逗号，这用来说明辅助类的函数不带任何参数。当函数至少有一个参数时，应该省略尾部的逗号。

注册一个继承已经在pybind11中注册的带虚函数的类，同样需要为其添加辅助类，即便它没有定义或重载任何虚函数：

```
class Husky : public Dog {};
class PyHusky : public Husky {
public:
    using Husky::Husky; // Inherit constructors
    std::string go(int n_times) override {
PYBIND11_OVERRIDE_PURE(std::string, Husky, go, n_times); }
    std::string name() override { PYBIND11_OVERRIDE(std::string, Husky,
name, ); }
    std::string bark() override { PYBIND11_OVERRIDE(std::string, Husky,
bark, ); }
};
```

我们可以使用模板辅助类将简化这类重复的绑定工作，这对有多个虚函数的基类尤其有用：

```
template <class AnimalBase = Animal> class PyAnimal : public AnimalBase {
public:
    using AnimalBase::AnimalBase; // Inherit constructors
    std::string go(int n_times) override {
PYBIND11_OVERRIDE_PURE(std::string, AnimalBase, go, n_times); }
    std::string name() override { PYBIND11_OVERRIDE(std::string,
AnimalBase, name, ); }
};
template <class DogBase = Dog> class PyDog : public PyAnimal<DogBase> {
public:
    using PyAnimal<DogBase>::PyAnimal; // Inherit constructors
    // Override PyAnimal's pure virtual go() with a non-pure one:
    std::string go(int n_times) override { PYBIND11_OVERRIDE(std::string,
DogBase, go, n_times); }
    std::string bark() override { PYBIND11_OVERRIDE(std::string, DogBase,
bark, ); }
};
```

这样，我们只需要一个辅助方法来定义虚函数和纯虚函数的重载了。只是这样编译器就需要生成许多额外的方法和类。

下面我们在pybind11中注册这些类：

```
py::class_<Animal, PyAnimal<>> animal(m, "Animal");
py::class_<Dog, Animal, PyDog<>> dog(m, "Dog");
py::class_<Husky, Dog, PyDog<Husky>> husky(m, "Husky");
// ... add animal, dog, husky definitions
```

注意，Husky不需要一个专门的辅助类，因为它没定义任何新的虚函数和纯虚函数的重载。

Python中的使用示例：

```
class ShihTzu(Dog):
    def bark(self):
        return "yip!"
```

3.4 非公有析构函数

如果一个类拥有私有或保护的析构函数（例如单例类），通过pybind11绑定类时编译器将会报错。本质的问题是`std::unique_ptr`智能指针负责管理实例的生命周期需要引用析构函数，即便没有资源需要回收。Pybind11提供了辅助类`py::nodelete`来禁止对析构函数的调用。这种情况下，C++侧负责析构对象避免内存泄漏就十分重要。

```
/* ... definition ... */

class MyClass {
private:
    ~MyClass() { }
};

/* ... binding code ... */

py::class_<MyClass, std::unique_ptr<MyClass, py::nodelete>>(m, "MyClass")
    .def(py::init<>())
```

3.5 隐式转换

假设项目中有A和B两个类型，A可以直接转换为B。

```
py::class_<A>(m, "A")
    /// ... members ...

py::class_<B>(m, "B")
    .def(py::init<A>())
    /// ... members ...

m.def("func",
    [](const B &) { /* .... */ }
);
```

如果想func函数传入A类型的参数a，Pyhton侧需要这样写`func(B(a))`，而C++则可以直接使用`func(a)`，自动将A类型转换为B类型。

这种情形下（B有一个接受A类型参数的构造函数），我们可以使用如下声明来让Python侧也支持类似的隐式转换：

```
py::implicitly_convertible<A, B>();
```

3.6 重载操作符

假设有这样一个类`Vector2`，它通过重载操作符实现了向量加法和标量乘法。

```
class Vector2 {
public:
    Vector2(float x, float y) : x(x), y(y) { }

    Vector2 operator+(const Vector2 &v) const { return Vector2(x + v.x, y
+ v.y); }
    Vector2 operator*(float value) const { return Vector2(x * value, y *
value); }
    Vector2& operator+=(const Vector2 &v) { x += v.x; y += v.y; return
*this; }
    Vector2& operator*=(float v) { x *= v; y *= v; return *this; }

    friend Vector2 operator*(float f, const Vector2 &v) {
        return Vector2(f * v.x, f * v.y);
    }

    std::string toString() const {
        return "[" + std::to_string(x) + ", " + std::to_string(y) + "]";
    }
private:
    float x, y;
};
```

操作符绑定代码如下：

```
#include <pybind11/operators.h>

PYBIND11_MODULE(example, m) {
    py::class_<Vector2>(m, "Vector2")
        .def(py::init<float, float>())
        .def(py::self + py::self)
        .def(py::self += py::self)
        .def(py::self *= float())
        .def(float() * py::self)
        .def(py::self * float())
        .def(-py::self)
        .def("__repr__", &Vector2::toString);
}
```

`.def(py::self * float())`是如下代码的简短标记：

```
.def("__mul__", [](const Vector2 &a, float b) {
    return a * b;
}, py::is_operator())
```

3.7 深拷贝支持

Python通常在赋值中使用引用。有时需要一个真正的拷贝，以防止修改所有的拷贝实例。`copy`模块提供了这样的拷贝能力。

在Python3中，带pickle支持的类自带深拷贝能力。但是，自定义`__copy__`和`__deepcopy__`方法能够提高拷贝的性能。在Python2.7中，由于pybind11只支持cPickle，要想实现深拷贝，自定义这两个方法必须实现。

对于一些简单的类，可以使用拷贝构造函数来实现深拷贝。如下所示：

```
py::class_<Copyable>(m, "Copyable")
    .def("__copy__",  [](const Copyable &self) {
        return Copyable(self);
    })
    .def("__deepcopy__", [](const Copyable &self, py::dict) {
        return Copyable(self);
    }, "memo"_a);
```

> Note: 本例中不会复制动态属性。

3.8 多重继承

pybind11支持绑定多重继承的类，只需在将所有基类作为`class_`的模板参数即可：

```
py::class_<MyType, BaseType1, BaseType2, BaseType3>(m, "MyType")
    ...
```

基类间的顺序任意，甚至可以穿插使用别名或者holder类型，pybind11能够自动识别它们。唯一的要求就是第一个模板参数必须是类型本身。

允许Python中定义的类继承多个C++类，也允许混合继承C++类和Python类。

有一个关于该特性实现的警告：当仅指定一个基类，实际上有多个基类时，pybind11会认为它并没有使用多重继承，这将导致未定义行为。对于这个问题，我们可以在类构造函数中添加`multiple_inheritance`的标识。

```
py::class_<MyType, BaseType2>(m, "MyType", py::multiple_inheritance());
```

当模板参数列出了多个基类时，无需使用该标识。

3.9 绑定protected成员函数

通常不可能向Python公开protected 成员函数：

```
class A {
protected:
    int foo() const { return 42; }
};

py::class_<A>(m, "A")
    .def("foo", &A::foo); // error: 'foo' is a protected member of 'A'
```

因为非公有成员函数意味着外部不可调用。但我们还是希望在Python派生类中使用protected 函数。我们可以通过下面的方式来实现：

```
class A {
protected:
    int foo() const { return 42; }
};

class Publicist : public A { // helper type for exposing protected
functions
public:
    using A::foo; // inherited with different access modifier
};

py::class_<A>(m, "A") // bind the primary class
    .def("foo", &Publicist::foo); // expose protected methods via the
publicist
```

因为 `&Publicist::foo` 和 `&A::foo` 准确地说是同一个函数（相同的签名和地址），仅仅是获取方式不同。 `Publicist` 的唯一意图，就是将函数的作用域变为 `public` 。

如果是希望公开在Python侧重载的 `protected` 虚函数，可以将publicist pattern与之前提到的trampoline相结合：

```cpp
class A {
public:
    virtual ~A() = default;

protected:
    virtual int foo() const { return 42; }
};

class Trampoline : public A {
public:
    int foo() const override { PYBIND11_OVERRIDE(int, A, foo, ); }
};

class Publicist : public A {
public:
    using A::foo;
};

py::class_<A, Trampoline>(m, "A") // <-- `Trampoline` here
    .def("foo", &Publicist::foo); // <-- `Publicist` here, not
`Trampoline`!
```

3.10 绑定final类

在C++11中，我们可以使用 `findal` 关键字来确保一个类不被继承。 `py::is_final` 属性则可以用来确保一个类在Python中不被继承。底层的C++类型不需要定义为final。

```cpp
class IsFinal final {};

py::class_<IsFinal>(m, "IsFinal", py::is_final());
```

在Python中试图继承这个类，将导致错误：

```
class PyFinalChild(IsFinal):
    pass

TypeError: type 'IsFinal' is not an acceptable base type
```

4. 异常处理

4.1 C++内置异常到Python异常的转换

当Python通过pybind11调用C++代码时，pybind11将捕获C++异常，并将其翻译为对应的Python异常后抛出。这样Python代码就能够处理它们。

pybind11定义了`std::exception`及其标准子类，和一些特殊异常到Python异常的翻译。由于它们不是真正的Python异常，所以不能使用Python C API来检查。相反，它们是纯C++异常，当它们到达异常处理器时，pybind11将其翻译为对应的Python异常。

Exception thrown by C++	Translated to Python exception type
`std::exception`	`RuntimeError`
`std::bad_alloc`	`MemoryError`
`std::domain_error`	`ValueError`
`std::invalid_argument`	`ValueError`
`std::length_error`	`ValueError`
`std::out_of_range`	`IndexError`
`std::range_error`	`ValueError`
`std::overflow_error`	`OverflowError`
`pybind11::stop_iteration`	`StopIteration` (used to implement custom iterators)
`pybind11::index_error`	`IndexError` (used to indicate out of bounds access in `__getitem__`, `__setitem__`, etc.)
`pybind11::key_error`	`KeyError` (used to indicate out of bounds access in `__getitem__`, `__setitem__` in dict-like objects, etc.)
`pybind11::value_error`	`ValueError` (used to indicate wrong value passed in `container.remove(...)`)
`pybind11::type_error`	`TypeError`

Exception thrown by C++	Translated to Python exception type
`pybind11::buffer_error`	`BufferError`
`pybind11::import_error`	`ImportError`
`pybind11::attribute_error`	`AttributeError`
Any other exception	`RuntimeError`

异常翻译不是双向的。即上述异常不会捕获源自Python的异常。Python的异常，需要捕获`pybind11::error_already_set`。

这里有个特殊的异常，当入参不能转化为Python对象时，`handle::call()`将抛出`cast_error`异常。

4.2 注册定制异常翻译

如果上述默认异常转换策略不够用，pybind11也提供了注册自定义异常翻译的支持。类似于pybind11 class，异常翻译也可以定义在模块内或global。要注册一个使用C++异常的`what()`方法将C++到Python的异常转换，可以使用下面的方法：

```
py::register_exception<CppExp>(module, "PyExp");
```

这个调用在指定模块创建了一个名称为PyExp的Python异常，并自动将CppExp相关的异常转换为PyExp异常。

相似的函数可以注册模块内的异常翻译：

```
py::register_local_exception<CppExp>(module, "PyExp");
```

方法的第三个参数handle可以指定异常的基类：

```
py::register_exception<CppExp>(module, "PyExp", PyExc_RuntimeError);
py::register_local_exception<CppExp>(module, "PyExp", PyExc_RuntimeError);
```

这样，PyExp异常可以捕获PyExp和RuntimeError。

Python内置的异常类型可以参考Python文档Standard Exceptions，默认的基类为`PyExc_Exception`。

`py::register_exception_translator(translator)` 和 `py::register_local_exception_translator(translator)` 提供了更高级的异常翻译功能，它可以注册任意的异常类型。函数接受一个无状态的回调函数`void(std::exception_ptr)`。

9.3 在C++中处理Python异常

当C++调用Python函数时（回调函数或者操作Python对象），若Python有异常抛出，pybind11会将Python异常转化为`pybind11::error_already_set`类型的异常，它包含了一个C++字符串描述和实际的Python异常。`error_already_set`用于将Python异常传回Python（或者在C++侧处理）。

Exception raised in Python	Thrown as C++ exception type
Any Python `Exception`	`pybind11::error_already_set`

举个例子：

```
try {
    // open("missing.txt", "r")
    auto file = py::module_::import("io").attr("open")("missing.txt",
"r");
    auto text = file.attr("read")();
    file.attr("close")();
} catch (py::error_already_set &e) {
    if (e.matches(PyExc_FileNotFoundError)) {
        py::print("missing.txt not found");
    } else if (e.matches(PyExc_PermissionError)) {
        py::print("missing.txt found but not accessible");
    } else {
        throw;
    }
}
```

该方法并不适用与C++到Python的翻译，Python侧抛出的异常总是被翻译为`error_already_set`.

```
try {
    py::eval("raise ValueError('The Ring')");
} catch (py::value_error &boromir) {
    // Boromir never gets the ring
    assert(false);
} catch (py::error_already_set &frodo) {
    // Frodo gets the ring
    py::print("I will take the ring");
}

try {
    // py::value_error is a request for pybind11 to raise a Python
exception
    throw py::value_error("The ball");
} catch (py::error_already_set &cat) {
    // cat won't catch the ball since
    // py::value_error is not a Python exception
    assert(false);
} catch (py::value_error &dog) {
    // dog will catch the ball
    py::print("Run Spot run");
    throw;  // Throw it again (pybind11 will raise ValueError)
}
```

9.4 处理Python C API的错误

尽可能地使用pybind11 wrappers代替直接调用Python C API。如果确实需要直接使用Python C API，除了需要手动管理引用计数外，还必须遵守pybind11的错误处理协议。

在调用Python C API后，如果Python返回错误，需要调用`throw py::error_already_set();`语句，让pybind11来处理异常并传递给Python解释器。这包括对错误设置函数的调用，如`PyErr_SetString`。

```
PyErr_SetString(PyExc_TypeError, "C API type error demo");
throw py::error_already_set();

// But it would be easier to simply...
throw py::type_error("pybind11 wrapper type error");
```

也可以调用 `PyErr_Clear` 来忽略错误。

任何Python错误必须被抛出或清除，否则Python/pybind11将处于无效的状态。

9.5 处理unraiseable异常

如果Python调用的C++析构函数或任何标记为 `noexcept(true)` 的函数抛出了异常，该异常不会传播出去。如果它们在调用图中抛出或捕捉不到任何异常，c++运行时将调用std::terminate()立即终止程序。在C++析构函数中调用Python尤其需要注意异常的捕获，必须捕获所有 `error_already_set` 类型的异常，并使用 `error_already_set::discard_as_unraisable()` 来抛弃Python异常。

类似的，在类 `__del__` 方法引发的Python异常也不会传播，但被Python作为unraisable错误记录下来。在Python 3.8+中，将触发system hook，并记录auditing event日志。

任何noexcept函数应该使用try-catch代码块来捕获 `error_already_set` （或其他可能出现的异常）。pybind11包装的Python异常并非真正的Python异常，它是pybind11捕获并转化的C++异常。noexcept函数不能传播这些异常。我们可以将它们转换为Python异常，然后丢弃 `discard_as_unraisable` ，如下所示。

```
void nonthrowing_func() noexcept(true) {
    try {
        // ...
    } catch (py::error_already_set &eas) {
        // Discard the Python error using Python APIs, using the C++ magic
        // variable __func__. Python already knows the type and value and
of the
        // exception object.
        eas.discard_as_unraisable(__func__);
    } catch (const std::exception &e) {
        // Log and discard C++ exceptions.
        third_party::log(e);
    }
}
```

5. 类型转换

6. python C++接口

7. 杂项

7.1 关于便利宏的说明

pybind11提供了一些便利宏如`PYBIND11_DECLARE_HOLDER_TYPE()`和`PYBIND11_OVERRIDE_*`。由于这些宏只是在预处理中计算(预处理程序没有类型的概念)，它们会被模板参数中的逗号搞混。如：

```
PYBIND11_OVERRIDE(MyReturnType<T1, T2>, Class<T3, T4>, func)
```

预处理器会将其解释为5个参数（逗号分隔），而不是3个。有两种方法可以处理这个问题：使用类型别名，或者使用`PYBIND11_TYPE`包裹类型。

```
// Version 1: using a type alias
using ReturnType = MyReturnType<T1, T2>;
using ClassType = Class<T3, T4>;
PYBIND11_OVERRIDE(ReturnType, ClassType, func);

// Version 2: using the PYBIND11_TYPE macro:
PYBIND11_OVERRIDE(PYBIND11_TYPE(MyReturnType<T1, T2>),
                  PYBIND11_TYPE(Class<T3, T4>), func)
```

`PYBIND11_MAKE_OPAQUE`宏不需要上述解决方案。

7.2 全局解释器锁（GIL）

在Python中调用C++函数时，默认会持有GIL。`gil_scoped_release`和`gil_scoped_acquire`可以方便地在函数体中释放和获取GIL。这样长时间运行的C++代码可以通过Python线程实现并行化。示例如下：

```
class PyAnimal : public Animal {
public:
    /* Inherit the constructors */
    using Animal::Animal;

    /* Trampoline (need one for each virtual function) */
    std::string go(int n_times) {
        /* Acquire GIL before calling Python code */
        py::gil_scoped_acquire acquire;

        PYBIND11_OVERRIDE_PURE(
            std::string, /* Return type */
            Animal,      /* Parent class */
            go,          /* Name of function */
            n_times      /* Argument(s) */
        );
    }
};

PYBIND11_MODULE(example, m) {
    py::class_<Animal, PyAnimal> animal(m, "Animal");
    animal
        .def(py::init<>())
        .def("go", &Animal::go);

    py::class_<Dog>(m, "Dog", animal)
        .def(py::init<>());

    m.def("call_go", [](Animal *animal) -> std::string {
        /* Release GIL before calling into (potentially long-running) C++
code */
        py::gil_scoped_release release;
        return call_go(animal);
    });
}
```

我们可以使用`call_guard`策略来简化`call_go`的封装：

```
m.def("call_go", &call_go, py::call_guard<py::gil_scoped_release>());
```

mdbook-mermaid

快速上手Mermaid流程图

本文主要介绍了如何快速上手 `Mermaid` 流程图,不用贴图上传也不用拖拉点拽绘制,基于源码实时渲染流程图,操作简单易上手,广泛被集成于主流编辑器,包括 `markdown` 写作环境.

通过本节内容你将学习到以下主要内容:

- 了解什么是流程图以及`Mermaid`流程图;
- 掌握并能记住如何绘制`Mermaid`流程图;
- 了解 `Gitbook` 写作环境的相关集成插件.

什么是Mermaid流程图

关键词

```
- 项目地址
- 在线编辑
- 官方文档
```

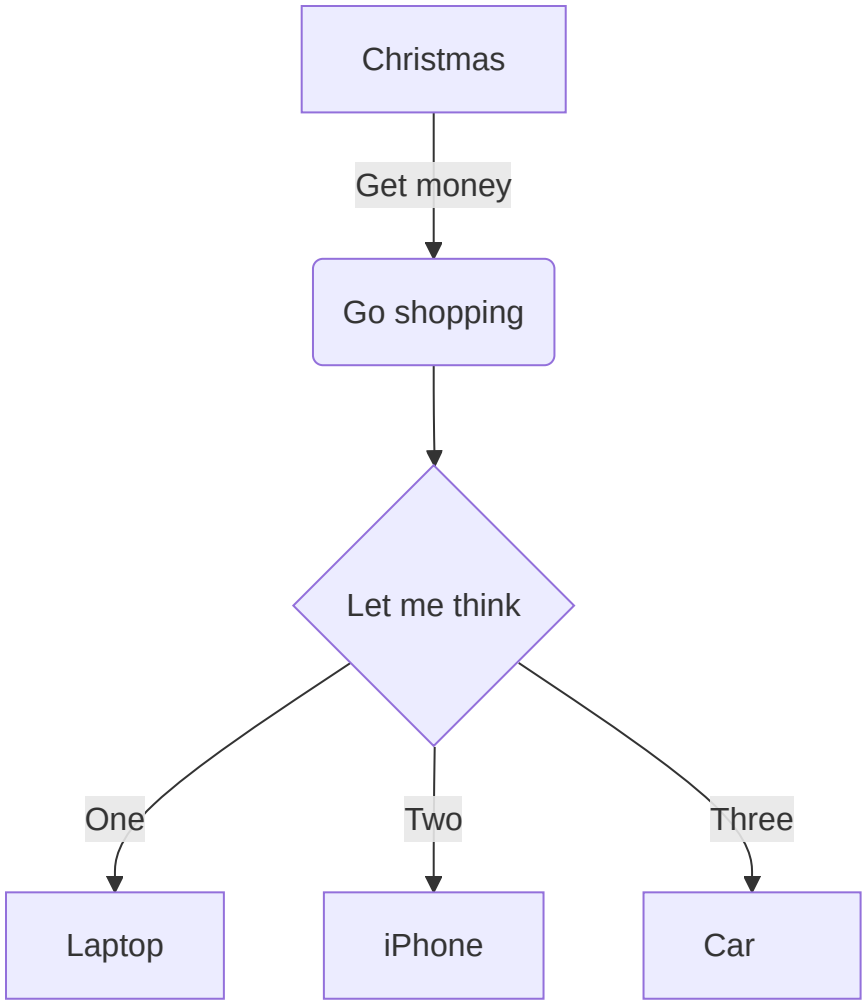
千言万语不如一张图,使用图形展示事物处理流程的图形称之为**流程图**.

`Mermaid`是一个基于 `Javascript` 的图解和制图工具.它基于 `markdown` 语法来简化和加速生成流程图的过程,也不止于生成流程图.

源码

```
graph TD
  A[Christmas] -->|Get money| B(Go shopping)
  B --> C{Let me think}
  C -->|One| D[Laptop]
  C -->|Two| E[iPhone]
  C -->|Three| F[fa:fa-car Car]
```

效果



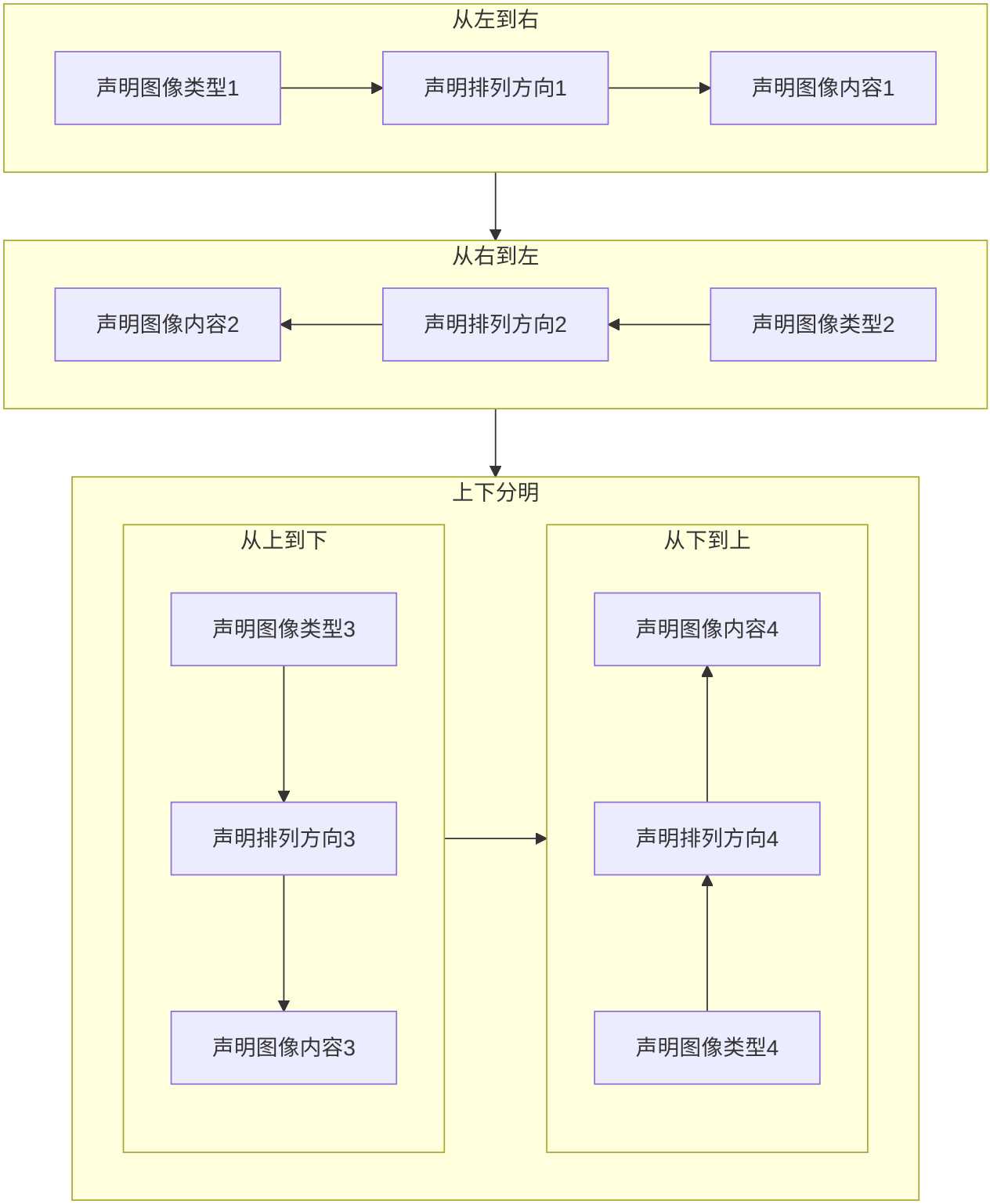
- 项目地址: https://github.com/mermaid-js/mermaid
- 在线编辑: https://mermaidjs.github.io/mermaid-live-editor/
- 官方文档: https://mermaid-js.github.io/mermaid/#/flowchart
- 官方参考: https://mermaid.nodejs.cn/syntax/flowchart.html

Mermaid流程图快速入门

布局方向

关键词

```
+ TB
+ BT
+ LR
+ RL
```


从左到右
声明图像类型1
声明排列方向1
声明图像内容1
从右到左
声明图像内容2
声明排列方向2
声明图像类型2
上下分明
从上到下
声明图像类型3
声明排列方向3
声明图像内容3
从下到上
声明图像内容4
声明排列方向4
声明图像类型4

流程图布局方向,由四种基本方向组成,分别是英文单词: `top` (上), `bottom` (下), `left` (左)和 `right` (右).其中可选值: `TB` (从上到下), `BT` (从下到上), `LR` (从左往右)和 `RL` (从右往左)四种.

> **核心**: 仅支持上下左右四个垂直方向,是英文单词首字母大写缩写.

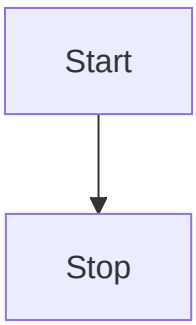
- TB

> 从上到下: from **T**op to **B**ottom

源码

```
graph TB
    Start --> Stop
```

效果



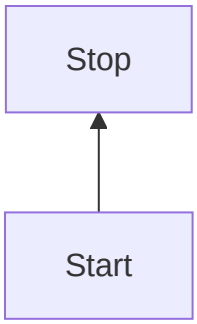
- BT

> 从下到上: from **B**ottom to **T**op

源码

```
graph BT
    Start --> Stop
```

效果



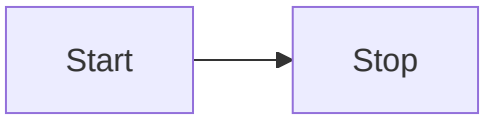
- LR

> 从左往右: from **L**eft to **R**ight

源码

```
graph LR
    Start --> Stop
```

效果



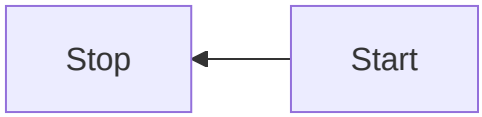
- RL

> 从右往左: from **R**ight to **L**eft

源码

```
graph RL
    Start --> Stop
```

效果



形状

关键词

```
- 节点形状
    + [矩形]
        - [[暂不支持]]
        - [(圆柱)]
        - [{暂不支持}]
        - [/平行四边形/]
        - [\平行四边形\]
        - [/梯形\]
        - [\梯形/]
    + (圆角矩形)
        - ((圆形))
        - ([体育场])
        - ({暂不支持})
    + {菱形}
        - {{六边形}}
        - {[暂不支持]}
        - {(暂不支持)}
    + >不对称矩形]
```

流程图节点形状,默认支持矩形和圆两种基本形状,包括基本形状的简单变体,支持嵌套组合形式,其中 `[]` 表示矩形, `()` 表示圆弧, `{}` 表示尖角(窃以为 `<>` 更适合)等等.

> **核心**: 最外层代表主形状,内层辅助修饰.

一次性节点

一次性节点,默认表现为矩形节点,其文本内容直接显示 `id` 的值,适合后续不会出现多次引用的情况.

> `id` 建议直接写成有意义的文本描述而不是当成唯一标识.

源码

```
graph TD
    id
```

效果

可重复节点

可重复节点,指定节点形状,其文本内容不再是 `id` 的值而是 `<node shape>` 的值,适合后续出现多次引用相同节点的情况.

> `id` 代表节点的唯一标识,当前节点的文本描述由 `<node shape>` 的值指定,建议 `id` 写成有意义的唯一标识.

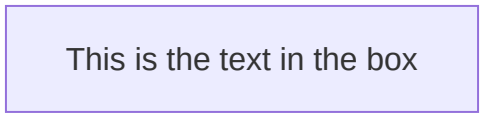
- 矩形

一般格式: `[node description]` , `[]` 中括号表示节点是**矩形**形状, `node description` 是节点的描述文本.

源码

```
graph LR
    id1[This is the text in the box]
```

效果



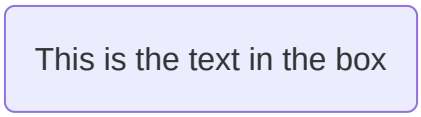
- 圆角矩形

一般格式: `(node description)` , `()` 小括号表示节点是**圆角矩形**形状, `node description` 是节点的描述文本.

源码

```
graph LR
    id1(This is the text in the box)
```

效果



- 体育场

一般格式: `([node description])` , `()` 小括号嵌套 `[]` 中括号表示节点是大弧度的圆角矩形形状,也就是**体育场**形状, `node description` 是节点的描述文本.

源码

```
graph LR
    id1([This is the text in the box])
```

效果

- 圆柱

一般格式: `[(node description)]` , `[]` 中括号嵌套 `()` 小括号表示节点是**圆柱**形状, `node description` 是节点的描述文本.

源码

```
graph LR
    id1[(Database)]
```

效果

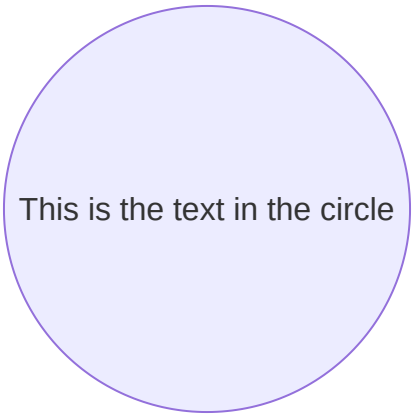
- 圆形

一般格式: `((node description))` , `()` 小括号嵌套 `()` 小括号表示节点是**圆形**形状, `node description` 是节点的描述文本.

源码

```
graph LR
    id1((This is the text in the circle))
```

效果



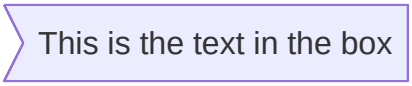
- 不对称矩形

一般格式: `>node description]` ,左边是右尖括号 `>` ,右边是右中括号 `]` 表示**不对称矩形**形状, `node description` 是节点的描述文本.

源码

```
graph LR
    id1>This is the text in the box]
```

效果



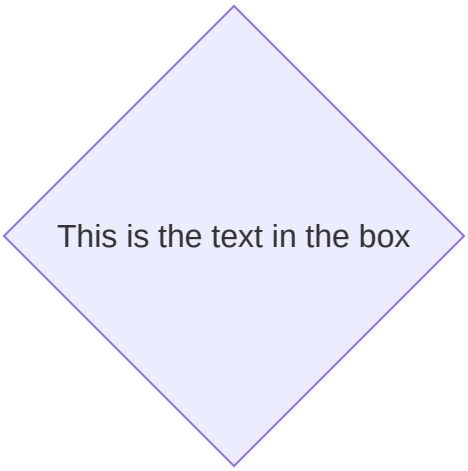
- 菱形

一般格式: `{node description}` , `{}` 大括号表示**菱形**形状, `node description` 是节点的描述文本.

源码

```
graph LR
    id1{This is the text in the box}
```

效果



- 六角形

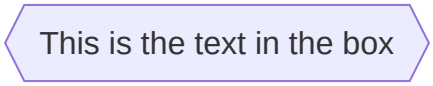
一般格式: `{{node description}}` , `{}` 大括号嵌套 `{}` 大括号表示**六角形**形状, `node description` 是节点的描述文本.

源码

```
graph LR
    id1\{\{This is the text in the box\}\}
```

> `Gitbook` 语法中双大括号 `{}` 表示特殊意义,上述源码只能转义处理,实际上并不需要 `\` 进行转义.

效果



- 平行四边形

> 一般格式: `[/node description/]` , `[]` 中括号嵌套 `//` 左斜杠表示**左斜平行四边形**形状, `node description` 是节点的描述文本.

源码

```
graph TD
    id1[/This is the text in the box/]
```

效果

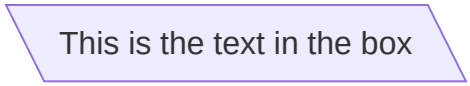
- 平行四边形

> 一般格式: `[\node description\]` , `[]` 中括号嵌套 `\\` 右斜杠表示**右斜平行四边形**形状, `node description` 是节点的描述文本.

源码

```
graph TD
    id1[\This is the text in the box\]
```

效果



- 梯形

一般格式: `[/node description\]` , `[]` 中括号嵌套 `/\` 左右斜杠表示**上短下长梯形**形状, `node description` 是节点的描述文本.

源码

```
graph TD
    A[/Christmas\]
```

效果

- 另一种梯形

一般格式: `[\node description/]` , `[]` 中括号嵌套 `\/` 右左斜杠表示**上长下短梯形**形状, `node description` 是节点的描述文本.

源码

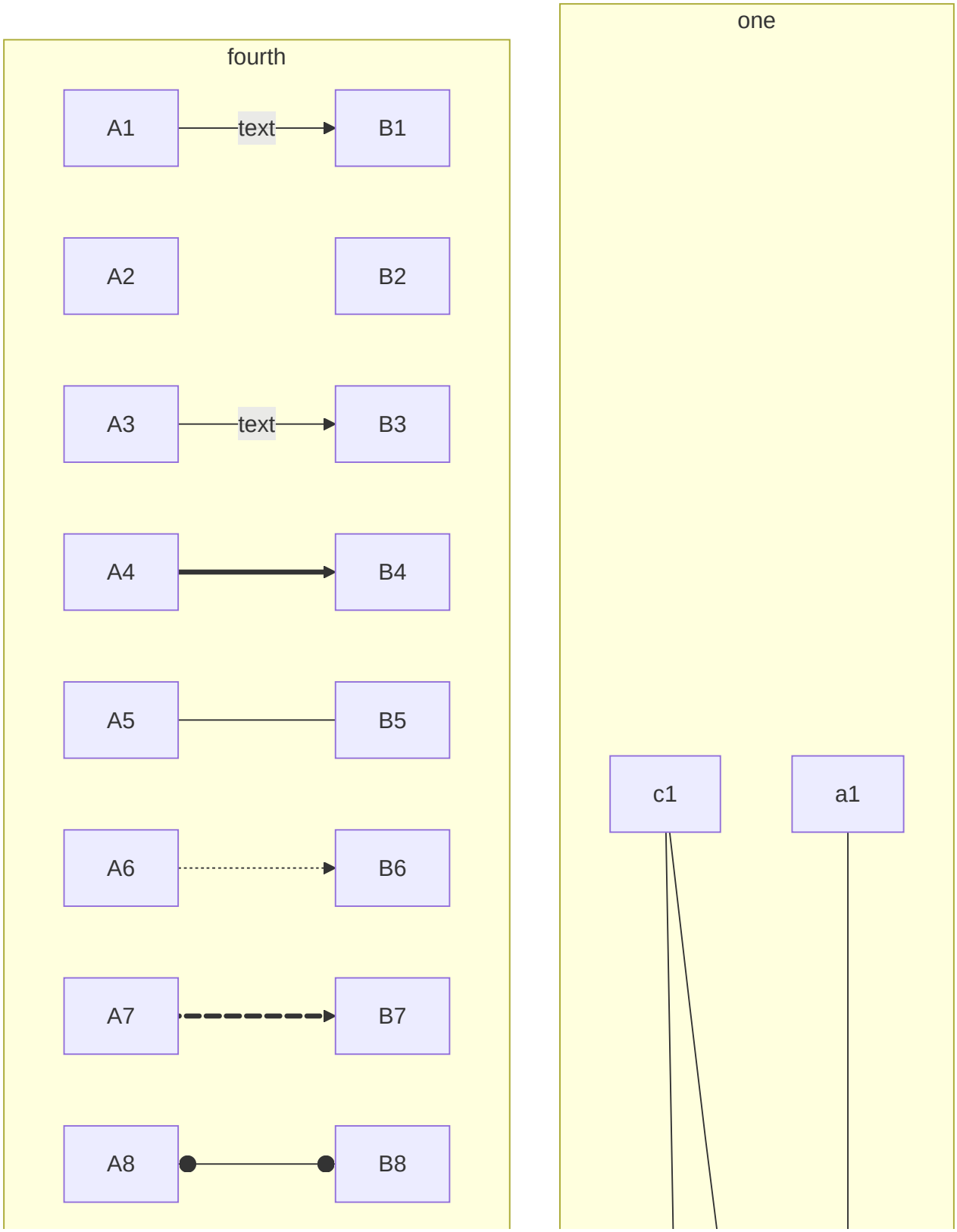
```
graph TD
    B[\Go shopping/]
```

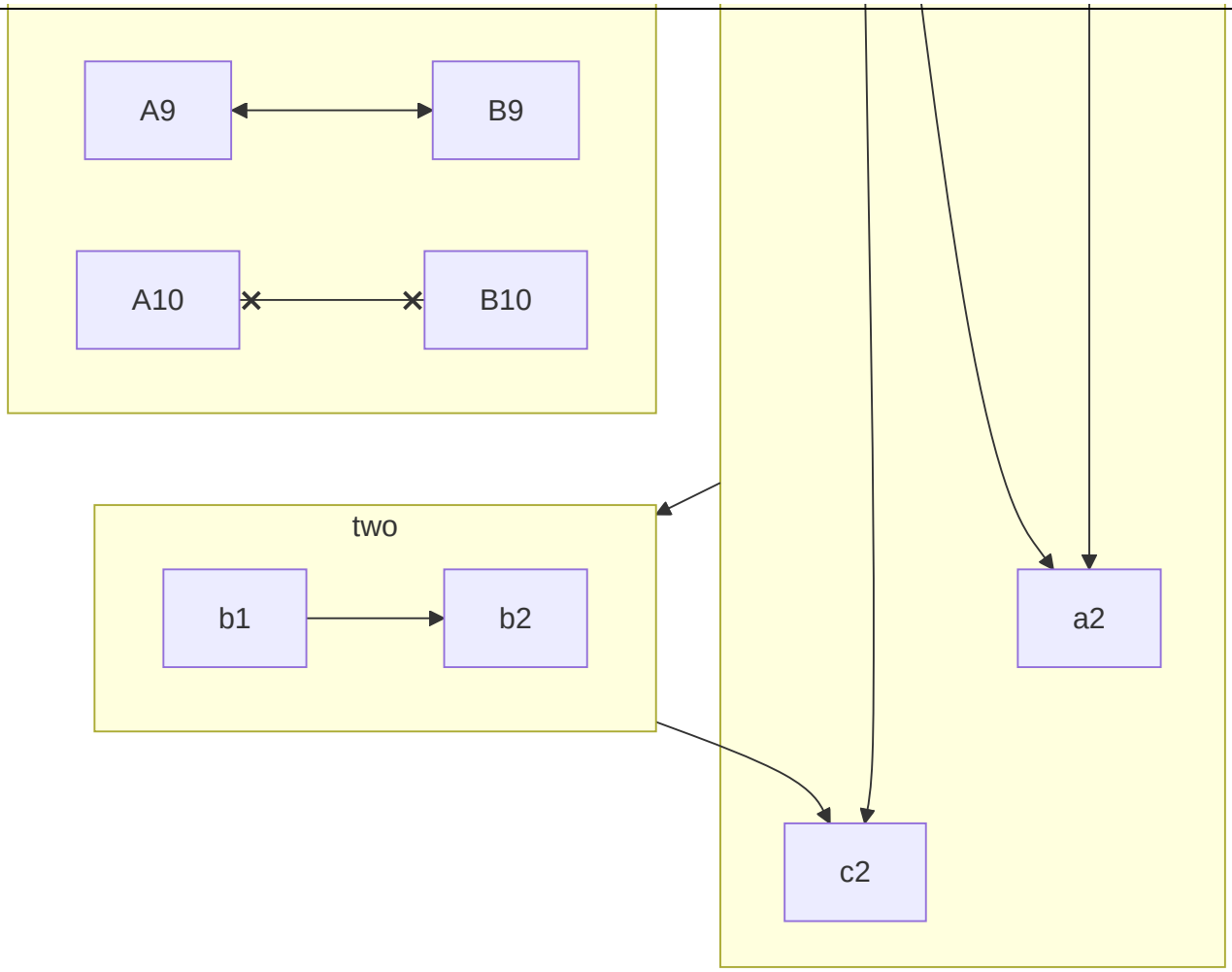
效果

连接线

关键词

```
+ 实线/虚线
    - --
    - -.
+ 有箭头/无箭头
    - >
    - -
+ 有描述/无描述
    - 实线
        + --描述文字
        + |描述文字|
    - 虚线
        + -.描述文字
        + |描述文字|
+ 加粗
    - ==
+ 组合形式
    - -->
    - --- 
    - -.->
    - -.-
    - 有描述实线有箭头
        + --描述文字-->
        + -->|描述文字|
    - 有描述实线无箭头
        + --描述文字---
        + ---|描述文字|
    - 有描述虚线有箭头
        + -.描述文字-.->
        + -.->|描述文字|
    - 有描述虚线无箭头
        + -.描述文字-.-
        + -.-|描述文字|
    - ==>
    - ===
    - 有描述加粗实线有箭头(2)
        + ==描述文字==>
        + ==>|描述文字|
    - 有描述加粗实线无箭头(2)
        + ==描述文字===
        + ===|描述文字|
```


fourth
A1
text
B1
A2
B2
A3
text
B3
A4
B4
A5
B5
A6
B6
A7
B7
A8
B8
one
c1
a1



流程图连接线样式,支持实线和虚线以及有箭头样式和无箭头样式,除此之外还支持添加连接线描述文字,其中 `--` 代表实线,实线中间多一点 `-.-` 代表虚线,添加箭头用右尖括号 `>` ,没有箭头继续用短横线 `-`.

> **核心**: 先实线再虚线,先有箭头再去箭头,左边位置添加描述文字需要区分实现还是虚线,右边位置添加描述文字格式一致.

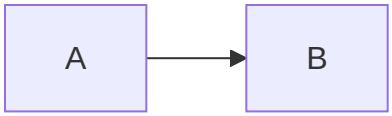
- 有箭头无描述实线

> 一般格式: `-->` ,其中 `--` 表示实线, `>` 表示有箭头.

源码

```
graph LR
    A-->B
```

效果



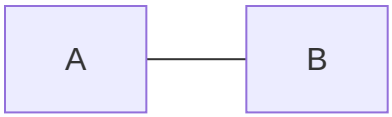
- 无箭头实线

> 一般格式: `---` ,其中 `--` 表示实线, `-` 表示无箭头.

源码

```
graph LR
    A --- B
```

效果



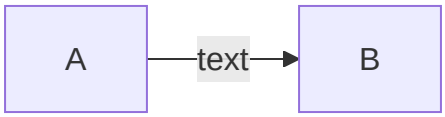
- 带描述的有箭头实线

> 一般格式: `--connection line description-->` ,其中左边的 `--` 添加到**实线左边位置**,右边的 `-->` 表示**带箭头的实线**.

源码

```
graph LR
    A-- text -->B
```

效果

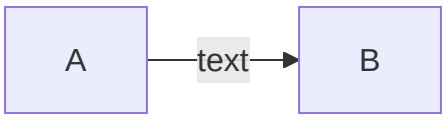


一般格式: `|connection line description|` ,其中 `||` 添加到**连接线右边位置**.

源码

```
graph LR
    A-->|text|B
```

效果



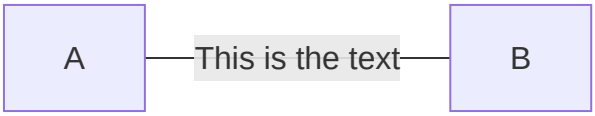
- 带描述的无箭头实线

一般格式: `--connection line description` ,其中左边的 `--` 添加到**实线左边位置**, 右边的 `---` 表示**不带箭头的实线**.

源码

```
graph LR
    A-- This is the text ---B
```

效果

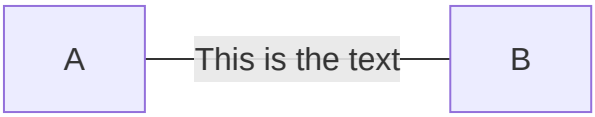


一般格式: `|connection line description|` ,其中 `||` 添加到**连接线右边位置**.

源码

```
graph LR
    A---|This is the text|B
```

效果



- 有箭头虚线

一般格式: `-.connection line description.->` ,其中左边的 `-.` 添加到**虚线左边位置**,右边的 `.->` 表示**带箭头的虚线**.

源码

```
graph LR
   A-. text .-> B
```

效果

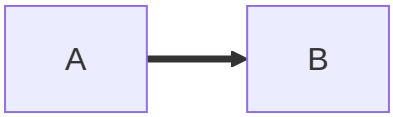
- 有箭头加粗实线

一般格式: `==>` ,表示加粗实线.

源码

```
graph LR
   A ==> B
```

效果



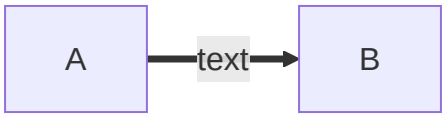
- 带描述的有箭头加粗实线

一般格式: `==connection line description` ,左边的 `==` 添加到加粗实现左边,右边的 `==>` 代表加粗实线.

源码

```
graph LR
   A == text ==> B
```

效果



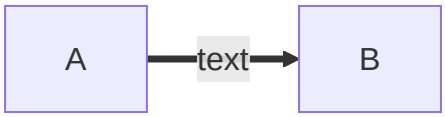
- 带描述的有箭头加粗实线

一般格式: `|connection line description|` ,其中 `||` 添加到**连接线右边位置**.

源码

```
graph LR
   A ==>|text| B
```

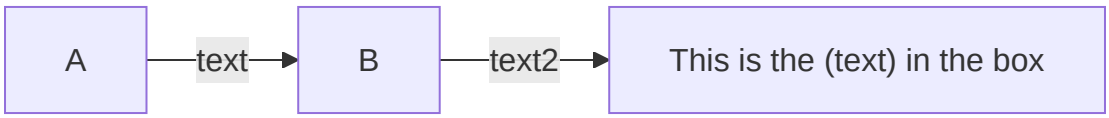
效果



高级用法

关键词

```
+ -->-->
+ &
+ ""
+ %%
+ subgraph
```



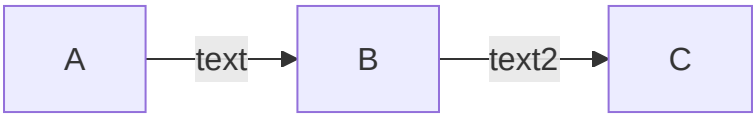
- 多节点链式连接

源码

支持链式连接方式, `A-->B-->C` 等价于 `A-->B` 和 `B-->C` 形式.

```
graph LR
   A -- text --> B -- text2 --> C
```

效果



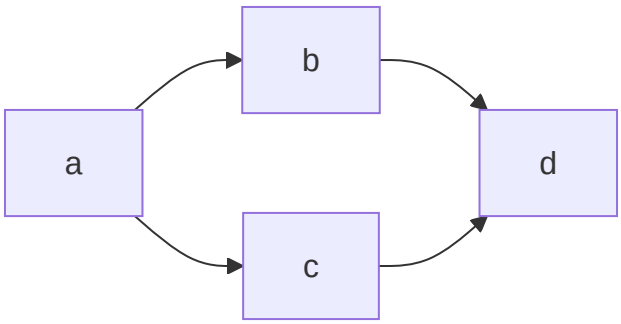
- 多节点共同连接

支持共同连接方式, `A-->B & C` 等价于 `A-->B` 和 `A-->C` 形式.

源码

```
graph LR
   a --> b & c--> d
```

效果



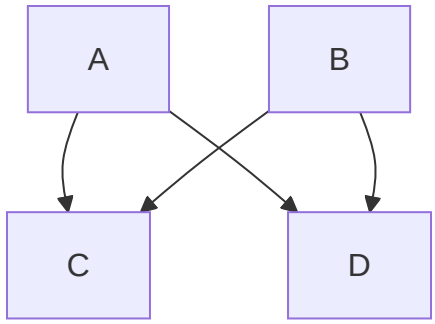
- 多节点相互连接

多节点共同连接的变体形式, `A & B --> C & D` 等价于 `A-->C` , `A-->D` , `B-->C` 和 `B-->D` 四种组合形式.

源码

```
graph TB
    A & B--> C & D
```

效果



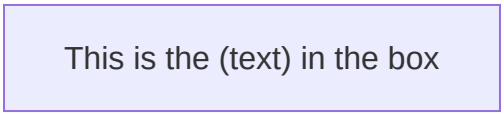
- 双引号包裹特殊字符

连接线描述文字存在特殊字符使用双引号 `""` 包裹处理,如遇到 `[]` 和 `()` 以及 `{}` 等特殊字符情况.

源码

```
graph LR
    id1["This is the (text) in the box"]
```

效果



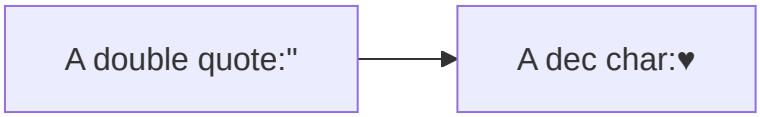
- 双引号包裹转义字符

支持 `Html` 转移字符

源码

```
graph LR
    A["A double quote:#quot;"] -->B["A dec char:#9829;"]
```

效果



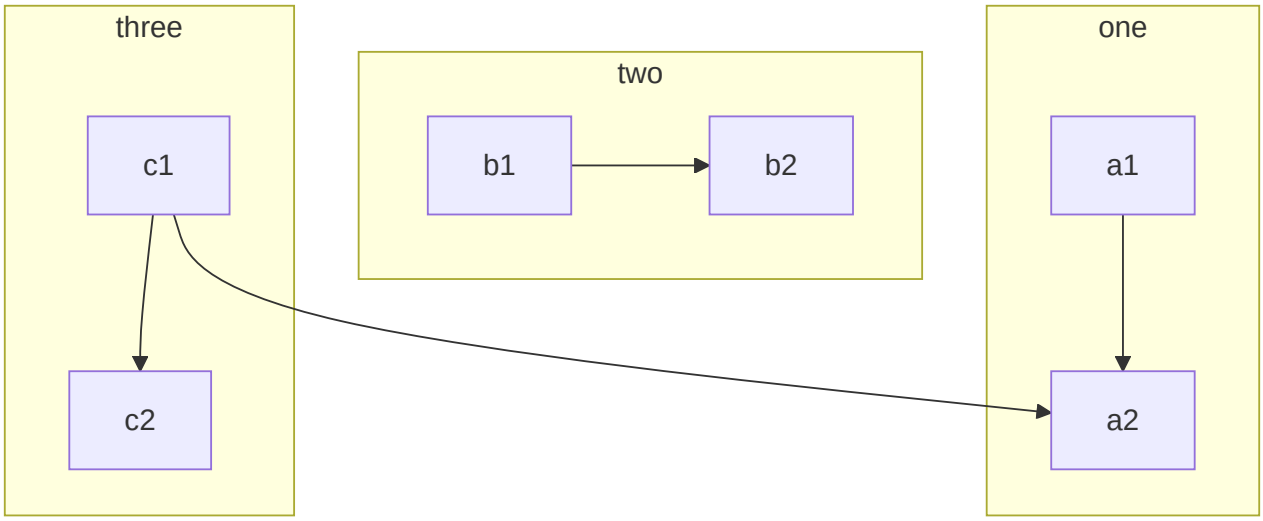
- 嵌套子流程图

定义

```
subgraph title
    graph definition
end
```

示例

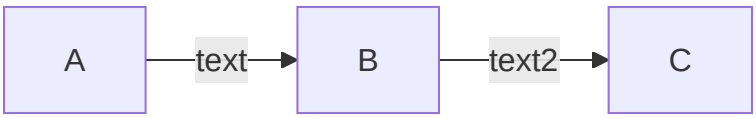
```
graph TB
    c1-->a2
    subgraph one
    a1-->a2
    end
    subgraph two
    b1-->b2
    end
    subgraph three
    c1-->c2
    end
```



- 注释语法

注释以 `%%` 开头并且独占一行.

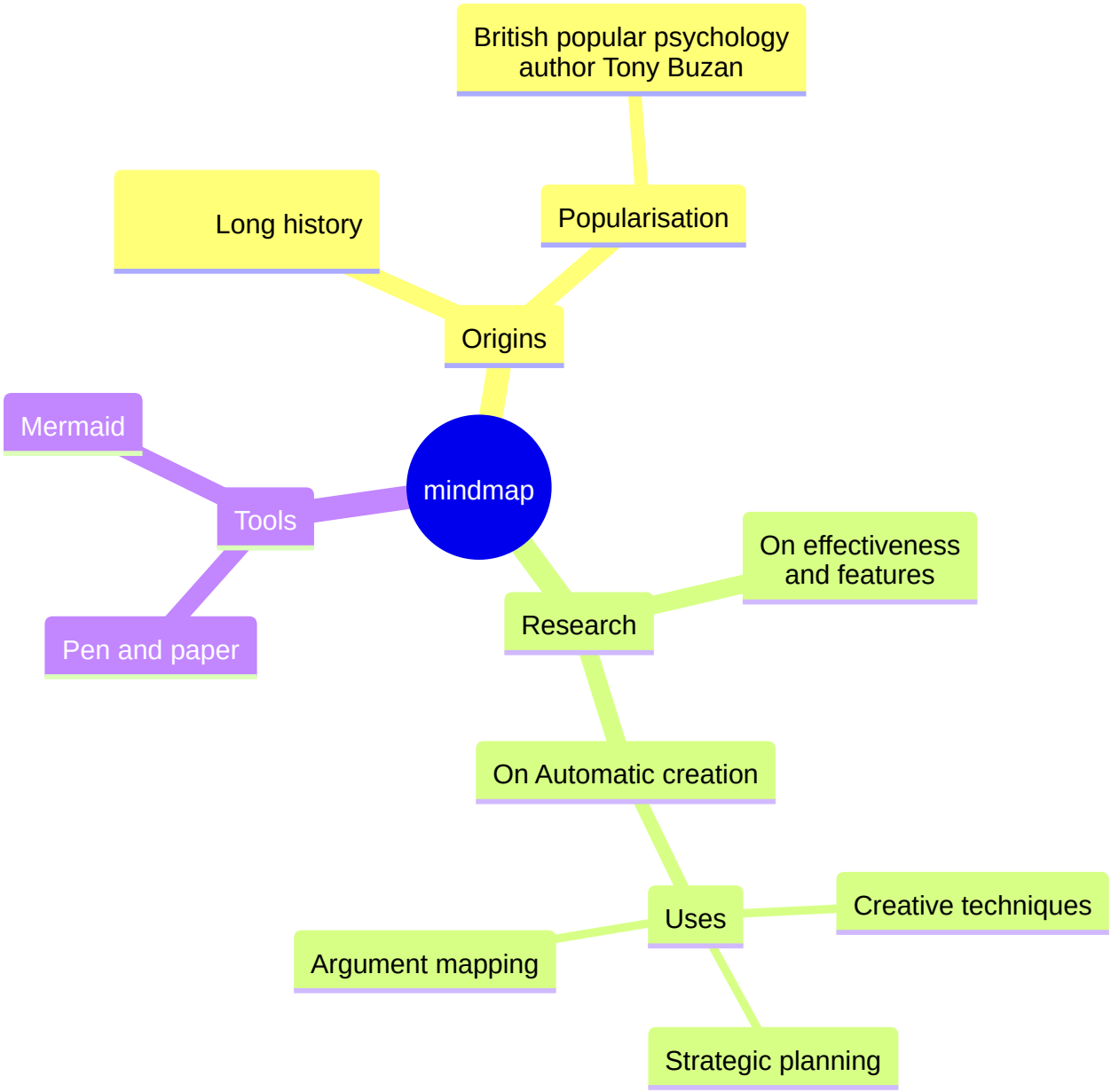
```
graph LR
%% this is a comment A -- text --> B{node}
   A -- text --> B -- text2 --> C
```



快速入门流程图回顾总结

关键词

```
- 英文单词缩写
- 几何化形状
- 有限语法
```



`Mermaid` 是一款开源的制图工具,可使用 `Markdown` 语法绘制流程图,支持更改流程图节点形状,添加描述文字以及更改连接线样式等等.

英文单词缩写

四种布局方向的值是英文单词首字母大写缩写形式,默认仅支持垂直方向.

中文	英文	示例
图解	graph	`graph` 流程图类型标识
子图	subgraph	`subgraph` 嵌套子流程图标识
上	top	`TB` 或 `BT` ,从上到下或从下到上的布局方向
下	bottom	`BT` 或 `TB` ,从下到上或从上到下的布局方向
左	left	`LR` 或 `RL` ,从左往右或从右往左的布局方向
右	right	`RL` 或 `LR` ,从右往左或从左往右的布局方向

几何化形状

键盘符号形象化几何形状,组合形式表示形状的叠加,其中最外层符号是主形状,嵌套符号是辅助形状.

- 基本单元

表示法	含义	类型	备注
`[]`	矩形	节点形状	支持
`()`	圆角矩形	节点形状	支持
`{}`	菱形	节点形状	支持
`<>`	菱形	节点形状	不支持
`--`	实线	连接线样式	支持
`-.`	虚线	连接线样式	支持

表示法	含义	类型	备注
==	加粗实线	连接线样式	支持
=:	加粗虚线	连接线样式	不支持
>	有箭头	连接线样式	支持
-	无箭头	连接线样式	支持
双竖线	右边连接线描述文字	连接线描述文字	支持
--	左边实线连接线描述文字	连接线描述文字	支持
-.	左边虚线连接线描述文字	连接线描述文字	支持
==	左边加粗实线连接线描述文字	连接线描述文字	支持
=:	左边加粗虚线连接线描述文字	连接线描述文字	不支持

- 组合单元

表示法	含义	类型	备注
[[]]	正方形	节点形状	不支持
[()]	圆柱体	节点形状	支持
[{}]	棱柱体	节点形状	不支持
(())	圆形	节点形状	支持
([])	体育场	节点形状	支持
({})	圆弧	节点形状	不支持
双大括号	六边形	节点形状	支持
{[]}	正多边形	节点形状	不支持
{()}	圆弧	节点形状	不支持

表示法	含义	类型	备注
`--->`	实线带箭头	连接线样式	支持
`---`	实线无箭头	连接线样式	支持
`-.>`	虚线带箭头	连接线样式	不支持
`-.->`	虚线带箭头	连接线样式	支持
`.->`	虚线带箭头	连接线样式	支持
`-.-`	虚线无箭头	连接线样式	支持
`.-`	虚线无箭头	连接线样式	支持
`==>`	加粗实线带箭头	连接线样式	支持
`===`	加粗实线无箭头	连接线样式	支持
`=:>`	加粗虚线带箭头	连接线样式	不支持
`=:=>`	加粗虚线带箭头	连接线样式	不支持
`=:=`	加粗虚线无箭头	连接线样式	不支持
`:=`	加粗虚线无箭头	连接线样式	不支持
双竖线	右边连接线描述文字	连接线描述文字	支持

表示法	含义	类型	备注
`--connection line description-->`	左边实线带箭头连接线描述文字	连接线描述文字	支持
`-.connection line description-.->`	左边虚线带箭头连接线描述文字	连接线描述文字	支持
`--connection line description---`	左边实线无箭头连接线描述文字	连接线描述文字	支持
`-.connection line description-.-`	左边虚线无箭头连接线描述文字	连接线描述文字	支持
`==connection line description==>`	左边加粗实线带箭头连接线描述文字	连接线描述文字	支持
`=:connection line description=:=>`	左边加粗虚线带箭头连接线描述文字	连接线描述文字	不支持
`==connection line description===`	左边加粗实线无箭头连接线描述文字	连接线描述文字	支持
`=:connection line description=:=`	左边加粗虚线无箭头连接线描述文字	连接线描述文字	不支持

有限语法

不论是节点形状还是连接线样式,语法支持是有限的,并不是随意组合的叠加状态,也可能随着后续更新会支持更多,一切以官方文档为主.

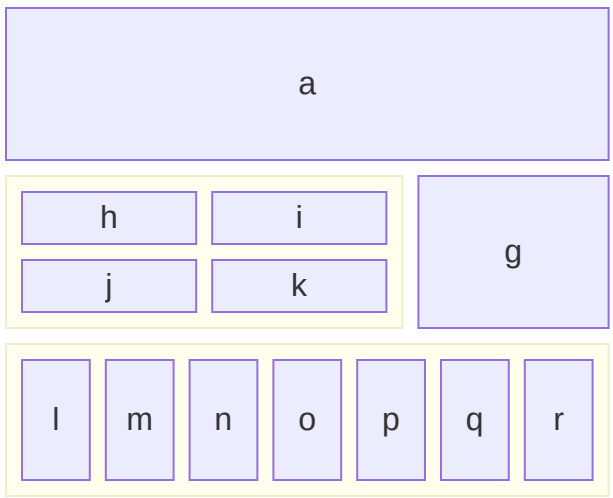
除了提供最基础的操作节点的能力之外,还可以根据 `JS` 和 `CSS` 相关知识高度自定义流程图行为表现,具体可参考官方文档.

> 官方文档: https://mermaid-js.github.io/mermaid/#/flowchart?id=styling-and-classes

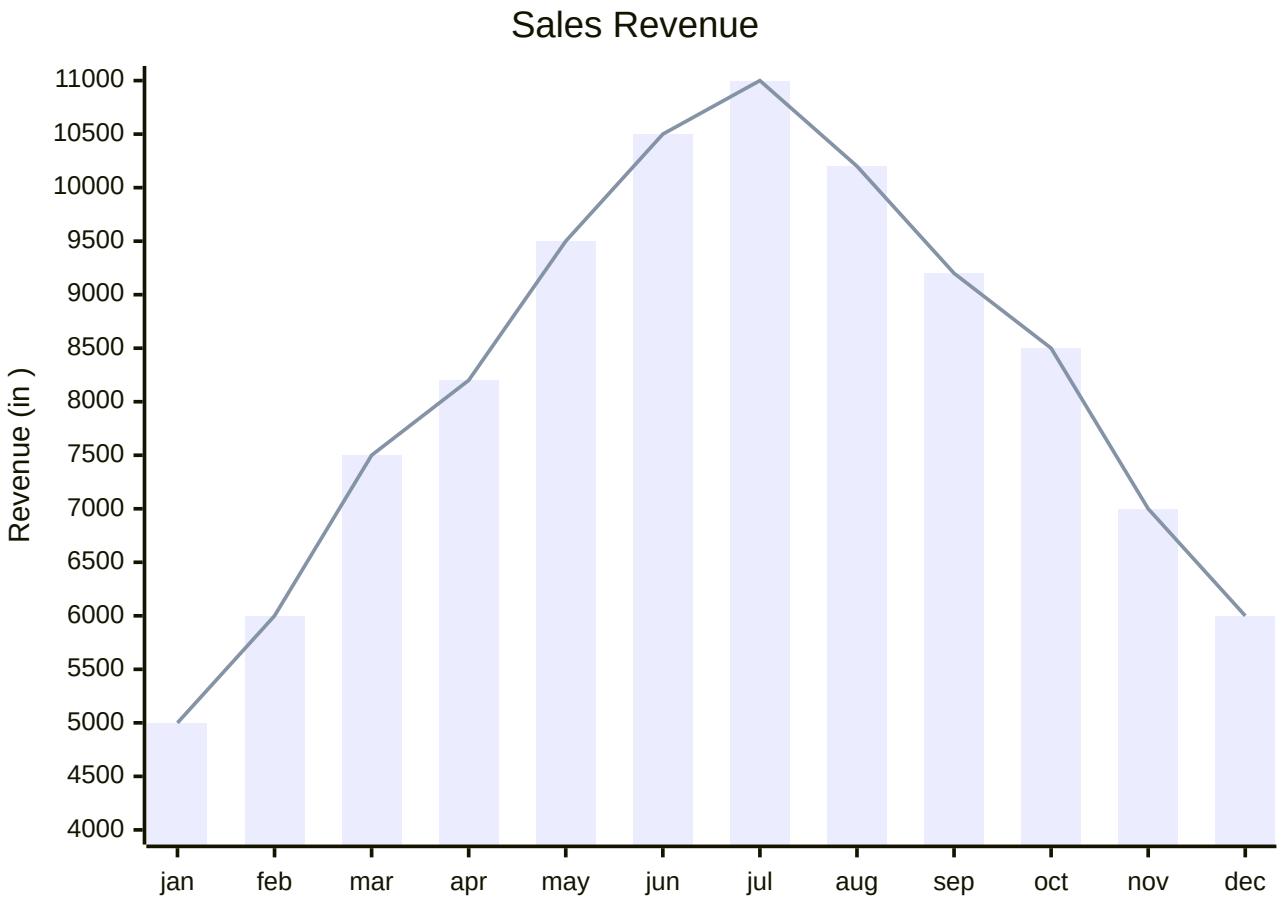
- 交互能力 Interaction : https://mermaid-js.github.io/mermaid/#/flowchart?id=interaction
- 外观样式 Styling and classes : https://mermaid-js.github.io/mermaid/#/flowchart?id=interaction
- 字体支持 Basic support for fontawesome: https://mermaid-js.github.io/mermaid/#/flowchart?id=basic-support-for-fontawesome
- 空格分隔 https://mermaid-js.github.io/mermaid/#/flowchart?id=graph-declarations-with-spaces-between-vertices-and-link-and-without-semicolon

其他图

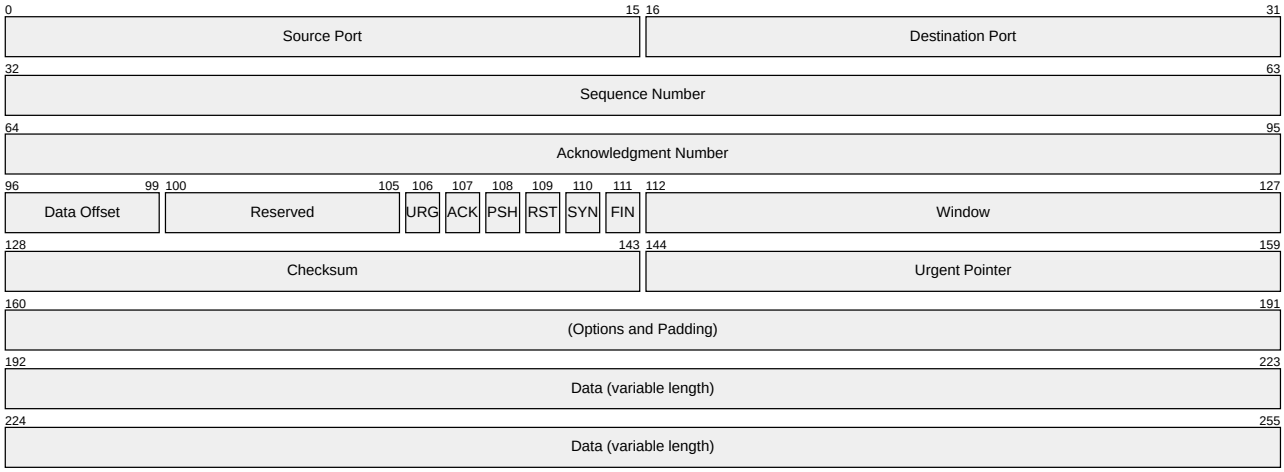
框图



XY图


Sales Revenue
Revenue (in)
11000
10500
10000
9500
9000
8500
8000
7500
7000
6500
6000
5500
5000
4500
4000
jan
feb
mar
apr
may
jun
jul
aug
sep
oct
nov
dec

数据包



TCP Packet

mdbook-admonish

The following admonishments are implemented by the mdbook-admonish plugin and are automatically themed to match Catppuccin.

Directives

All supported directives are listed below.

`note`

> **Note**
>
> Rust is a multi-paradigm, general-purpose programming language designed for performance and safety, especially safe concurrency.

`abstract`, `summary`, `tldr`

> **Abstract**
>
> Rust is a multi-paradigm, general-purpose programming language designed for performance and safety, especially safe concurrency.

`info`, `todo`

> **Info**
>
> Rust is a multi-paradigm, general-purpose programming language designed for performance and safety, especially safe concurrency.

`tip`, `hint`, `important`

> **Tip**
>
> Rust is a multi-paradigm, general-purpose programming language designed for performance and safety, especially safe concurrency.

`success`, `check`, `done`

> **Success**
>
> Rust is a multi-paradigm, general-purpose programming language designed for performance and safety, especially safe concurrency.

`question`, `help`, `faq`

> **Question**
>
> Rust is a multi-paradigm, general-purpose programming language designed for performance and safety, especially safe concurrency.

`warning`, `caution`, `attention`

Warning

Rust is a multi-paradigm, general-purpose programming language designed for performance and safety, especially safe concurrency.

`failure`, `fail`, `missing`

Failure

Rust is a multi-paradigm, general-purpose programming language designed for performance and safety, especially safe concurrency.

`danger`, `error`

Danger

Rust is a multi-paradigm, general-purpose programming language designed for performance and safety, especially safe concurrency.

`bug`

Bug

Rust is a multi-paradigm, general-purpose programming language designed for performance and safety, especially safe concurrency.

`example`

Example

Rust is a multi-paradigm, general-purpose programming language designed for performance and safety, especially safe concurrency.

`quote`, `cite`

Quote

Rust is a multi-paradigm, general-purpose programming language designed for performance and safety, especially safe concurrency.

Bienvenue sur notre site de développement 3D !

Bienvenue sur notre site dédié au développement 3D. Ici, vous trouverez des ressources, des tutoriels et des informations utiles pour vous lancer dans le monde passionnant de la 3D.

Info

A beautifully styled message.

Un example

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla et euismod nulla. Curabitur feugiat, tortor non consequat finibus, justo purus auctor massa, nec semper lorem quam in massa.

Une note

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla et euismod nulla. Curabitur feugiat, tortor non consequat finibus, justo purus auctor massa, nec semper lorem quam in massa.

Un warning

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla et euismod nulla. Curabitur feugiat, tortor non consequat finibus, justo purus auctor massa, nec semper lorem quam in massa.

Collapsing note

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla et euismod nulla. Curabitur feugiat, tortor non consequat finibus, justo purus auctor massa, nec semper lorem quam in massa.

Le javascript c'est yolo préférez Typescript

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla et euismod nulla. Curabitur feugiat, tortor non consequat finibus, justo purus auctor massa, nec semper lorem quam in massa.

Referencing and *dereferencing*

The opposite of *referencing* by using `&` is *dereferencing*, which is accomplished with the dereference operator, `*`.

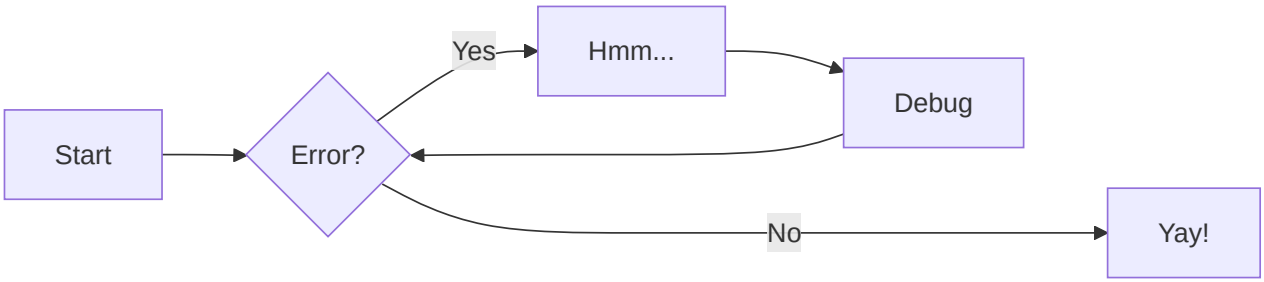
Bug

This syntax won't work in Python 3:

```
print "Hello, world!"
```

À propos de nous

Nous sommes une équipe passionnée par la 3D et nous avons pour mission de partager nos connaissances avec la communauté. Vous trouverez ici des articles, des exemples de code et des démonstrations pour vous aider à démarrer votre voyage dans le développement 3D.



Pour commencer

Si vous êtes nouveau dans le domaine de la 3D, ne vous inquiétez pas ! Notre page "Getting Started" vous guidera à travers les étapes essentielles pour démarrer rapidement.

Restons en contact

N’hésitez pas à nous suivre sur les réseaux sociaux pour rester à jour avec nos dernières publications et annonces. Si vous avez des questions ou des commentaires, n’hésitez pas à nous contacter !

Mizux

mdbook_mathjax

Section 1

Section 2

Chapter 2

Section 1

Section 2

Chapter 3

Section 1

Section 2

Chapter 4

Section 1

Section 2

Chapter 5

Section 1

Section 2

Chapter 6

Section 1

> **Note**
>
> Highlights information that users should take into account, even when skimming.

> **Tip**
>
> Optional information to help a user be more successful.

> **Important**
>
> Crucial information necessary for users to succeed.

> **Warning**
>
> Critical content demanding immediate user attention due to potential risks.

> **Caution**
>
> Negative potential consequences of an action.

Section 2

Here is an inline example, $\pi(\theta)$,

an equation,

$$\nabla f(x) \in \mathbb{R}^n,$$

and a regular $ symbol.

Define $f(x)$:

$$f(x) = x^2$$
$$x \in \mathbb{R}$$

```
graph TD
  A[ Anyone ] -->|Can help | B( Go to github.com/yuzutech/kroki )
  B --> C{ How to contribute? }
  C --> D[ Reporting bugs ]
  C --> E[ Sharing ideas ]
  C --> F[ Advocating ]
```

mdbook-katex

HTML:

$$e^{i\theta} = \cos\theta + i\sin\theta$$

$$\Rightarrow x + iy = re^{i\theta}$$

Markdown (requires `mdbook-katex`):

$$\oint_C f(x,y)\,\mathrm{d}A$$

Inspect element and use `Sources` tab (under `Debugger` on Firefox) to check that all CSS and fonts are properly loaded from GitHub pages instead of external CDN.

▼ Proof that $e^{ix} = \cos x + i\sin x$

$$\begin{aligned}
e^x &= \sum_{n=0}^{\infty} \frac{x^n}{n!} \implies e^{ix} = \sum_{n=0}^{\infty} \frac{(ix)^n}{n!} \\
\cos x &= \sum_{m=0}^{\infty} \frac{(-1)^m x^{2m}}{(2m)!} = \sum_{m=0}^{\infty} \frac{(ix)^{2m}}{(2m)!} \\
\sin x &= \sum_{s=0}^{\infty} \frac{(-1)^s x^{2s+1}}{(2s+1)!} = \sum_{s=0}^{\infty} \frac{(ix)^{2s+1}}{i(2s+1)!} \\
\cos x + i\sin x &= \sum_{l=0}^{\infty} \frac{(ix)^{2l}}{(2l)!} + \sum_{s=0}^{\infty} \frac{(ix)^{2s+1}}{(2s+1)!} = \sum_{n=0}^{\infty} \frac{(ix)^n}{n!} \\
&= e^{ix}
\end{aligned}$$

Fourier Transform:

$$f(t) = \int_{-\infty}^{\infty} F(\omega) i^{4t\omega} d\omega$$

$$F(\omega) = \int_{-\infty}^{\infty} f(t) i^{-4t\omega} dt$$

Pauli Matrices:

$$\sigma_1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

$$\sigma_2 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$$

$$\sigma_3 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

mdbook-whichlang

```c
#include <stdio.h>

int main(void) {
    printf("Hello World\n");
}
```

```cpp
#include <iostream>

int main()
{
   std::cout << "Hello World" << std::endl;
}
```



```javascript
console.log("Hello World");
```

```typescript
console.log("Hello World");
```

```
Console.log("Hello World")
```

```
h1 {
  color: blue;
}
```

```
set syntax=ruby
```

init.lua

```
print("Hello World")
```

```
print("Hello World")
```



```
echo "Hello World"
```

hello.zig

```
const std = @import("std");

pub fn main() !void {
    const stdout = std.io.getStdOut().writer();
    try stdout.print("Hello, {s}!\n", .{"world"});
}
```

```
package main

import "core:fmt"

main :: proc() {
        fmt.println("Hellope!")
}
```

WA hello.wat

```
(module
    (import "wasi_unstable" "fd_write"
        (func $fd_write (param i32 i32 i32 i32) (result i32))
    )

    (memory 1)
    (export "memory" (memory 0))

    (data (i32.const 0) "\08\00\00\00\0c\00\00\00Hello World\n")

    (func $main (export "_start")
        i32.const 1
        i32.const 0
        i32.const 1
        i32.const 20
        call $fd_write
        drop
    )
)
```

mdbook-langtabs

Some code

let's try with a simple example:

```
LogChannel(n"DEBUG", "hello world");
```

```
print("Hello, World!")
```

```
fn main() { println!("hello world"); }
```

```
print("Hello World")
```

```
#include <iostream>

int main() {
    std::cout << "Hello World!";
    return 0;
}
```

```
some:
  interesting:
    - property
```

```
{
  "some": { "interesting": ["property"] }
}
```



```
<some>
  <interesting>
    <property />
  </interesting>
</some>
```

contrary to code blocks, `inline` and `fenced` are left untouched.

it should also work when deeply nested:

1. outer:
    1. inner:

```
GameInstance
  .GetStatusEffectSystem(this.GetGame())
  .ApplyStatusEffect(
    this.GetEntityID(),
    t"BaseStatusEffect.SplinterAddicted",
    this.GetRecordID(),
    this.GetEntityID());
```

Hello World

Here's a simple "Hello World" example in different languages:

```rust
fn main() {
    println!("Hello, World!");
}
```

```python
def main():
    print("Hello, World!")

if __name__ == "__main__":
    main()
```



```javascript
function main() {
    console.log("Hello, World!");
}

main();
```

```go
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")
}
```

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Simple Function Example

Here's how you might define a function to calculate factorial in different languages:

```rust
fn factorial(n: u64) -> u64 {
    match n {
        0 | 1 => 1,
        _ => n * factorial(n - 1)
    }
}

fn main() {
    println!("5! = {}", factorial(5)); // Outputs: 5! = 120
}
```

```python
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(f"5! = {factorial(5)}")  # Outputs: 5! = 120
```

```javascript
function factorial(n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

console.log(`5! = ${factorial(5)}`);  // Outputs: 5! = 120
```



```go
package main

import "fmt"

func factorial(n uint64) uint64 {
    if n <= 1 {
        return 1
    }
    return n * factorial(n-1)
}

func main() {
    fmt.Printf("5! = %d\n", factorial(5))  // Outputs: 5! = 120
}
```

```java
public class FactorialExample {
    static long factorial(int n) {
        if (n <= 1) return 1;
        return n * factorial(n - 1);
    }

    public static void main(String[] args) {
        System.out.println("5! = " + factorial(5));  // Outputs: 5! = 120
    }
}
```

Data Structures Example

Here's how you might implement a simple stack in different languages:

```rust
struct Stack<T> {
    items: Vec<T>,
}

impl<T> Stack<T> {
    fn new() -> Self {
        Stack { items: Vec::new() }
    }

    fn push(&mut self, item: T) {
        self.items.push(item);
    }

    fn pop(&mut self) -> Option<T> {
        self.items.pop()
    }

    fn is_empty(&self) -> bool {
        self.items.is_empty()
    }
}

fn main() {
    let mut stack = Stack::new();
    stack.push(1);
    stack.push(2);
    stack.push(3);

    while let Some(item) = stack.pop() {
        println!("Popped: {}", item);
    }
}
```

```python
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        return None

    def is_empty(self):
        return len(self.items) == 0

# Usage
stack = Stack()
stack.push(1)
stack.push(2)
stack.push(3)

while not stack.is_empty():
    print(f"Popped: {stack.pop()}")
```

```javascript
```

```
class Stack {
    constructor() {
        this.items = [];
    }

    push(item) {
        this.items.push(item);
    }

    pop() {
        if (!this.isEmpty()) {
            return this.items.pop();
        }
        return null;
    }

    isEmpty() {
        return this.items.length === 0;
    }
}

// Usage
const stack = new Stack();
stack.push(1);
stack.push(2);
stack.push(3);

while (!stack.isEmpty()) {
    console.log(`Popped: ${stack.pop()}`);
}
```

```
GO
```

```go
package main

import "fmt"

type Stack struct {
    items []int
}

func (s *Stack) Push(item int) {
    s.items = append(s.items, item)
}

func (s *Stack) Pop() (int, bool) {
    if s.IsEmpty() {
        return 0, false
    }

    index := len(s.items) - 1
    item := s.items[index]
    s.items = s.items[:index]
    return item, true
}

func (s *Stack) IsEmpty() bool {
    return len(s.items) == 0
}

func main() {
    stack := Stack{}
    stack.Push(1)
    stack.Push(2)
    stack.Push(3)

    for !stack.IsEmpty() {
        item, _ := stack.Pop()
        fmt.Printf("Popped: %d\n", item)
    }
}
```

```java
import java.util.ArrayList;

public class StackExample {
    static class Stack<T> {
        private ArrayList<T> items = new ArrayList<>();

        public void push(T item) {
            items.add(item);
        }

        public T pop() {
            if (isEmpty()) {
                return null;
            }
            return items.remove(items.size() - 1);
        }

        public boolean isEmpty() {
            return items.isEmpty();
        }
    }

    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        stack.push(1);
        stack.push(2);
        stack.push(3);

        while (!stack.isEmpty()) {
            System.out.println("Popped: " + stack.pop());
        }
    }
}
```

kroki

kroki-graphviz

```
digraph G {
  a -> b [dir=both color="red:blue"]
  c -> d [dir=none color="green:red;0.25:blue"]
}
```

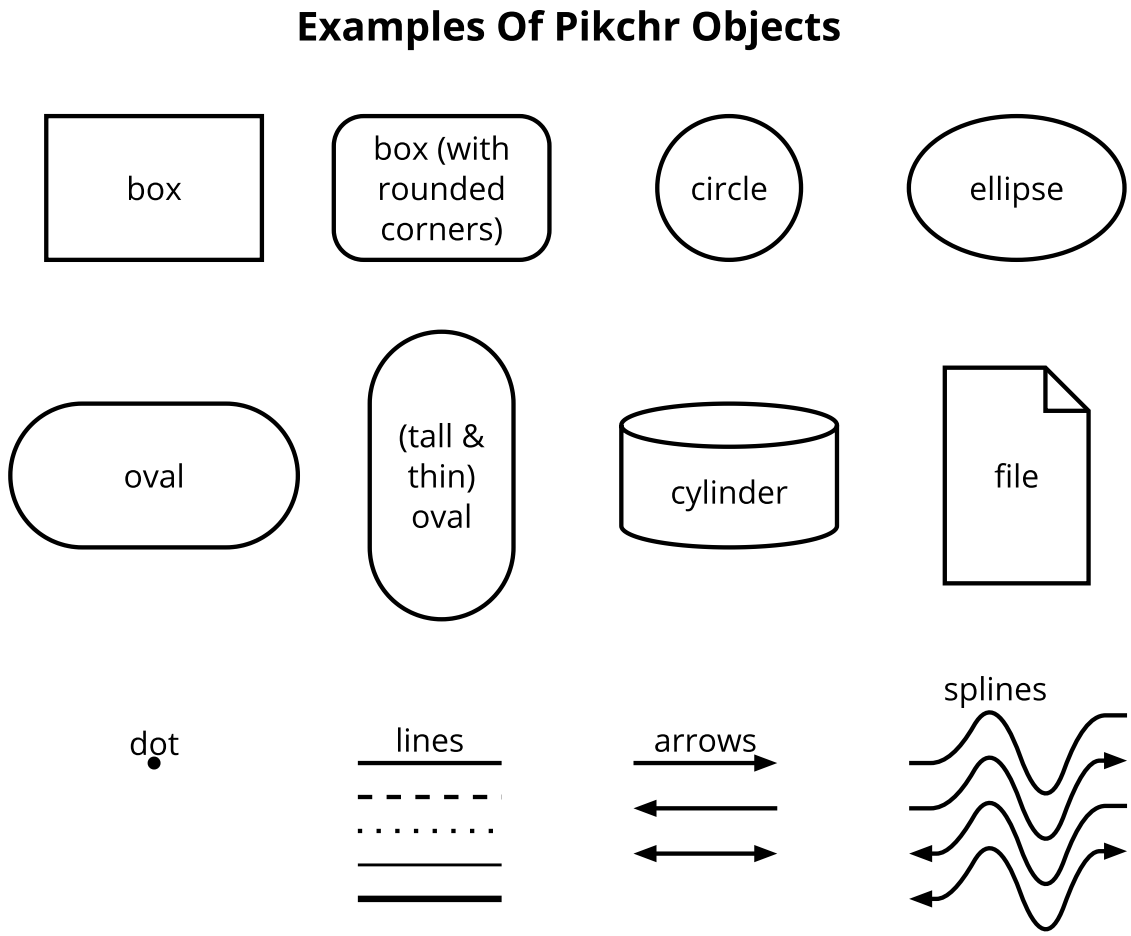
kroki-plantuml

```
@startuml
clock   "Clock_0"   as C0 with period 50
clock   "Clock_1"   as C1 with period 50 pulse 15 offset 10
binary  "Binary"  as B
concise "Concise" as C
robust  "Robust"  as R
analog  "Analog"  as A

@0
C is Idle
R is Idle
A is 0

@100
B is high
C is Waiting
R is Processing
A is 3

@300
R is Waiting
A is 1
@enduml
```

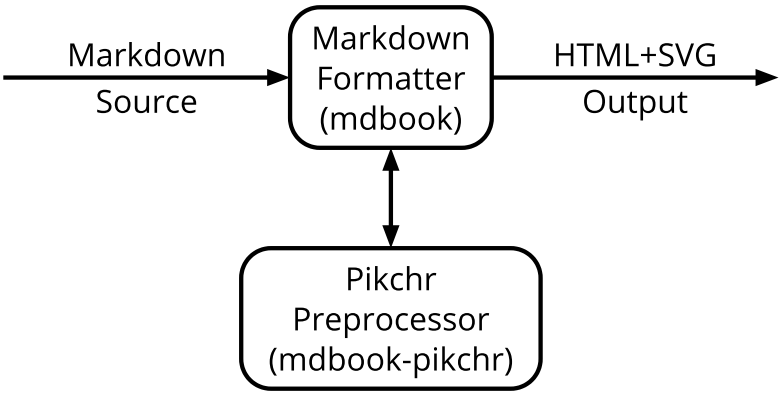
kroki-symbolator

```
module demo_device #(
    //# {{}}
    parameter SIZE = 8,
    parameter RESET_ACTIVE_LEVEL = 1
) (
    //# {{clocks|Clocking}}
    input wire clock,
    //# {{control|Control signals}}
    input wire reset,
    input wire enable,
    //# {{data|Data ports}}
    input wire [SIZE-1:0] data_in,
    output wire [SIZE-1:0] data_out
);
  // ...
endmodule
```

pikchr-demo1

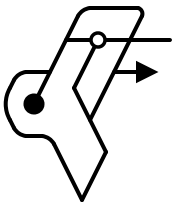


pikchr-demo2



svgbob-demo

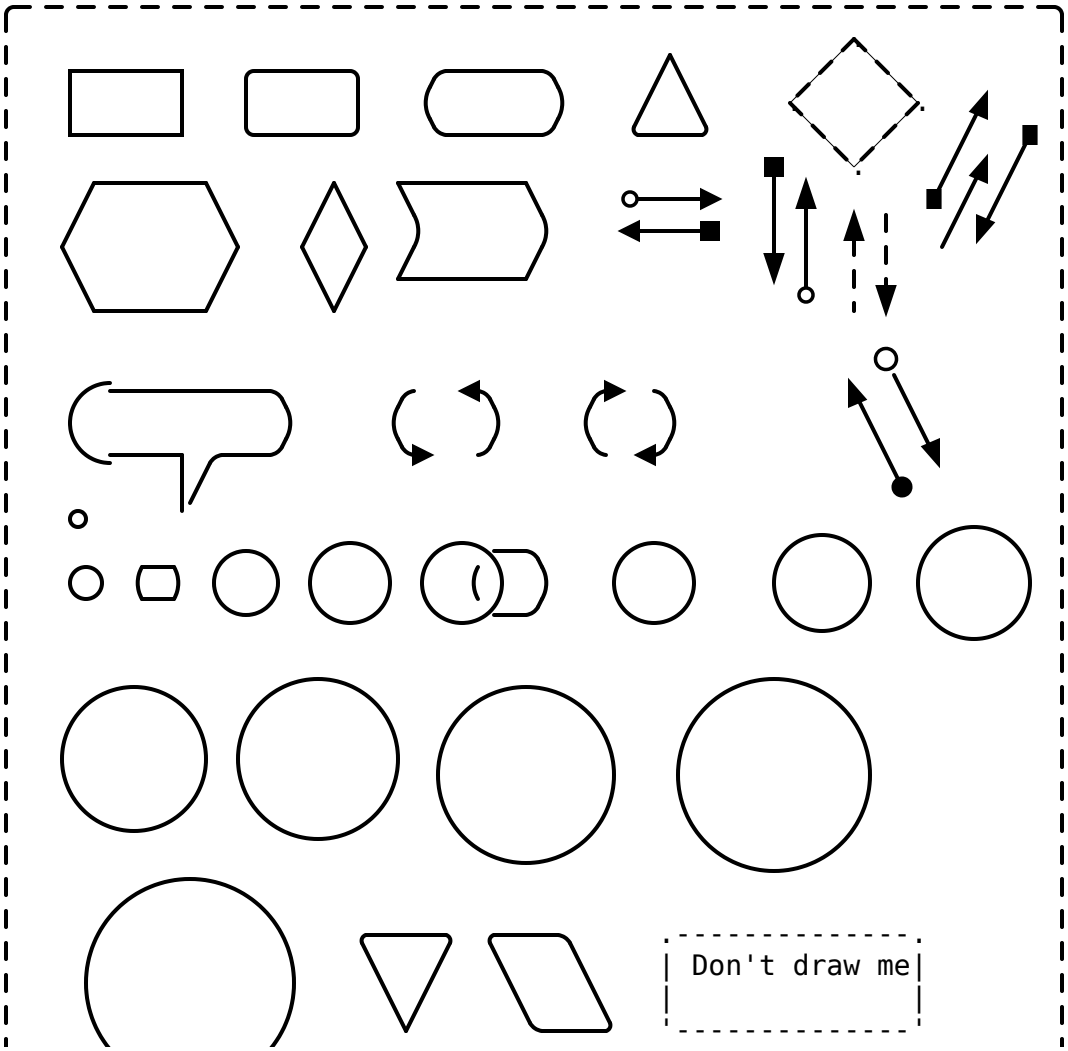
Svgbob is a diagramming model
which uses a set of typing characters
to approximate the intended shape.

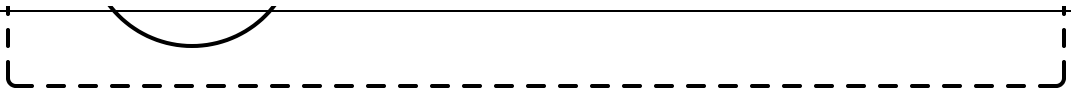


It uses a combination of characters
which are readily available on your keyboards.

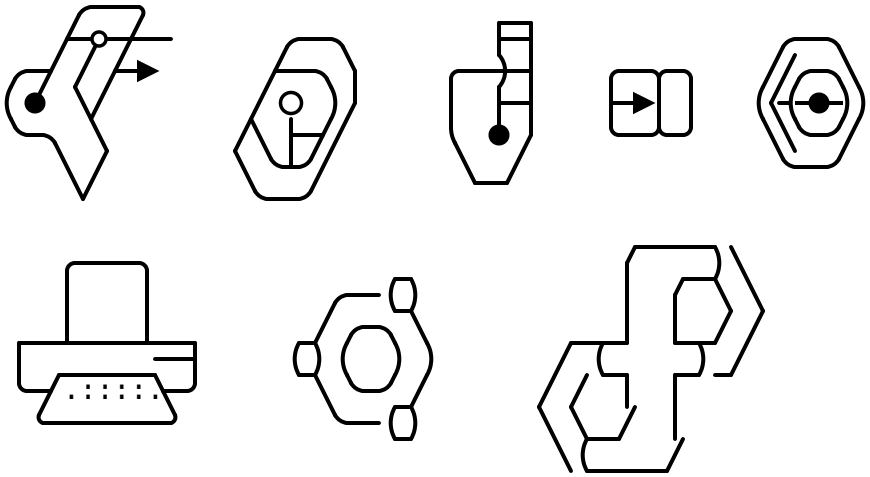
What can it do?

Basic shapes

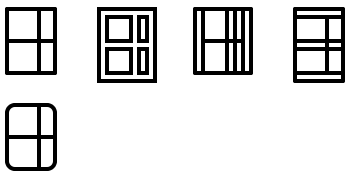




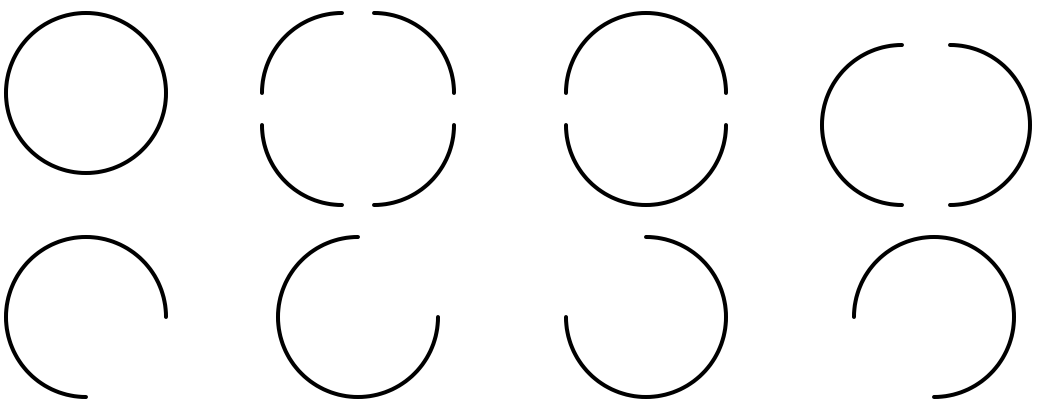
Quick logo scribbles



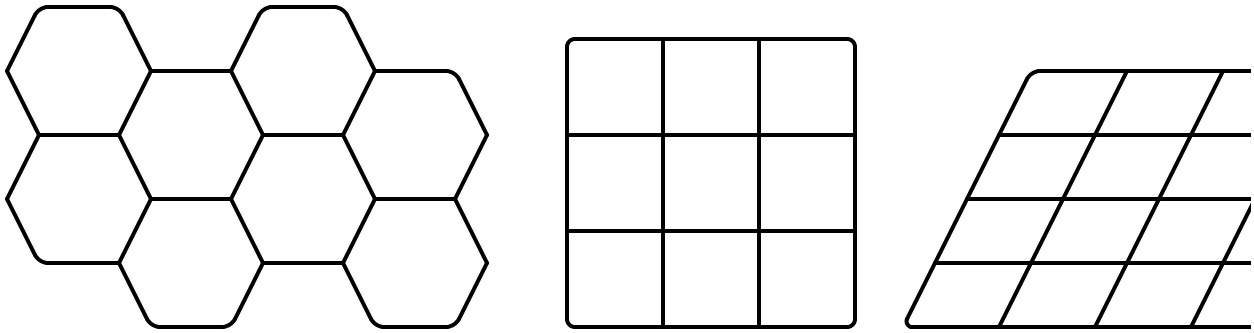
Even unicode box drawing characters are supported

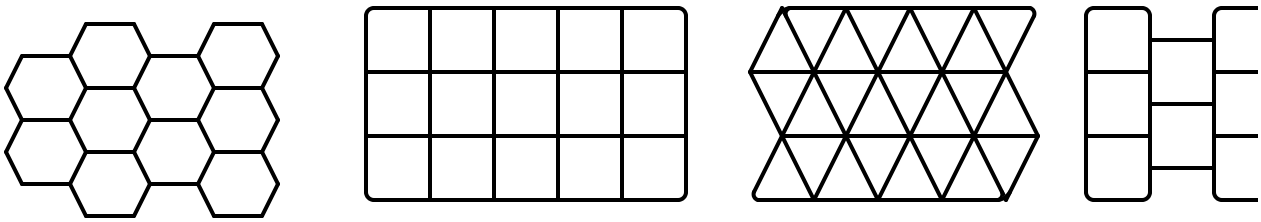


Circle, quarter arcs, half circles, 3/4 quarter arcs

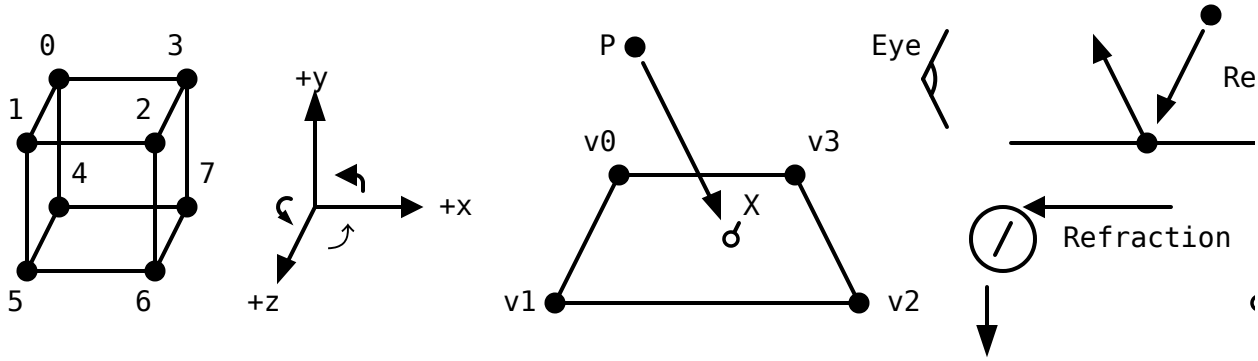


Grids





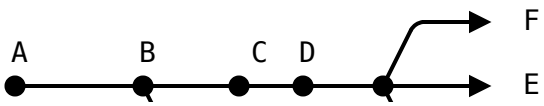
Graphics Diagram

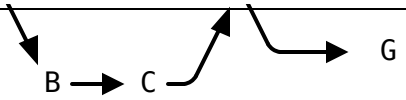


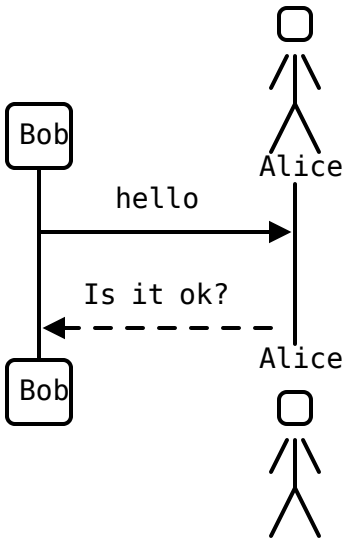
CJK characters

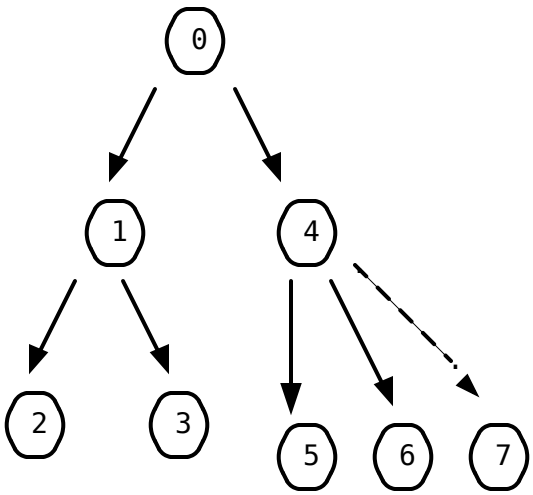
文件系统	调度器	MMU

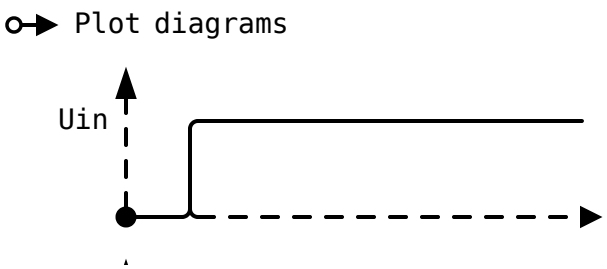
Sequence Diagrams

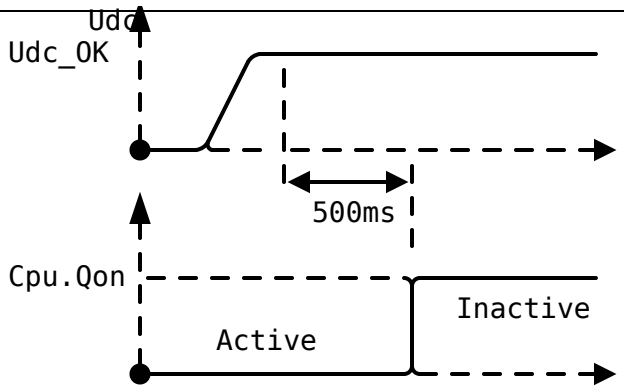



B
C
G

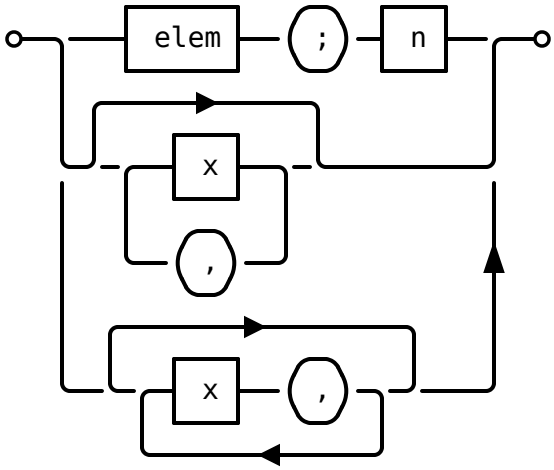

Bob
Alice
hello
Is it ok?
Alice
Bob

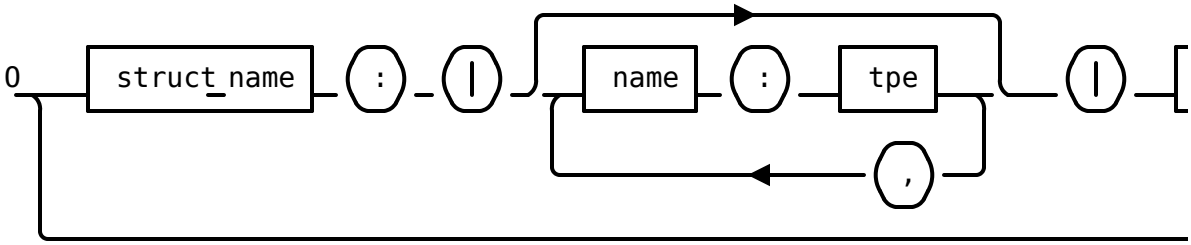

0
1
4
2
3
5
6
7


Plot diagrams
Uin

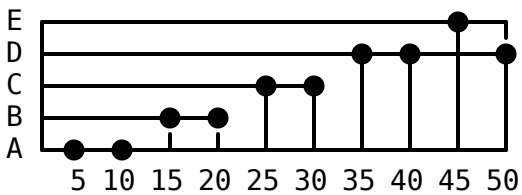


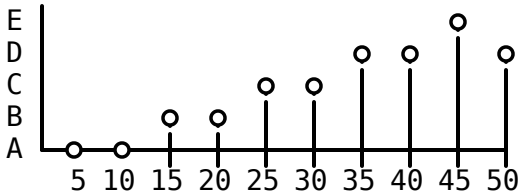
Railroad diagrams

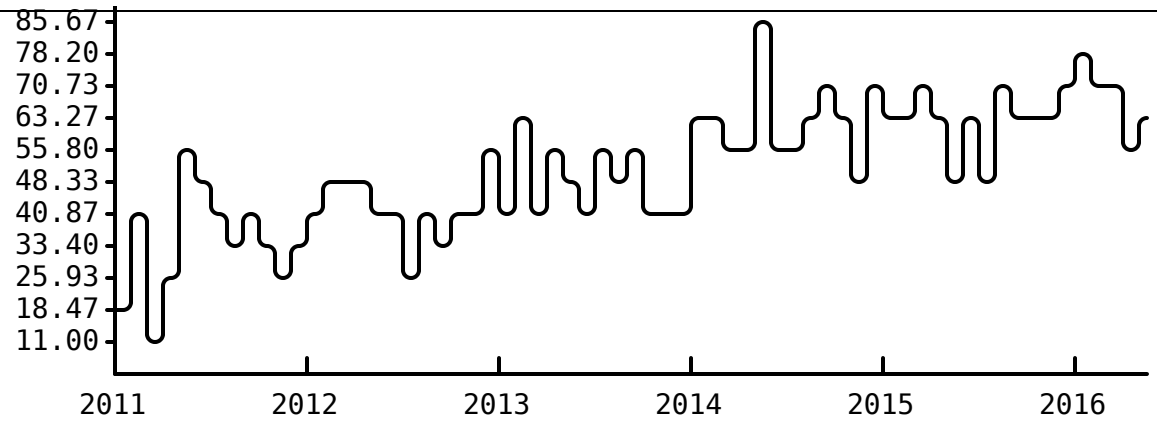




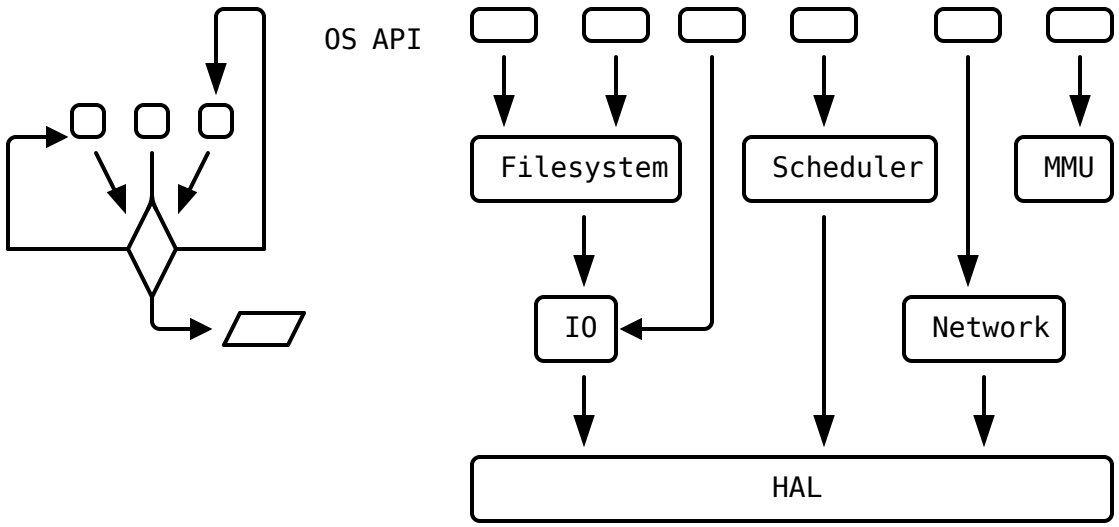
Statistical charts



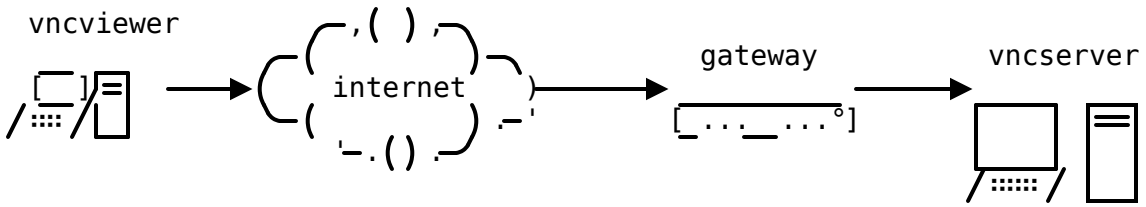


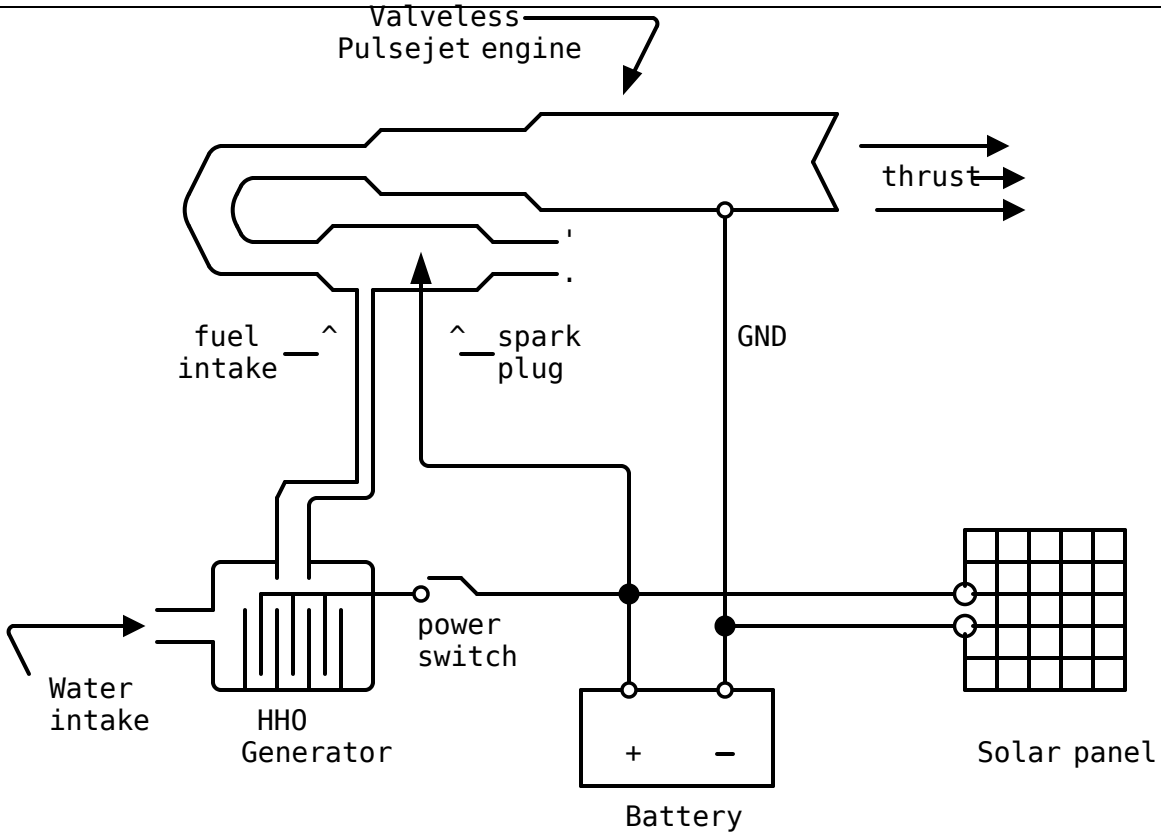


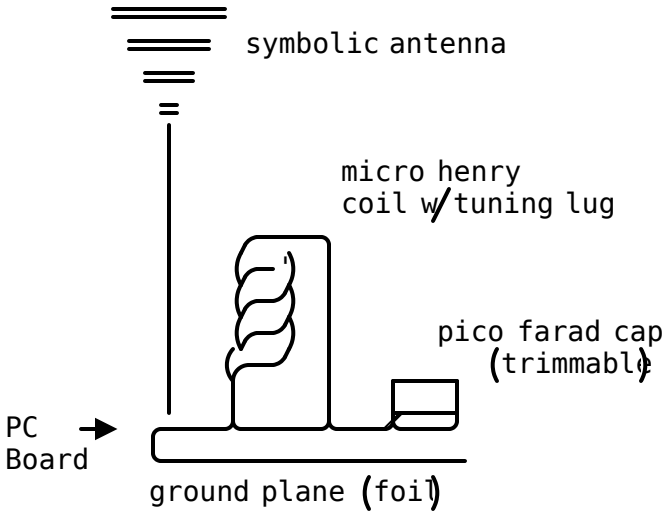
Flow charts



Block diagrams

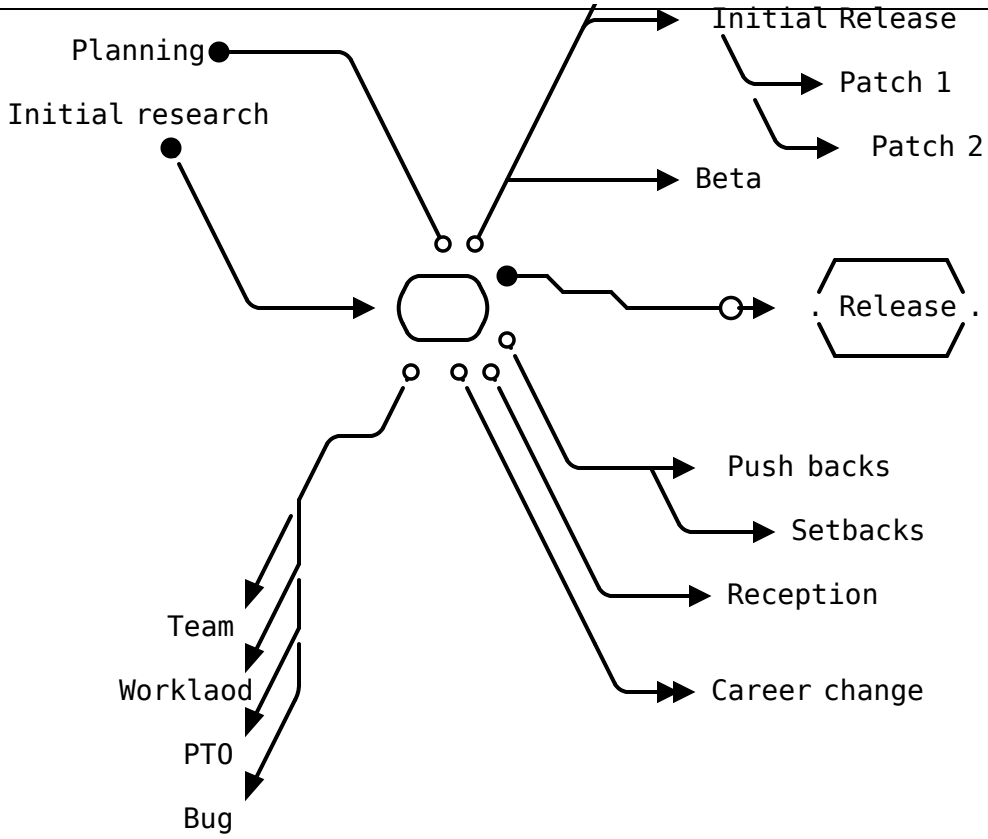




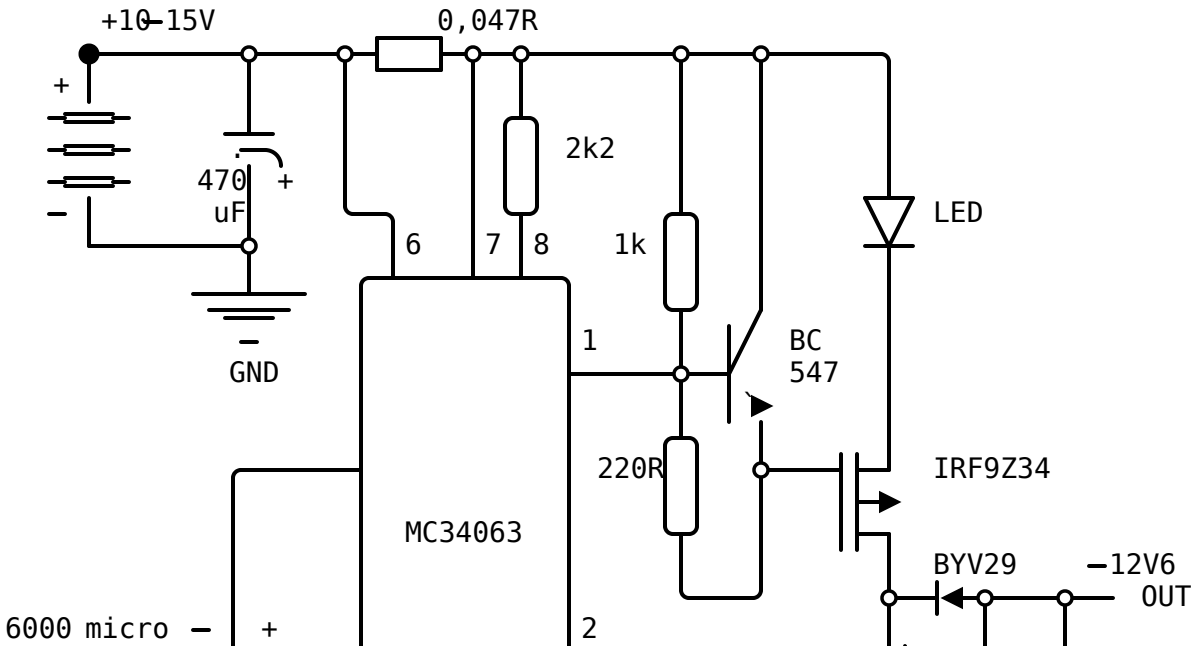


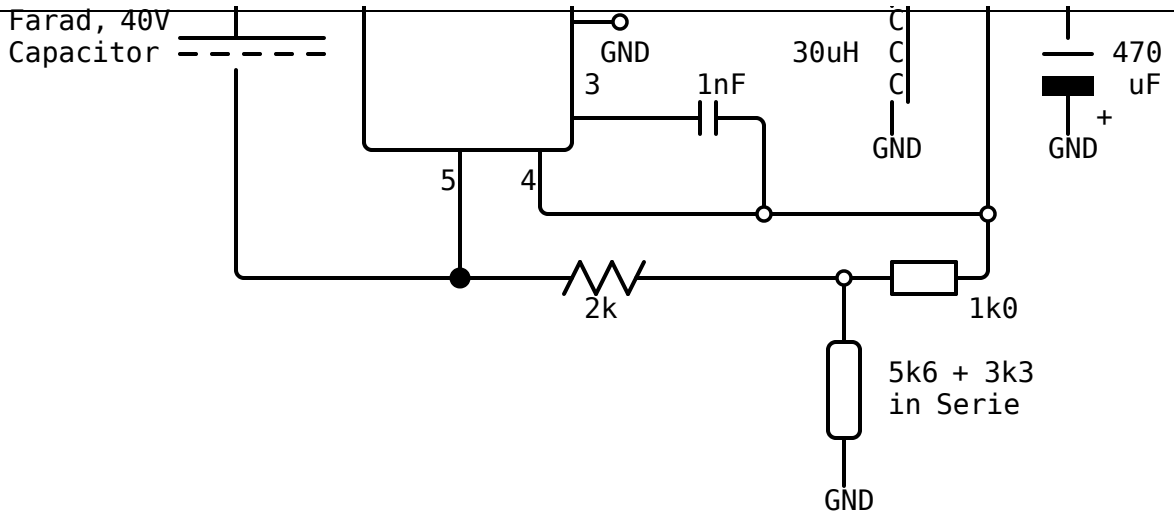
Mindmaps

Alpha

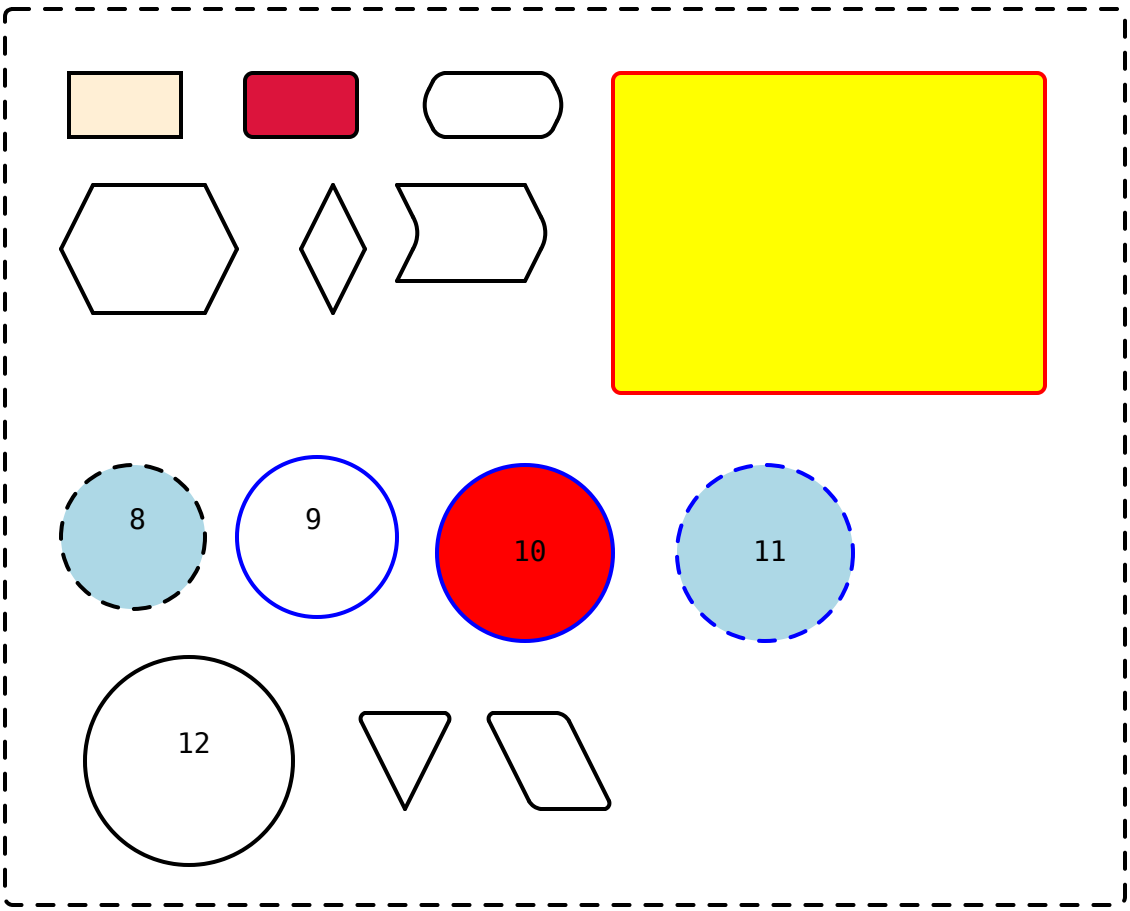


It can do complex stuff such as circuit diagrams



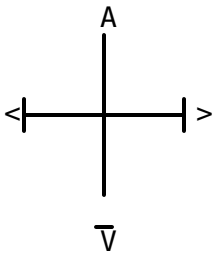


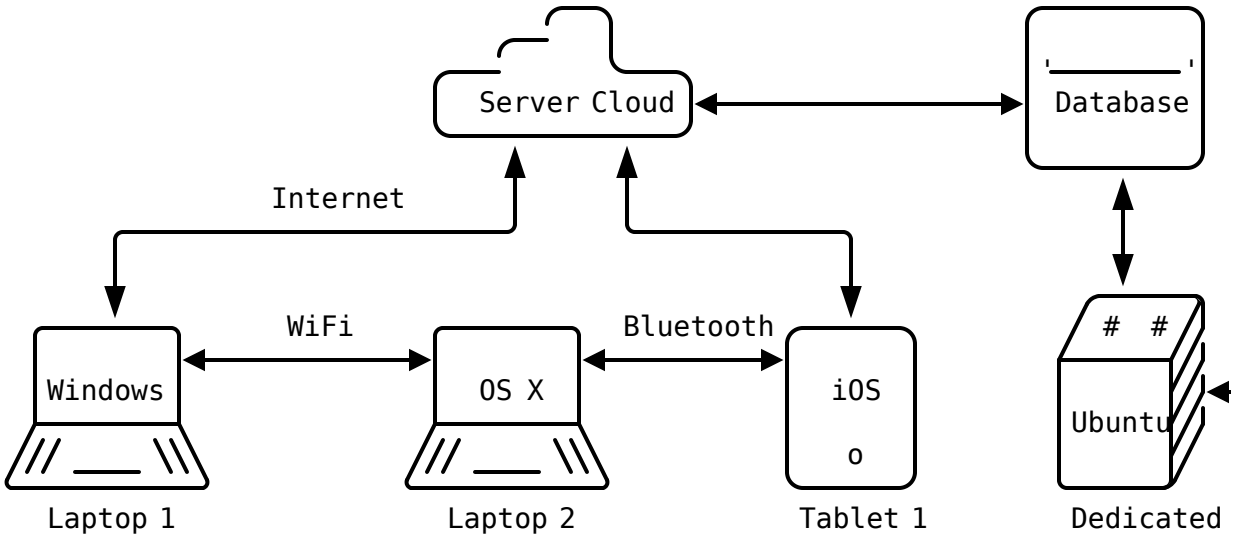
Latest addition: Styling of tagged shapes

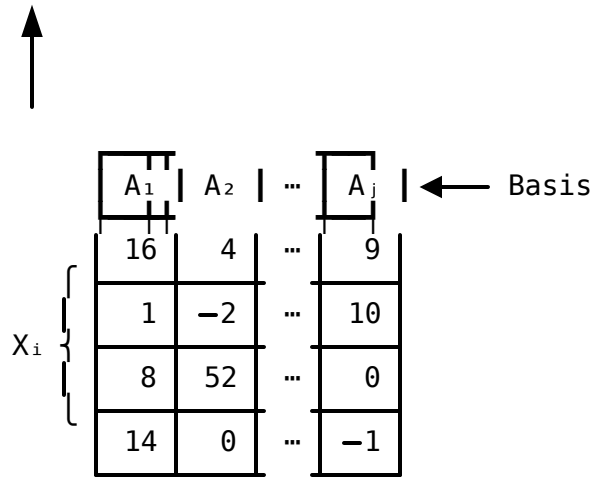


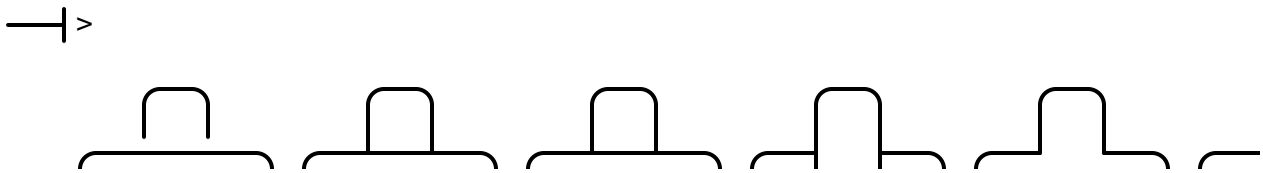
- Plain text format
  Ultimately portable, backward compatible and future proof.
- Degrades gracefully
  Even when not using a graphical renderer, it would still looks good as text based diagrams. Paste the text in your source code.
- Easiest to use. Anyone knows how to edit text.

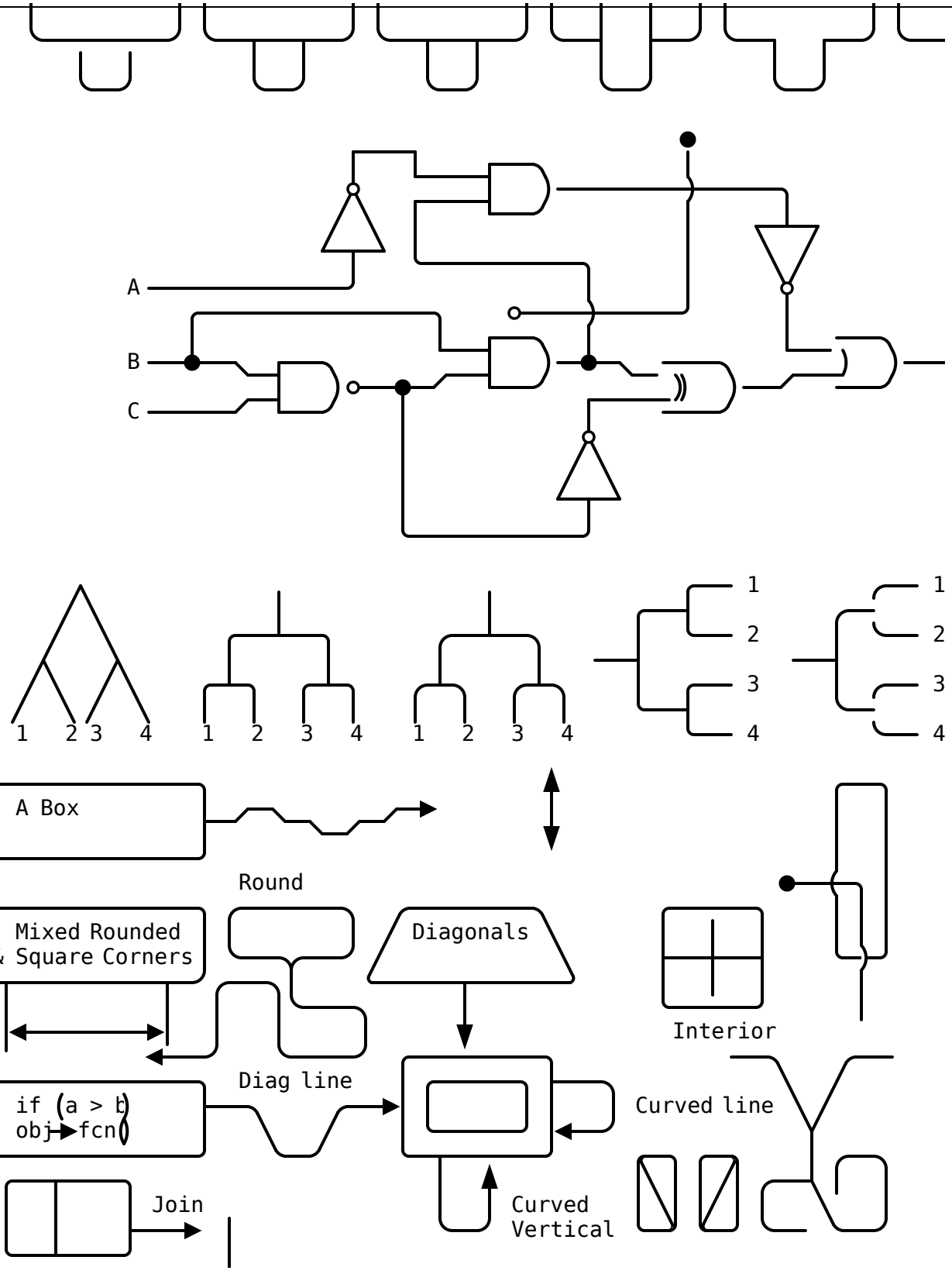
svgbob-otherdemo

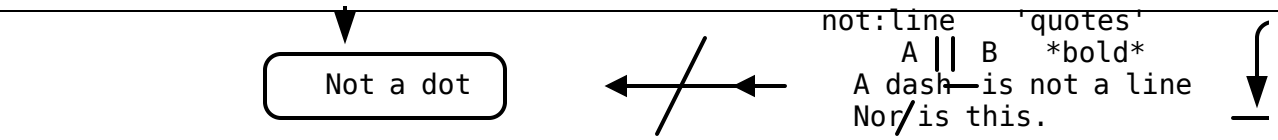

A
<
>

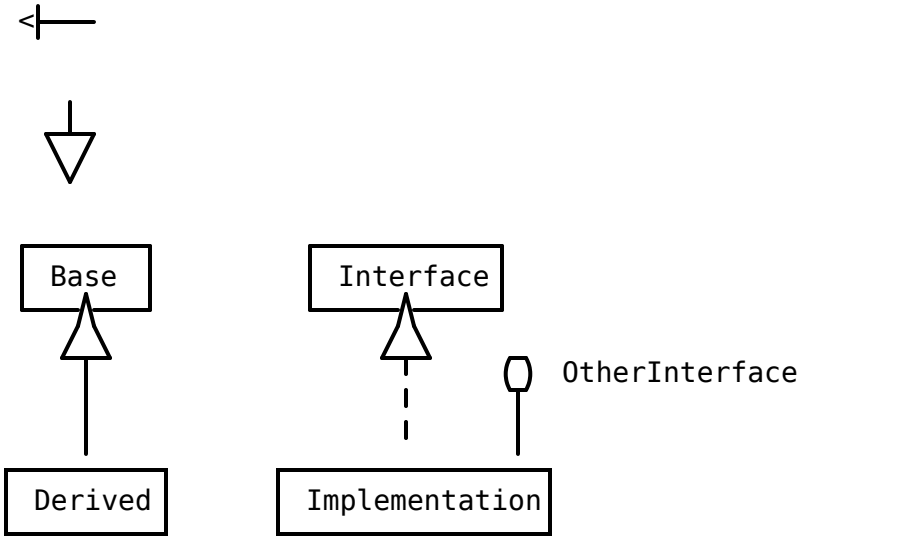

Server Cloud
Database
Internet
WiFi
Bluetooth
Windows
OS X
iOS
o
Ubuntu
Laptop 1
Laptop 2
Tablet 1
Dedicated

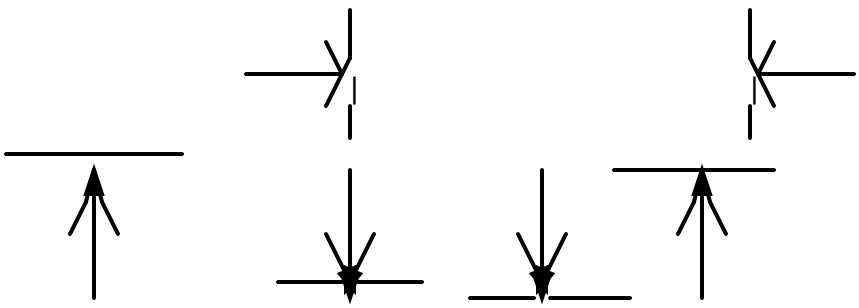

A1
A2
Aj
Basis
16 4 9
1 −2 10
8 52 0
14 0 −1
Xi




A
B
C
1 2 3 4
1 2 3 4
1 2 3 4
1
2
3
4
1
2
3
4
A Box
Round
Mixed Rounded
& Square Corners
Diagonals
Interior
Diag line
if (a > b)
obj->fcn()
Curved line
Join
Curved
Vertical

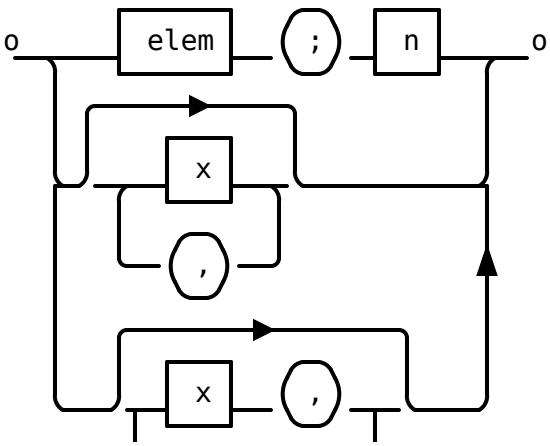


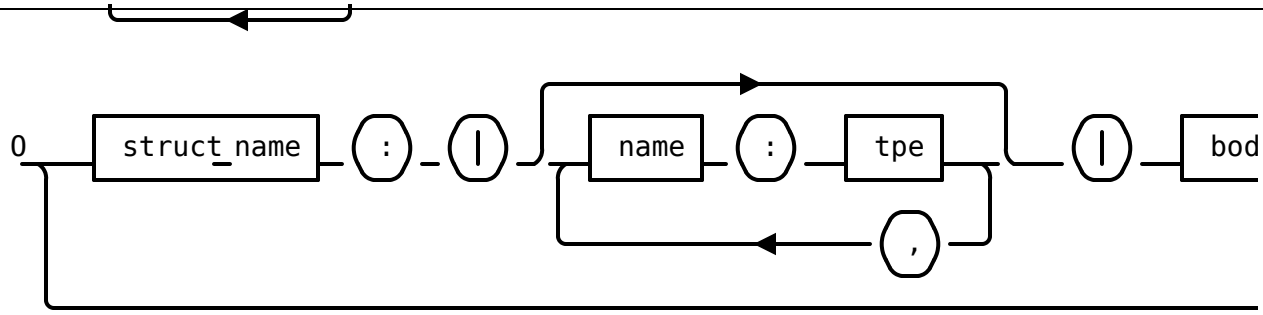




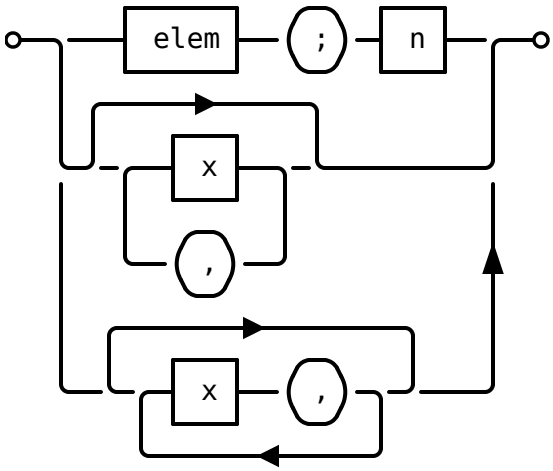
Poor man's rail road

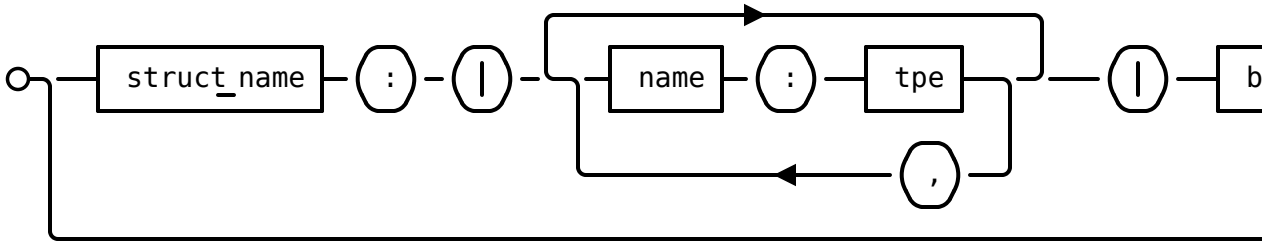
Style #1

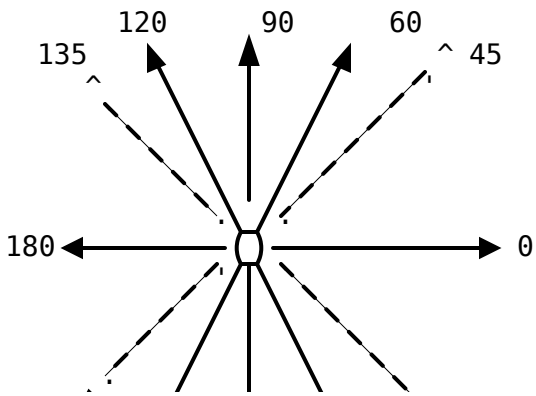


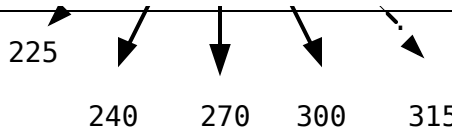

0
struct_name
:
name
:
tpe
,
bod

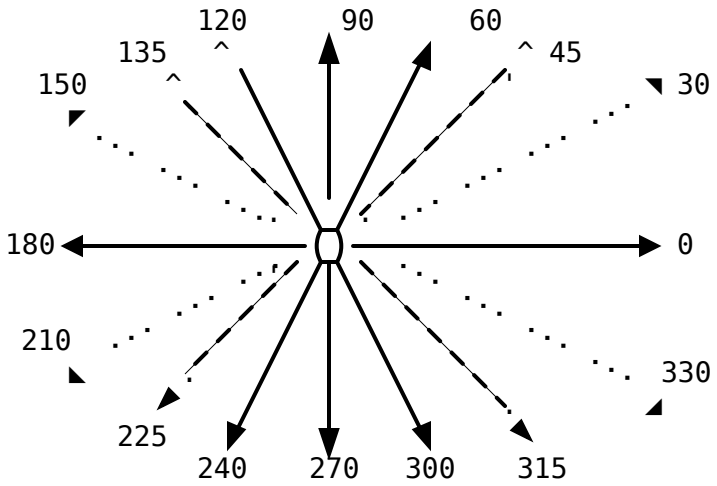
Style #2

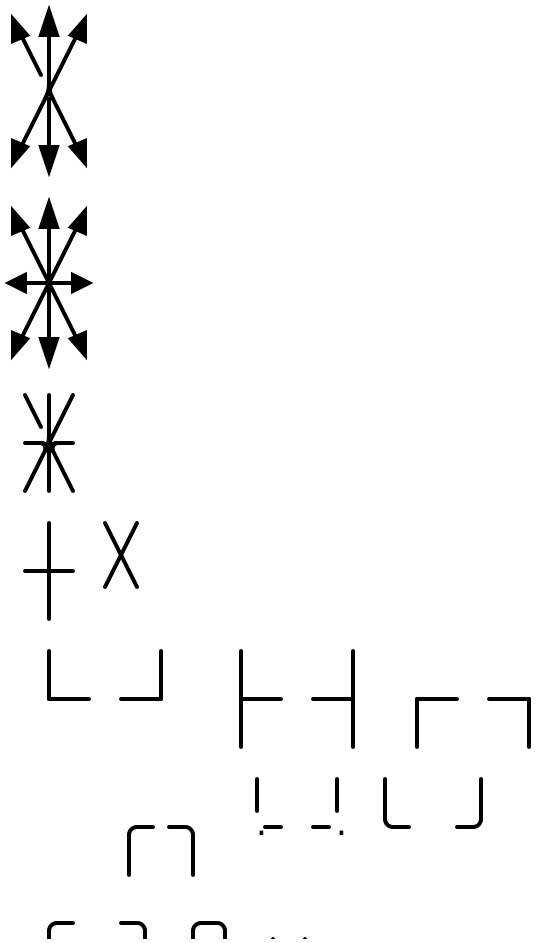

elem
;
n
x
,
x
,

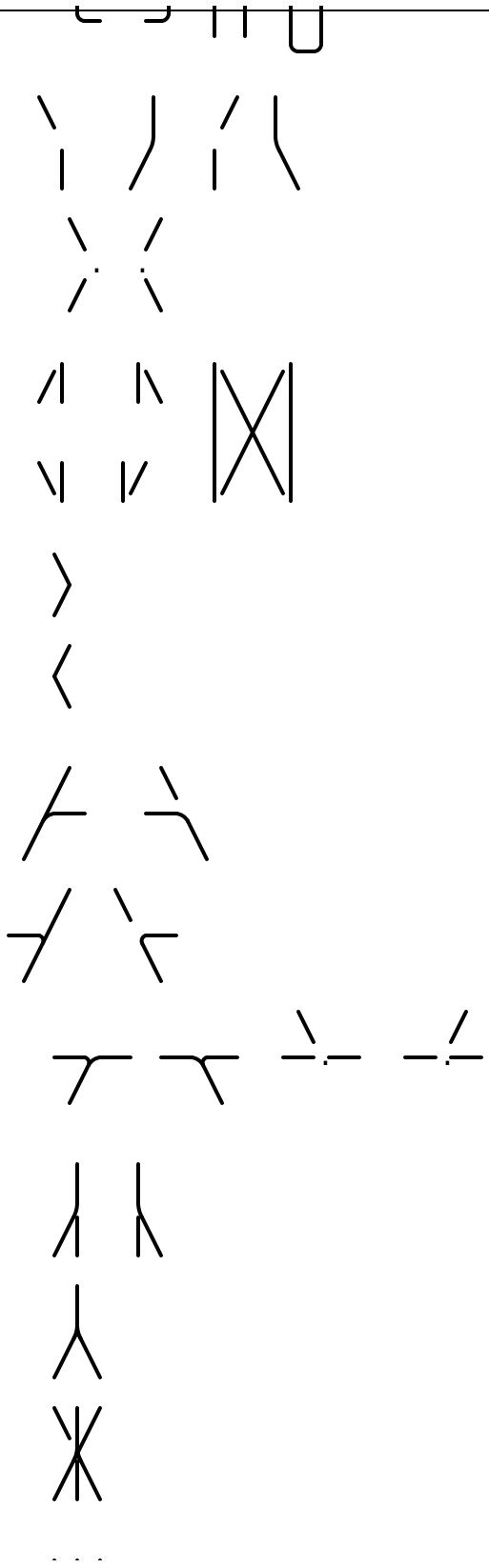

struct_name
:
name
:
tpe
,
b

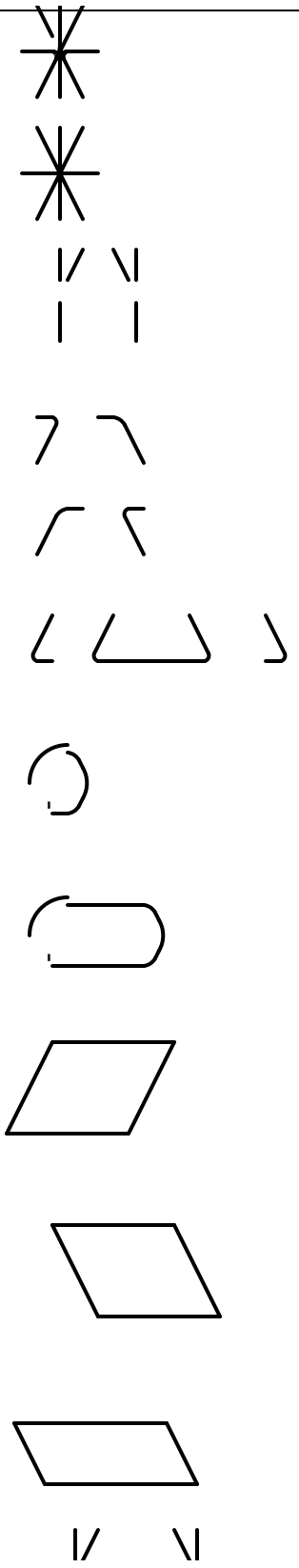

120
90
60
135
45
180
0

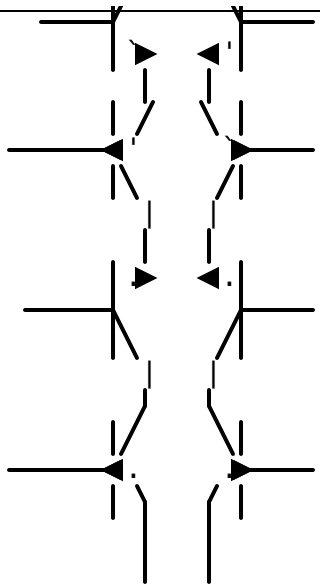

225
240
270
300
315

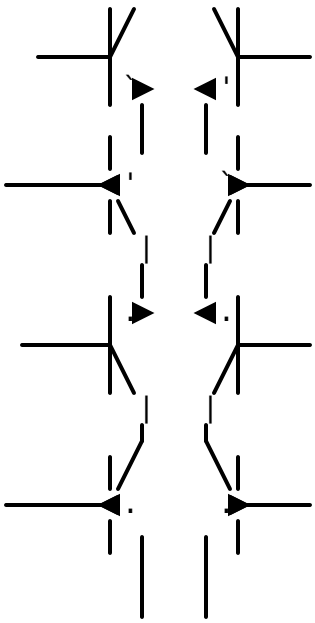

120
90
60
135
45
150
30
180
0
210
330
225
240
270
300
315











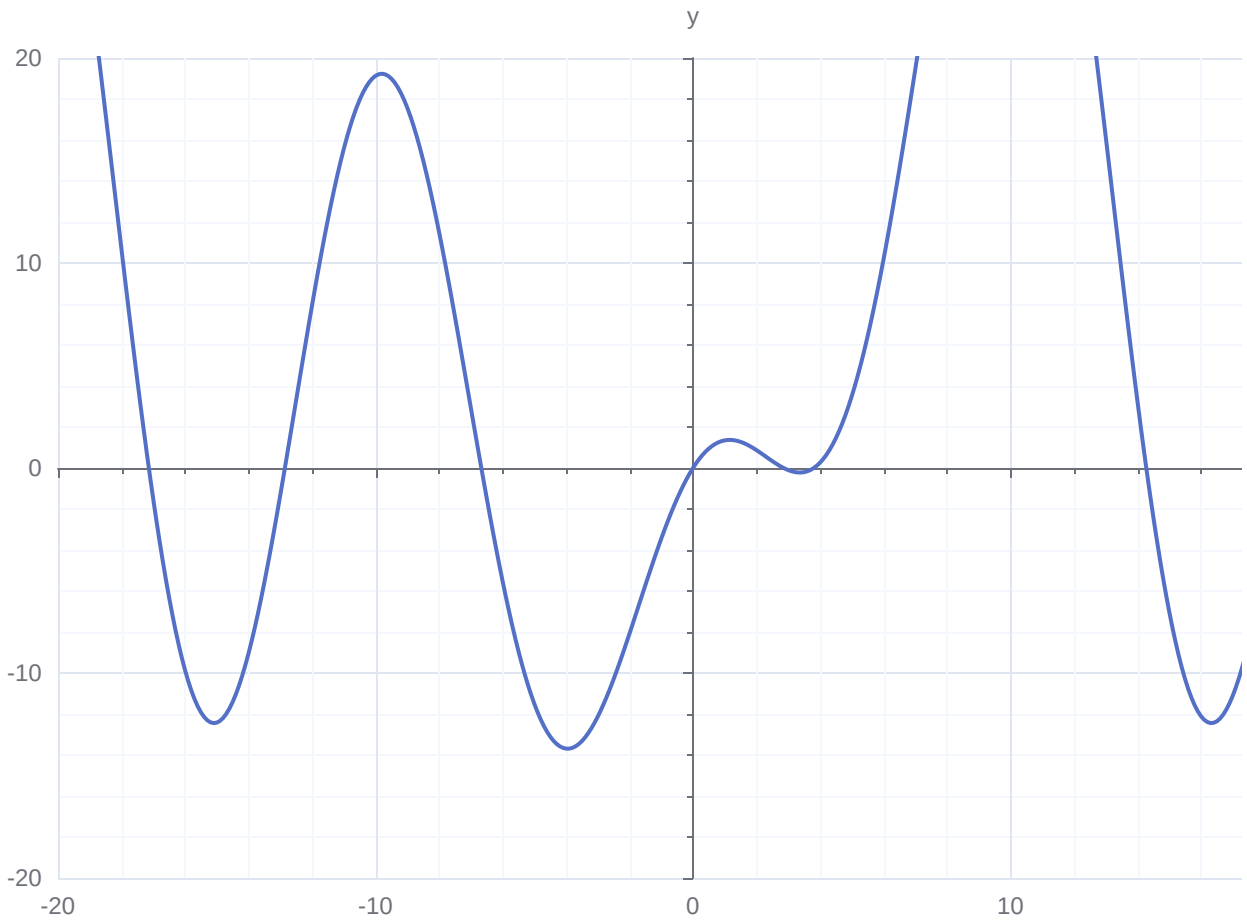
ECharts

echarts-multi-function

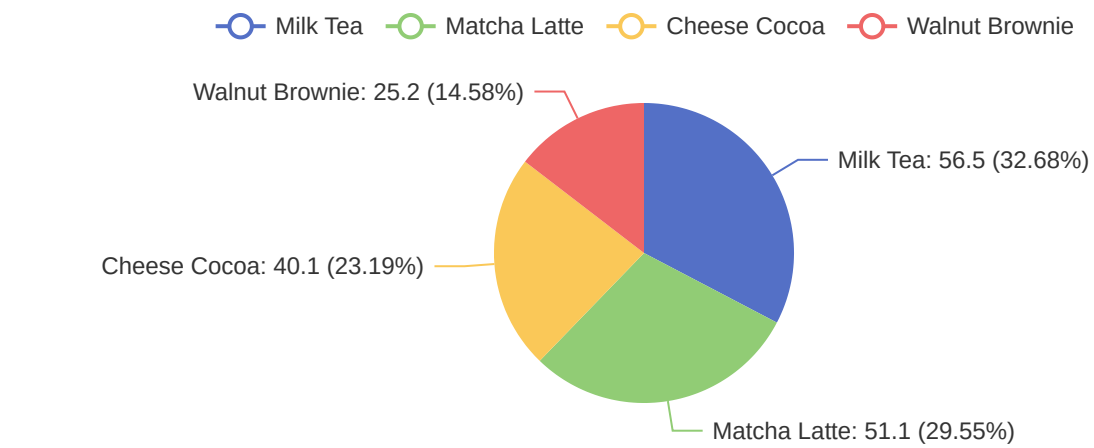
linear quadraticIn quadraticOut quadraticInOut cubicIn

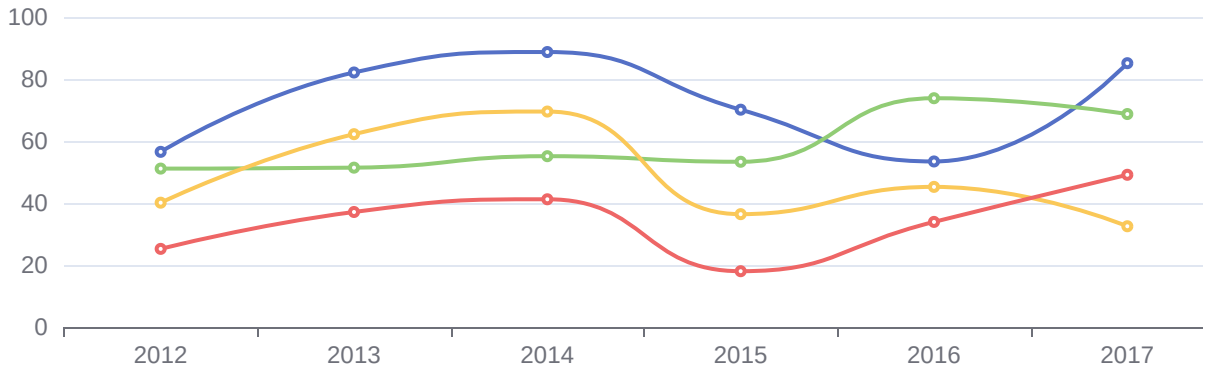

cubicInOut
quarticIn
quarticOut
quarticInOut
quinticIn
quinticInOut
sinusoidalIn
sinusoidalOut
sinusoidalInOut
exponentialIn
exponentialInOut
circularIn
circularOut
circularInOut
elasticIn
elasticInOut
backIn
backOut
backInOut
bounceIn
bounceInOut
Different Easing Functions

echarts-function



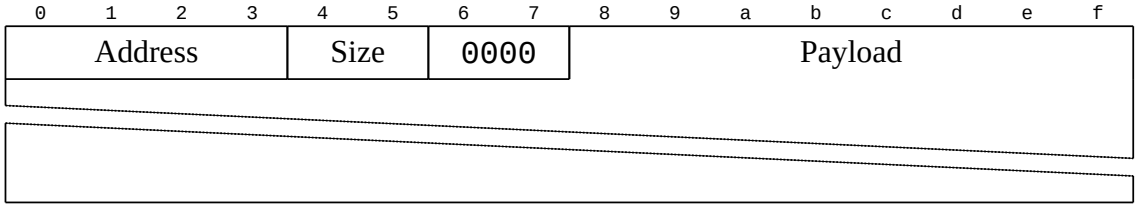
echarts-dataset



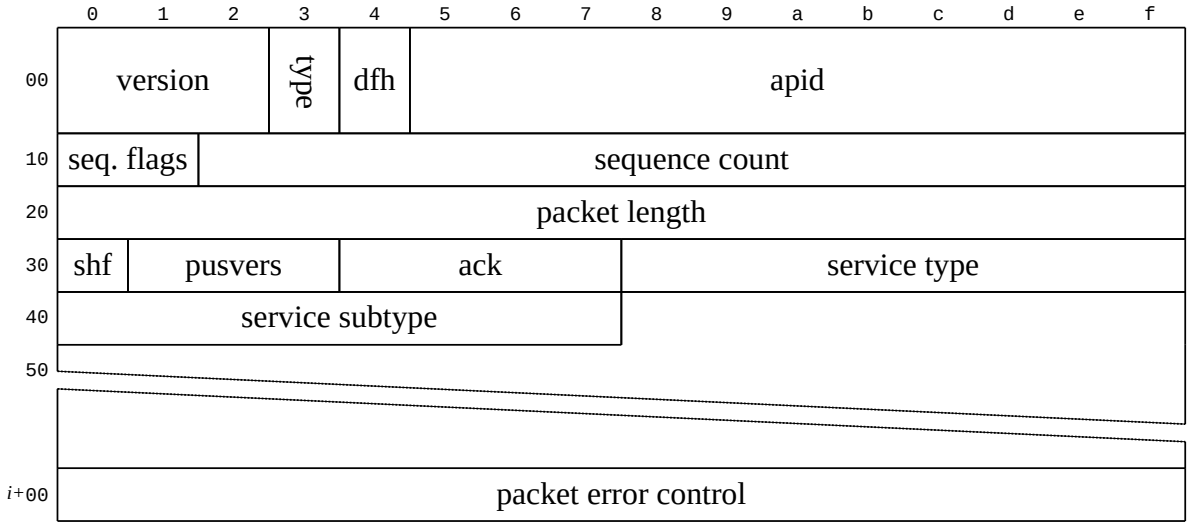


Bytefield

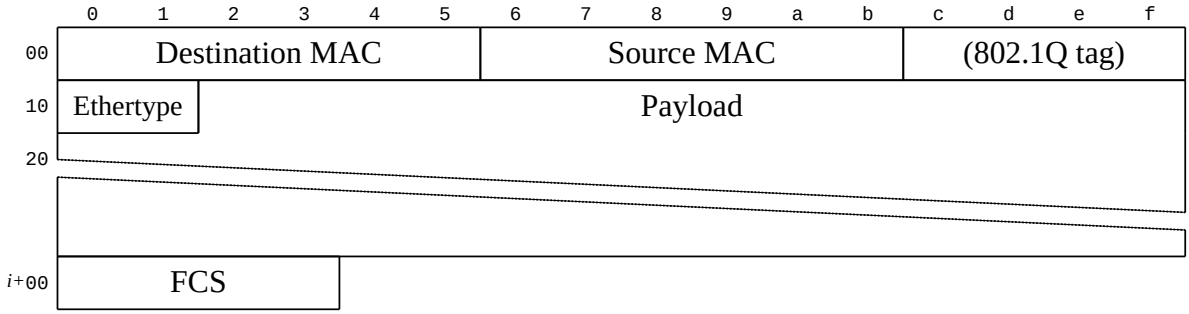
Basic.



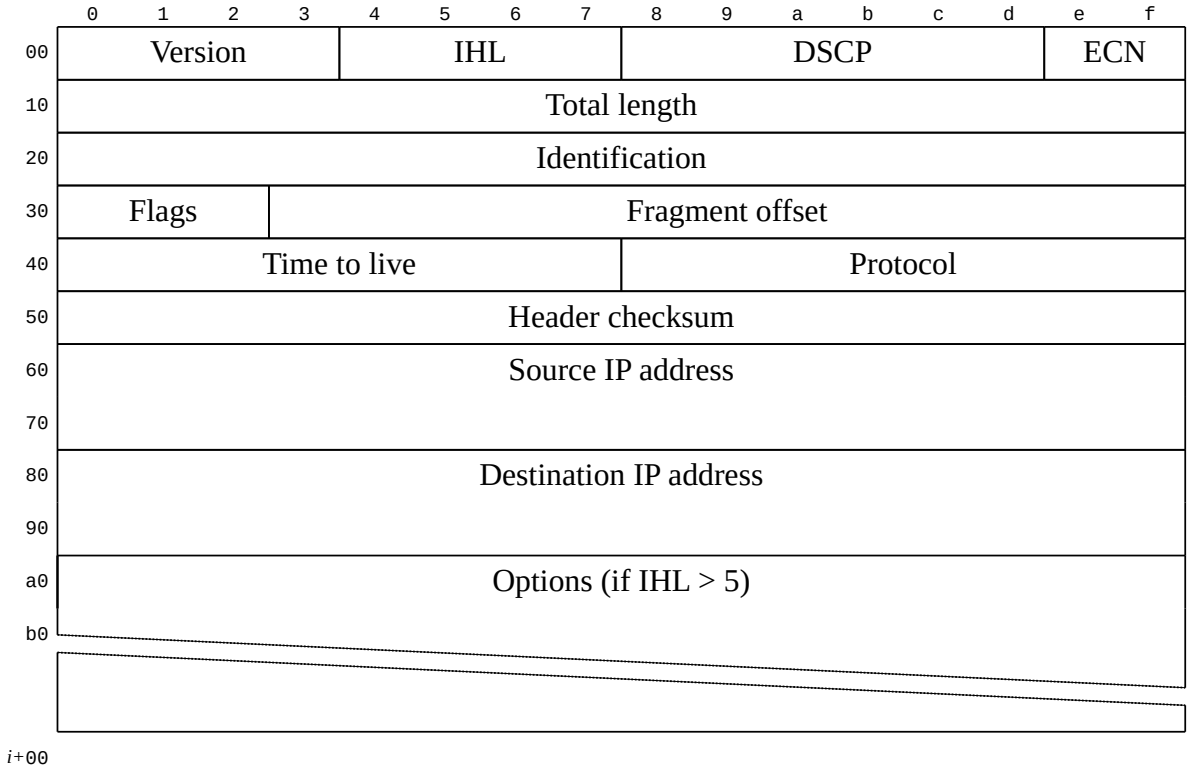
ccsds.edn



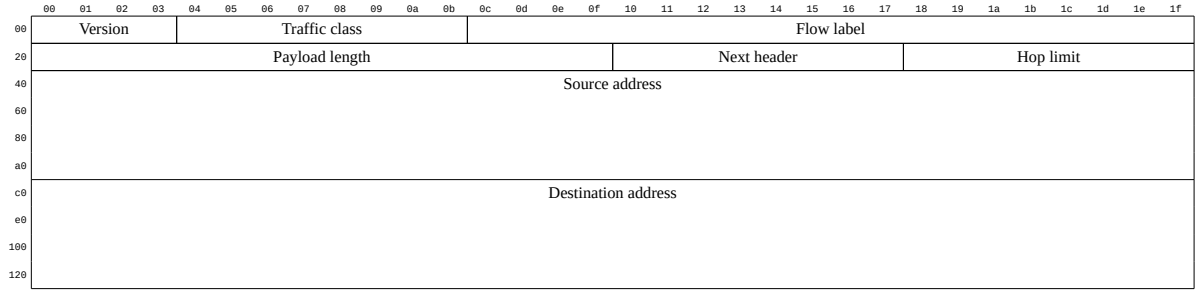
ethernet.edn



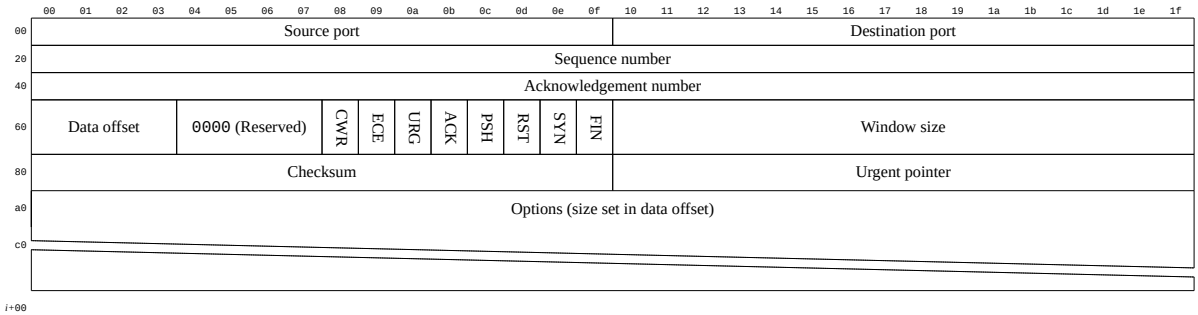
ipv4.edn



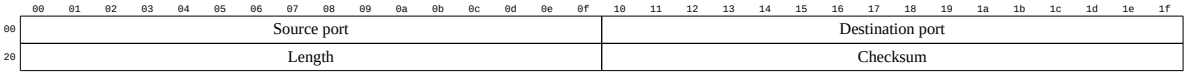
ipv6.edn



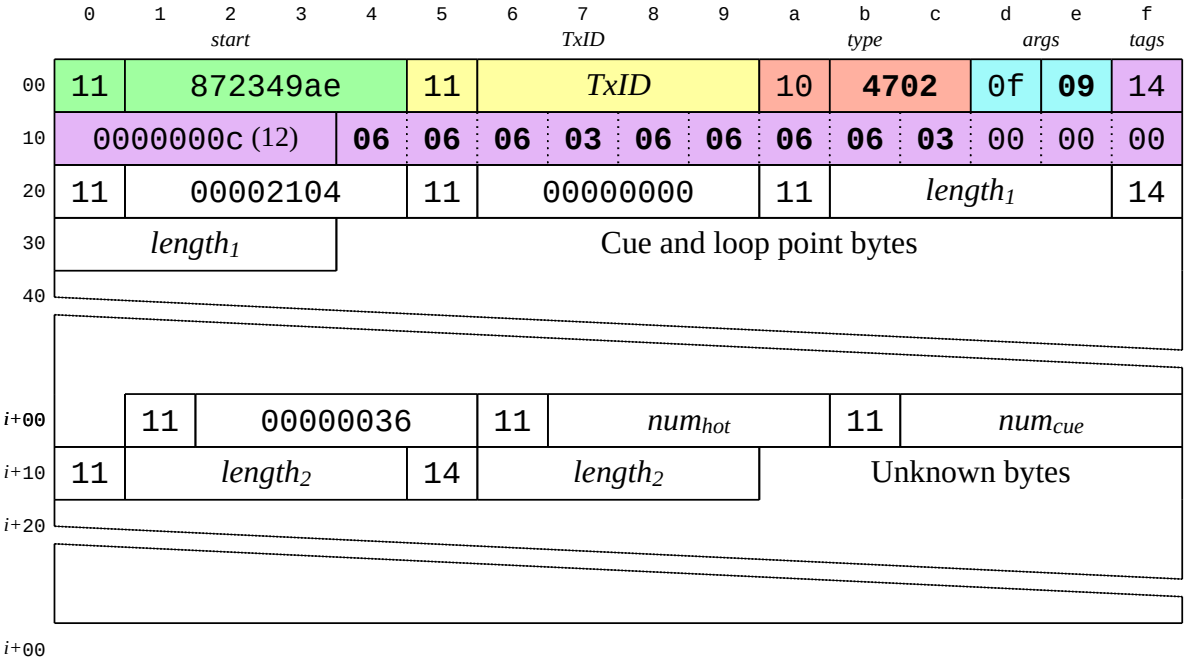
tcp.edn



udp.edn



test.edn



Latex Pictures

```
# node-tikzjax
chemfig
tikz-cd
circuitikz
pgfplots
array
amsmath
amstext
amsfonts
amssymb
tikz-3dplot
```

tikz demo1

```
% demo1
\documentclass[tikz]{standalone}

\usepackage{pgfplots}
\pgfplotsset{compat=1.16}

\begin{document}

\begin{tikzpicture}
\begin{axis}[colormap/viridis]
\addplot3[
        surf,
        samples=18,
        domain=-3:3
]
{exp(-x^2-y^2)*x};
\end{axis}
\end{tikzpicture}

\end{document}
```

tikz demo2

```
% demo2
\documentclass[tikz]{standalone}

\begin{document}
  \begin{tikzpicture}[domain=0:4]
    \draw[very thin,color=gray] (-0.1,-1.1) grid (3.9,3.9);
    \draw[->] (-0.2,0) -- (4.2,0) node[right] {$x$};
    \draw[->] (0,-1.2) -- (0,4.2) node[above] {$f(x)$};
    \draw[color=red]    plot (\x,\x)             node[right] {$f(x) =x$};
    \draw[color=blue]   plot (\x,{sin(\x r)})    node[right] {$f(x) = \sin
x$};
    \draw[color=orange] plot (\x,{0.05*exp(\x)}) node[right] {$f(x) =
\frac{1}{20} \mathrm e^x$};
  \end{tikzpicture}
\end{document}
```

tikz demo3

```
% demo3
\documentclass[tikz]{standalone}

\usepackage{circuitikz}
\begin{document}

\begin{circuitikz}[american, voltage shift=0.5]
\draw (0,0)
to[isource, l=$I_0$, v=$V_0$] (0,3)
to[short, -*, i=$I_0$] (2,3)
to[R=$R_1$, i>_=$i_1$] (2,0) -- (0,0);
\draw (2,3) -- (4,3)
to[R=$R_2$, i>_=$i_2$]
(4,0) to[short, -*] (2,0);
\end{circuitikz}

\end{document}
```

tikz demo4

```
% demo4
\documentclass[tikz]{standalone}

\usepackage{tikz-cd}

\begin{document}

\begin{tikzcd}
  T
  \arrow[drr, bend left, "x"]
  \arrow[ddr, bend right, "y"]
  \arrow[dr, dotted, "{(x,y)}" description] & & \\
  K & X \times_Z Y \arrow[r, "p"] \arrow[d, "q"]
  & X \arrow[d, "f"] \\
  & Y \arrow[r, "g"]
  & Z
\end{tikzcd}

\quad \quad

\begin{tikzcd}[row sep=2.5em]
A' \arrow[rr,"f'"] \arrow[dr,swap,"a"] \arrow[dd,swap,"g'"] &&
  B' \arrow[dd,swap,"h'" near start] \arrow[dr,"b"] \\
& A \arrow[rr,crossing over,"f" near start] &&
  B \arrow[dd,"h"] \\
C' \arrow[rr,"k'" near end] \arrow[dr,swap,"c"] && D' \arrow[dr,swap,"d"]
\\
& C \arrow[rr,"k"] \arrow[uu,<-,crossing over,"g" near end]&& D
\end{tikzcd}

\end{document}
```

tikz demo5

```
% demo5
\documentclass[tikz]{standalone}

\begin{document}
\begin{tikzpicture}
\node[circle,
minimum width =30pt ,
minimum height =30pt ,draw=blue] (1) at(0,2){$x_1$};
\node[circle,
minimum width =30pt ,
minimum height =30pt ,draw=blue] (2) at(0,0){$x_2$};
\node[circle,
minimum width =30pt ,
minimum height =30pt ,draw=orange] (3) at(2,-1){$a_3^{(2)}$};
\node[circle,
minimum width =30pt ,
minimum height =30pt ,draw=orange] (4) at(2,1){$a_2^{(2)}$};
\node[circle,
minimum width =30pt ,
minimum height =30pt ,draw=orange] (5) at(2,3){$a_1^{(2)}$};
\node[circle,
minimum width =30pt ,
minimum height =30pt ,draw=orange] (6) at(4,-1){$a_3^{(3)}$};
\node[circle,
minimum width =30pt ,
minimum height =30pt ,draw=orange] (7) at(4,1){$a_2^{(3)}$};
\node[circle,
minimum width =30pt ,
minimum height =30pt ,draw=orange] (8) at(4,3){$a_1^{(3)}$};
\node[circle,
minimum width =30pt ,
minimum height =30pt ,draw=purple] (9) at(6,2){$a_1^{(4)}$};
\node[circle,
minimum width =30pt ,
minimum height =30pt ,draw=purple] (10) at(6,0){$a_2^{(4)}$};
\draw[->] (1) --(3);
\draw[->] (1) --(4);
\draw[->] (1) --(5);
\draw[->] (2) --(3);
\draw[->] (2) --(4);
\draw[->] (2) --(5);
\draw[->] (3) --(6);
\draw[->] (3) --(7);
\draw[->] (3) --(8);
```

```
\draw[->] (4) --(6);
\draw[->] (4) --(7);
\draw[->] (4) --(8);
\draw[->] (5) --(6);
\draw[->] (5) --(7);
\draw[->] (5) --(8);
\draw[->] (6) --(9);
\draw[->] (6) --(10);
\draw[->] (7) --(9);
\draw[->] (7) --(10);
\draw[->] (8) --(9);
\draw[->] (8) --(10);
\end{tikzpicture}
\end{document}
```

tikz demo6

```
% demo6
% Author: Till Tantau
% Source: The PGF/TikZ manual
\documentclass[tikz]{standalone}

\begin{document}
\pagestyle{empty}
\begin{tikzpicture}[scale=3,cap=round]
% Local definitions
\def\costhirty{0.8660256}

% Colors
\colorlet{anglecolor}{green!50!black}
\colorlet{sincolor}{red}
\colorlet{tancolor}{orange!80!black}
\colorlet{coscolor}{blue}

% Styles
\tikzstyle{axes}=[]
\tikzstyle{important line}=[very thick]
\tikzstyle{information text}=[rounded corners,fill=red!10,inner sep=1ex]

% The graphic
\draw[style=help lines,step=0.5cm] (-1.4,-1.4) grid (1.4,1.4);

\draw (0,0) circle (1cm);

\begin{scope}[style=axes]
    \draw[->] (-1.5,0) -- (1.5,0) node[right] {$x$};
    \draw[->] (0,-1.5) -- (0,1.5) node[above] {$y$};

    \foreach \x/\xtext in {-1, -.5/-\frac{1}{2}, 1}
    \draw[xshift=\x cm] (0pt,1pt) -- (0pt,-1pt) node[below,fill=white]
            {$\xtext$};

    \foreach \y/\ytext in {-1, -.5/-\frac{1}{2}, .5/\frac{1}{2}, 1}
    \draw[yshift=\y cm] (1pt,0pt) -- (-1pt,0pt) node[left,fill=white]
            {$\ytext$};
\end{scope}

\filldraw[fill=green!20,draw=anglecolor] (0,0) -- (3mm,0pt) arc(0:30:3mm);
\draw (15:2mm) node[anglecolor] {$\alpha$};

\draw[style=important line,sincolor]
```

```
    (30:1cm) -- node[left=1pt,fill=white] {$\sin \alpha$} +(0,-.5);

\draw[style=important line,coscolor]
    (0,0) -- node[below=2pt,fill=white] {$\cos \alpha$} (\costhirty,0);

\draw[style=important line,tancolor] (1,0) --
    node [right=1pt,fill=white]
    {
    $\displaystyle \tan \alpha \color{black}=
    \frac{{\color{sincolor}\sin \alpha}}{\color{coscolor}\cos \alpha}$
    } (intersection of 0,0--30:1cm and 1,0--1,1) coordinate (t);

\draw (0,0) -- (t);

\draw[xshift=1.85cm] node [right,text width=6cm,style=information text]
    {
    The {\color{anglecolor} angle $\alpha$} is $30^\circ$ in the
    example ($\pi/6$ in radians). The {\color{sincolor}sine of
        $\alpha$}, which is the height of the red line, is
    \[
    {\color{sincolor} \sin \alpha} = 1/2.
    \]
    By the Theorem of Pythagoras we have ${\color{coscolor}\cos^2 \alpha}
+
    {\color{sincolor}\sin^2\alpha} =1$. Thus the length of the blue
    line, which is the {\color{coscolor}cosine of $\alpha$}, must be
    \[
    {\color{coscolor}\cos\alpha} = \sqrt{1 - 1/4} = \textstyle
    \frac{1}{2} \sqrt 3.
    \]%
    This shows that {\color{tancolor}$\tan \alpha$}, which is the
    height of the orange line, is
    \[
    {\color{tancolor}\tan\alpha} = \frac{{\color{sincolor}\sin
        \alpha}}{\color{coscolor}\cos \alpha} = 1/\sqrt 3.
    \]%
    };
\end{tikzpicture}

\end{document}
```

tikz demo7

```
% demo7
\documentclass[tikz]{standalone}

\usetikzlibrary {angles,calc,quotes}

\begin{document}
\begin{tikzpicture}[angle radius=.75cm]

\node (A) at (-2,0)     [red,left]   {$A$};
\node (B) at ( 3,.5)    [red,right]  {$B$};
\node (C) at (-2,2)     [blue,left]  {$C$};
\node (D) at ( 3,2.5)   [blue,right] {$D$};
\node (E) at (60:-5mm)  [below]      {$E$};
\node (F) at (60:3.5cm) [above]      {$F$};

\coordinate (X) at (intersection cs:first line={(A)--(B)}, second line=
{(E)--(F)});
\coordinate (Y) at (intersection cs:first line={(C)--(D)}, second line=
{(E)--(F)});

\path
    (A) edge [red, thick]  (B)
    (C) edge [blue, thick] (D)
    (E) edge [thick]       (F)
    pic ["$\alpha$", draw, fill=yellow]   {angle = F--X--A}
    pic ["$\beta$",  draw, fill=green!30] {angle = B--X--F}
    pic ["$\gamma$", draw, fill=yellow]   {angle = E--Y--D}
    pic ["$\delta$", draw, fill=green!30] {angle = C--Y--E};

\node at ($ (D)!.5!(B) $) [right=1cm,text width=6cm,rounded
corners,fill=red!20,inner sep=1ex]
    {
    When we assume that $\color{red}AB$ and $\color{blue}CD$ are
    parallel, i.\,e., ${\color{red}AB} \mathbin{\|} \color{blue}CD$,
    then $\alpha = \gamma$ and $\beta = \delta$.
    };
\end{tikzpicture}
\end{document}
```

tikz demo8

```
% demo8
\documentclass[tikz]{standalone}

\usepackage{chemfig}
\begin{document}

\chemfig{[:-90]HN(-[::-45](-[::-45]R)=[::+45]O)>[::+45]*4(-(=O)-N*5(-(<:(=
[::-60]O)-[::+60]OH)-(<[::+0])(<:[::-108])-S>)--)}

\end{document}
```

tikz demo9

```
\documentclass[tikz]{standalone}

\begin{document}

\begin{tikzpicture}

  \node (so32) [align=center] at (-5,-1) {heterotic\\$SO(32)$};
  \node (e8e8) [align=center] at (-3,4) {heterotic\\$E(8) \times E(8)$};
  \node (tiia) [align=center] at (4,3) {Type II A};
  \node (tiib) [align=center] at (5,-2) {Type II B};
  \node (ti) [align=center] at (0,-5) {Type I};

  \draw[bend left,<->] (so32) to node [below right,align=center] {compac-
\\tification} (e8e8);
  \draw[bend left,<->] (e8e8) to node [below left] {M-theory} (tiia);
  \draw[bend left,<->] (tiia) to node [below left] {T-duality} (tiib);
  \draw[bend left,<->] (tiib) to node [above left,align=center]
{orientifold\\action $\Omega$} (ti);
  \draw[bend left,<->] (ti) to node [above right] {S-duality} (so32);

  \begin{scope}
    \clip[bend right]
    (so32.east)
    to (e8e8.south)
    to (tiia.south)
    to (tiib.west)
    to (ti.north)
    to (so32.east);
    \foreach \c in
{so32.east,e8e8.south,tiia.south,tiib.west,ti.north,so32.east}{%
        \foreach \r in {1,...,6}{%
            \draw[DarkBlue] (\c) circle (\r*0.15cm);
          }
      }
  \end{scope}

  \draw[bend right,very thick,gray,fill,fill opacity=0.3] (so32.east) to
(e8e8.south) to (tiia.south) to (tiib.west) to (ti.north) to (so32.east);

  \node (mth) [align=center] at (0,0) {parameter space of\\[2ex]{\Large
\textbf{M-Theory}}};

\end{tikzpicture}
```

```
\end{document}
```

TexNote.

```
# latex    c/c++/rust
\#      ==> \\\#
\$      ==> \\\$
\%      ==> \\\%
\&      ==> \\\&
\_      ==> \\\_
\{      ==> \\\{
\}      ==> \\\}
\~{}    ==> \\\~\{\}
\^{}    ==> \\\^\{\}
\       ==> \\
\<      ==> \\\<
\>      ==> \\\>
```

Latex Document.

Latex demo

test in margin here

LaTeX.js Showcase

made with ♥ by Michael Brade

2017–2021

Abstract

This document will show most of the features of LaTeX.js while at the same time being a gentle introduction to LaTeX. In the appendix, the API as well as the format of custom macro definitions in JavaScript will be explained.

1 Characters

It is possible to input any UTF-8 character either directly or by character code using one of the following:

- `\symbol{"00A9}`: ©
- `\char"A9`: ©
- `^^A9 or ^^^^00A9`: © or ©

Special characters, like those:

$ & % # _ { } ~ ^ \

have to be escaped. More than 200 symbols are accessible through macros. For instance: 30 °C is 86 °F. See also Section 11.

2 Spaces and Comments

Spaces and comments, of course, work just as they do in LaTeX. This is an example: Supercalifragilisticexpialidocious It does not matter whether you enter one or several spaces after a word, it will always be typeset as one space—unless you force several spaces, like now. New TeXusers may miss whitespaces after a command. Experienced TeX users are TeXperts, and know how to use whitespaces. Longer com-

ments can be embedded in the `comment` environment: This is another example for embedding comments in your document.

3 Dashes and Hyphens

LaTeX knows four kinds of dashes. Access three of them with different numbers of consecutive dashes. The fourth sign is actually not a dash at all—it is the mathematical minus sign:

daughter-in-law, X-rated
pages 13–67
yes—or no?
0, 1 and −1

The names for these dashes are: '-' hyphen, '–' en-dash, '—' em-dash, and '−' minus sign. LaTeX.js outputs the actual true unicode character for those instead of using the hypen-minus.

4 Text and Paragraphs, Ligatures

An empty line starts a new paragraph, and so does `\par`.

Like this. A new line usually starts automatically when the previous one is full. However, using `\newline` or `\\`,
one can force
to start a new line. Ligatures are supported as well, for instance:

fi, fl, ff, ffi, ffl ... instead of fi, fl, ff ...

Use `\/` to prevent a ligature.

4.1 Multicolumns

The multi-column layout, using the `multicols` environment, allows easy definition of multiple columns of text—just like in newspapers. The first and mandatoriy argument specifies the number of columns the text should be divided into. It is often convenient to spread some text over all columns, just before the multicolumn output. In LaTeX, this was needed to prevent any page break in between. To achieve this, the `multicols` environment has an optional second argument which can be used for this purpose. For instance, this text you are reading now was started with the argument `\subsection{Multicolumns}`.

5 Boxes

LaTeX.js supports most of the standard LaTeX boxes.

`\mbox{`*text*`}`

We already know one of them: it's called `\mbox`. It simply packs up a series of boxes into another one, and can be used to prevent LaTeX.js from breaking two words. As boxes can be put inside boxes, these horizontal box packers give you ultimate flexibility.

`\makebox[`*width*`][`*pos*`]{`*text*`}`

`\framebox[`*width*`][`*pos*`]{`*text*`}`

width defines the width of the resulting box as seen from the outside. The *pos* parameter takes a one letter value: **c**enter, flush**l**eft, or flush**r**ight. **s**pread is not really working in HTML. The command `framebox` works exactly the same as `makebox`, but it draws a box around the text. The following example shows you some things you could do with the `makebox` and `framebox` commands.

c e **n** t r a l

Guess I'm framed now!

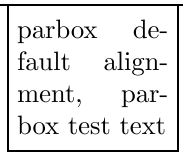
Bummer, I am too wide

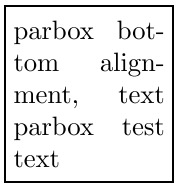
never mind, you need this?

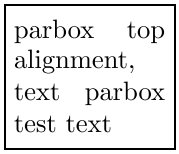
`\parbox[`*pos*`][`*height*`][`*inner-pos*`]{`*width*`}{`*text*`}`

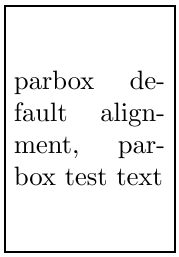
The `\parbox` command produces a box the contents of which are created in paragraph mode. However, only small pieces of text should be used, paragraph-making environments shouldn't be used inside a `\parbox` argument. For larger pieces of text, including ones containing a paragraph-making environment, you should use a `minipage` environment. By default LaTeX will position vertically a parbox so its center lines up with the center of the surrounding text line. When the optional position argument is present and equal either to '`t`' or '`b`', this allows you respectively to align either the top or bottom line in the parbox with the baseline of the surrounding text. You may also specify '`m`' for position to get the default behaviour. The optional *height* argument overrides the natural height of the box. The *inner-pos* argument controls the placement of the text inside the box, as follows; if it is not specified, *pos* is used. `t` text is placed at the top of the box. `c` text is centered in the box. `b` text is placed at the bottom of the box. `s` is not supported in HTML

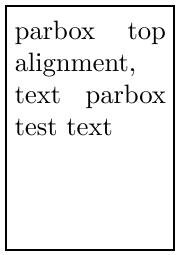
The following examples demonstrate simple positioning:

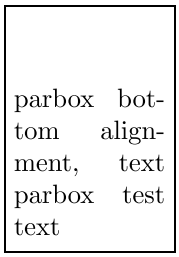
- simple alignment: Some text 

some text 



some text some text.
- alignment with a given height: Some text 

some text 

some text 

some text.

The following examples demonstrate all explicit *pos*/*inner-pos* combinations:

- center alignment: Some text parbox default alignment, parbox test text some text

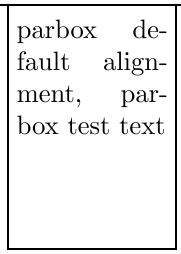
parbox top alignment, text parbox test text some text parbox bottom alignment, text parbox test text some text.

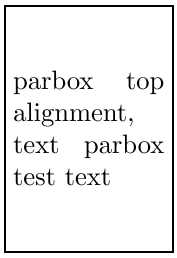
- top alignment: Some text parbox default alignment, parbox test text some text

parbox top alignment, text parbox test text some text parbox bottom alignment, text parbox test text some text.

- bottom alignment: Some text parbox default alignment, parbox test text some text

parbox top alignment, text parbox test text some text parbox bottom alignment, text parbox test text some text.

- top/bottom in one line: Some text 

some text 

some text.

5.1 Low-level box-interface

LaTeX.js supports the following low-level TeX commands: `\llap{`*text*`}`, `\rlap{`*text*`}`, and `\smash{`*text*`}`, as well as `\hphantom{`*text*`}`, `\vphantom{`*text*`}`, and `\phantom{`*text*`}`. A phantom looks like this: yes, now the phantom is gone. `\llap` could be used to put something in the margin. However, there are better alternatives for that. See `\marginpar`.

6 Spacing

The following horizontal spaces are supported:

Negative thin space: ‖
No space (natural): ||
Thin space: | | or | |
Normal space: | |
Normal space: | |
Non-break space: | |
en-space: | |
em-space: | |
2x em-space: | |
3cm horizontal space: | |

7 Environments

7.1 Lists: Itemize, Enumerate, and Description

The `itemize` environment is suitable for simple lists, the `enumerate` environment for enumerated lists, and the `description` environment for descriptions.

1. You can nest the list environments to your taste:
   - But it might start to look silly.

- With a dash.

2. Therefore remember:

   Stupid things will not become smart because they are in a list.

   Smart things, though, can be presented beautifully in a list.

important Technical note: Viewing this in Chrome, however, will show too much vertical space at the end of a nested environment (see above). On top of that, margin collapsing for inline-block boxes is not allowed. Maybe using `dl` elements is too complicated for this and a simple nested `div` should be used instead.

Lists can be deeply nested:

- list text, level one
  - list text, level two
    - list text, level three And a new paragraph can be started, too.
      - list text, level four And a new paragraph can be started, too. This is the maximum level.
      - list text, level four
    - list text, level three
  - list text, level two
- list text, level one
- list text, level one

7.2 Flushleft, Flushright, and Center

The `flushleft` environment:

This text is
left-aligned. LaTeX is not trying to make each line the same length.

The `flushright` environment:

This text is right-aligned. LaTeX is not trying to make each line the same length.

And the `center` environment:

At the centre
of the earth

7.3 Quote, Quotation, and Verse

The `quote` environment is useful for quotes, important phrases and examples. A typographical rule of thumb for the

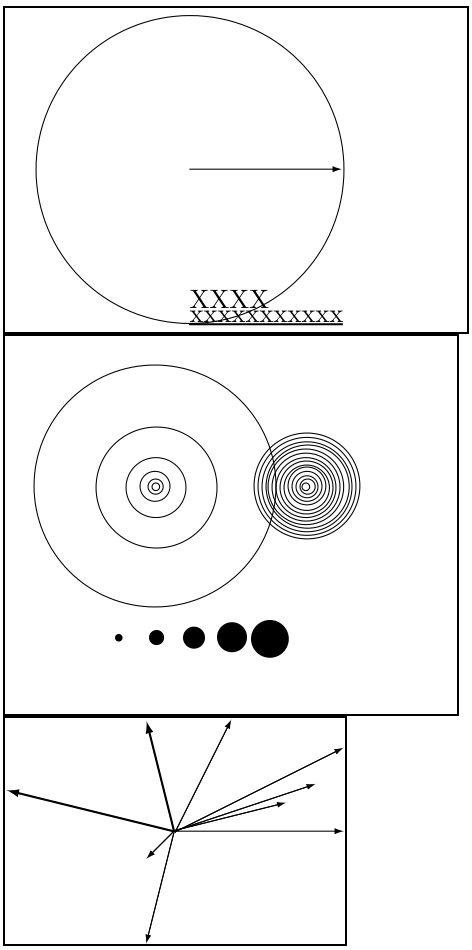
line length is:

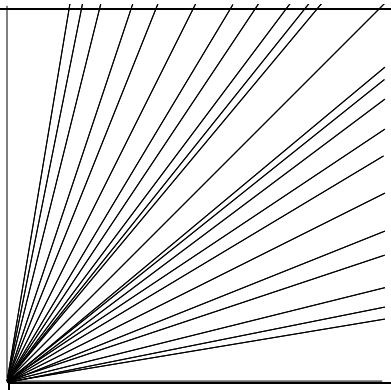
> On average, no line should be longer than 66 characters.

There are two similar environments: the `quotation` and the `verse` environments. The `quotation` environment is useful for longer quotes going over several paragraphs, because it indents the first line of each paragraph. The `verse` environment is useful for poems where the line breaks are important. The lines are separated by issuing a \\ at the end of a line and an empty line after each verse.

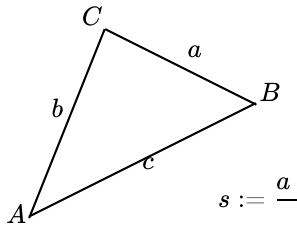
Humpty Dumpty sat on a wall:
Humpty Dumpty had a great fall.
All the King's horses and all the King's men
Couldn't put Humpty together again. —J.W. Elliott

7.4 Picture

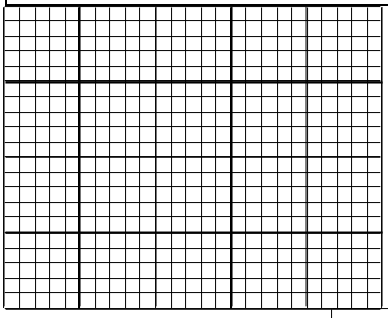


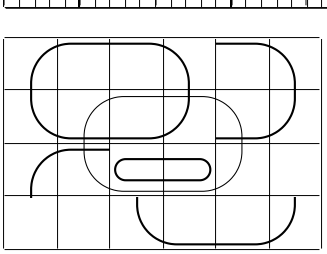


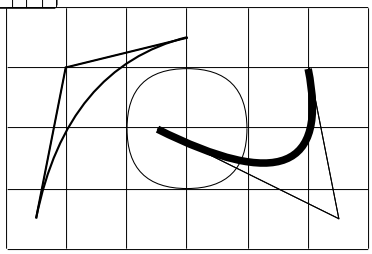
$$F = \sqrt{s(s-a)(s-b)(s-c)}$$



$$s := \frac{a+b+c}{2}$$







8 Labels and References

Labels can be attached to parts, chapters, sections, items of enumerations, footnotes, tables and figures. For instance: item 2 was important, and regarding fonts, read Section 12.1. And below, we can reference item 1(a)iB and 1b.

1. list text, level one
   (a) list text, level two

i. list text, level three And a new paragraph can be started, too.
   A. list text, level four And a new paragraph can be started, too. This is the maximum level.
   B. list text, level four
ii. list text, level three

(b) list text, level two

2. list text, level one
3. list text, level one

9 Mathematical Formulae

Math is typeset using KaTeX. Inline math: $f(x) = \int_{-\infty}^{\infty} \hat{f}(\xi)\, e^{2\pi i \xi x}\, d\xi$ as well as display math is supported:

$$f(n) = \begin{cases} \frac{n}{2}, & \text{if } n \text{ is even} \\ 3n+1, & \text{if } n \text{ is odd} \end{cases}$$

10 Groups

Today is Monday, April 28, 2025. Actually, what about some groups? They are nice.

11 Symbols

lowercase greek letters: αβγδεζηϑικλμνξοπρστυφχψωuppercase greek letters: ΑΒΓΔΕΖΗΘΙΚΛΜΝΞΟΠΡΣΤΥΦΧΨΩcurrencies: €¢£₤฿₡¤₫ƒ₤₦₱₩¥old-style numerals: 0123456789math: ‰ ‱ ‰‱ $\frac{131}{244}$ /÷×−±√¬∗^{123}arrows: ←↑→↓misc: ✓※ ª º‰‱ǀǁ ◯ ⒸⒸⓅⓇ℠ № ℞℮ ♪ ⅞non-ASCII: ÆæIJijŒœÞþßẞÐðØøĐđŁłŋJŋ

12 Fonts

Usually, LaTeX.js chooses the right font—just like LaTeX. In some cases, one might like to change fonts and sizes by hand. To do this, use the standard commands. The actual size of each font is a design issue and depends on the document class (in this case on the CSS file). The small and **bold** Romans ruled all of great big *Italy. You can also* emphasize *text if it is set in italics,* in a *sans-serif* font, `or in` *`typewriter`* `style`. The environment form of the font commands is available, too:

This whole paragraph is emphasized, for instance.

12.1 An advice

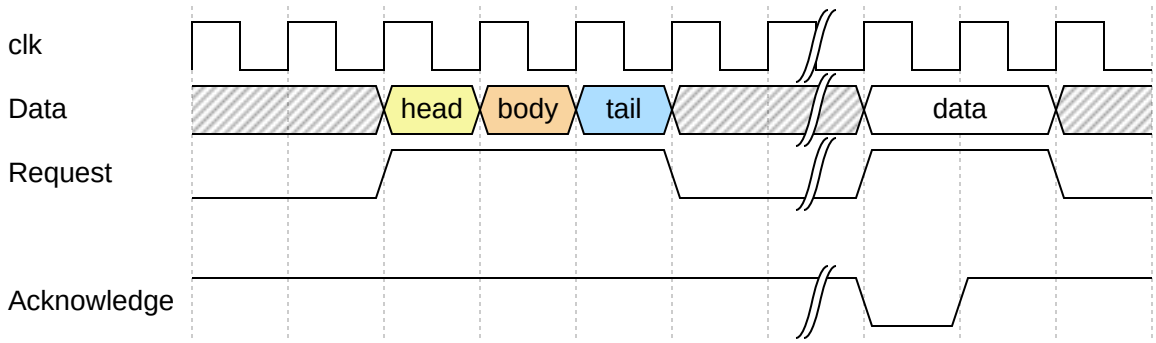
Remember! *The* MORE fonts you use in a `document`, *the* more READABLE and *beautiful it* becomes.

A Source

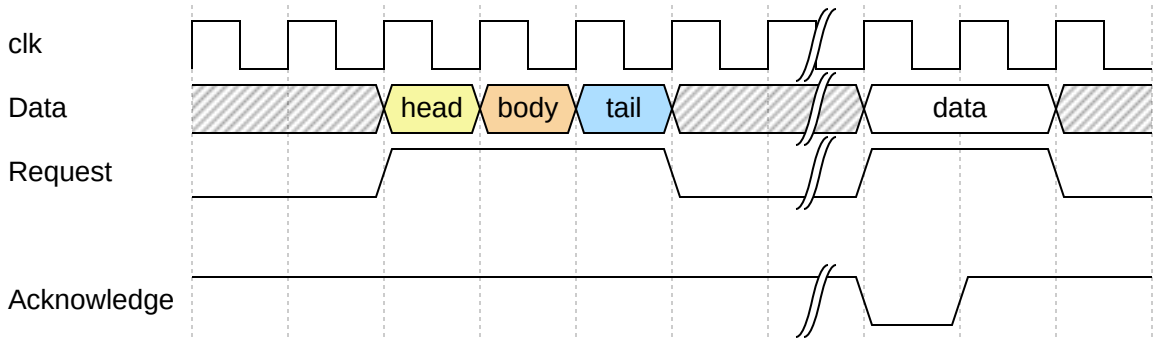
The source of LaTeX.js is here on GitHub: https://github.com/michael-brade/LaTeX.js

WaveDrom

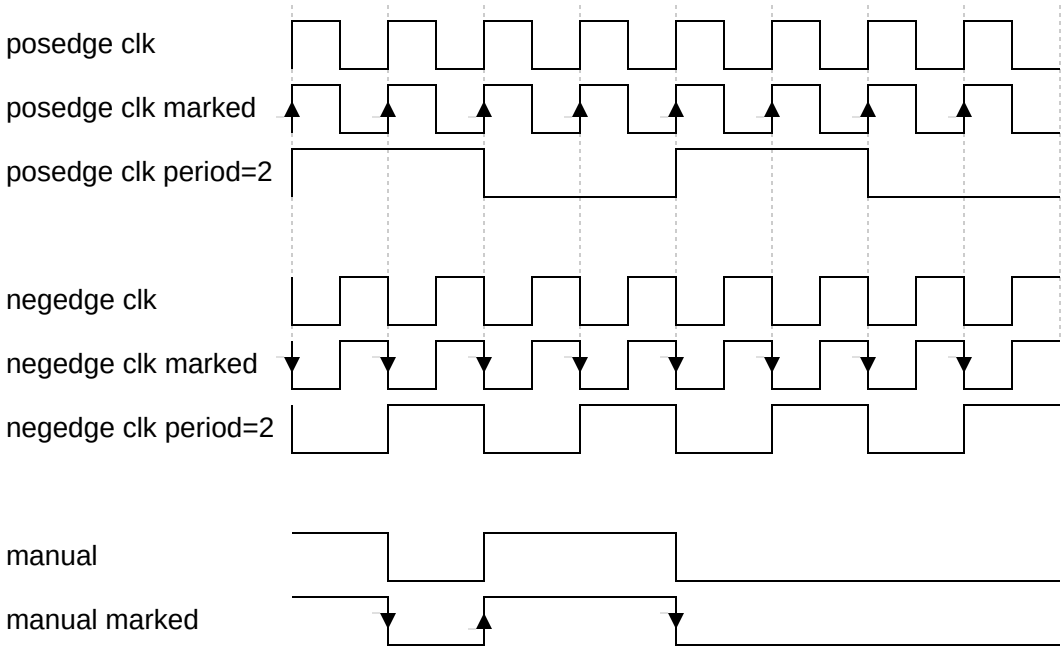
basic1



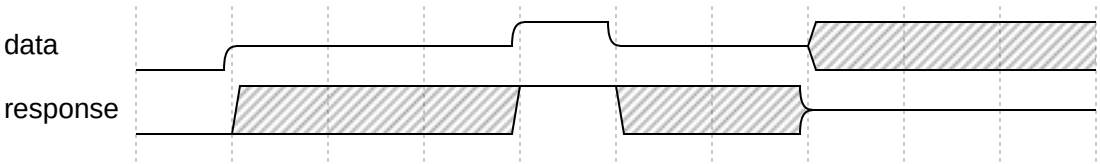
basic2



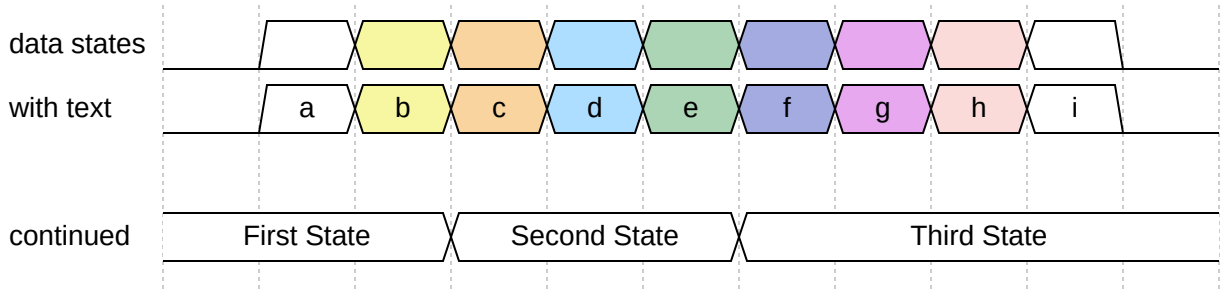
clock



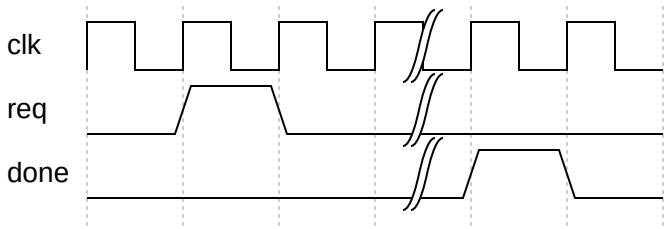
High Impedance & Undefined



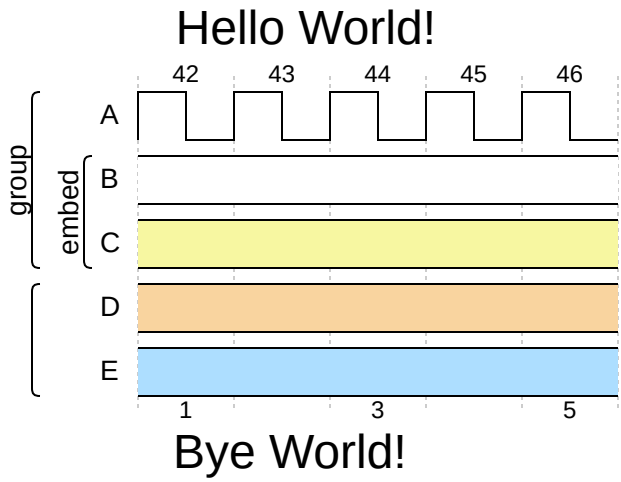
Data types



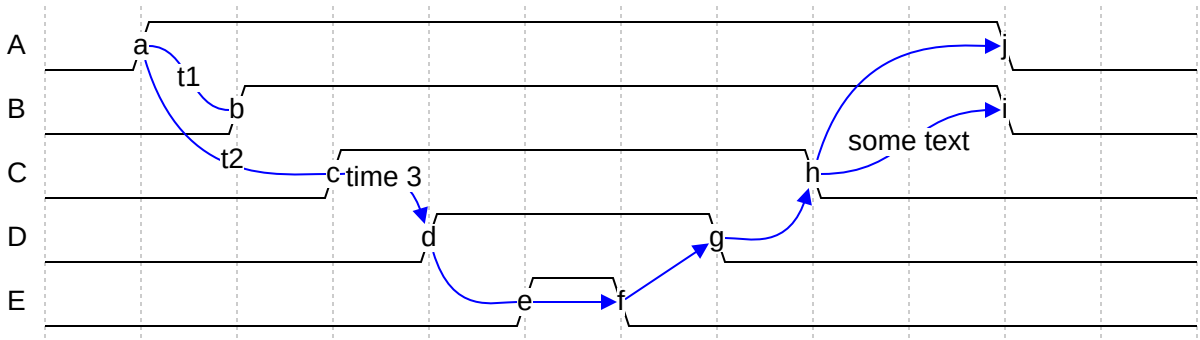
Graps



Groups



Edges



emoji-cheat-sheet

This cheat sheet is automatically generated from GitHub Emoji API and Unicode Full Emoji List.

Table of Contents

Smileys & Emotion

Face Smiling

ico	shortcode	ico	shortcode
😀	`grinning`	😃	`smiley`
😄	`smile`	😁	`grin`
😆	`laughing` `satisfied`	😅	`sweat_smile`
🤣	`rofl`	😂	`joy`
🙂	`slightly_smiling_face`	🙃	`upside_down_face`
😉	`wink`	😊	`blush`
😇	`innocent`		

Face Affection

ico	shortcode	ico	shortcode
🥰	`smiling_face_with_three_hearts`	😍	`heart_eyes`
🤩	`star_struck`	😘	`kissing_heart`
😗	`kissing`	☺️	`relaxed`
😚	`kissing_closed_eyes`	😙	`kissing_smiling_eyes`
🥲	`smiling_face_with_tear`		

Face Tongue

ico	shortcode	ico	shortcode
😋	yum	😛	stuck_out_tongue
😜	stuck_out_tongue_winking_eye	🤪	zany_face
😝	stuck_out_tongue_closed_eyes	🤑	money_mouth_face

Face Hand

ico	shortcode	ico	shortcode
🤗	hugs	🤭	hand_over_mouth
🤫	shushing_face	🤔	thinking

Face Neutral Skeptical

ico	shortcode	ico	shortcode
🤐	zipper_mouth_face	🤨	raised_eyebrow
😐	neutral_face	😑	expressionless
😶	no_mouth	🌫	face_in_clouds
😏	smirk	😒	unamused
🙄	roll_eyes	😬	grimacing
😮‍💨	face_exhaling	🤥	lying_face

Face Sleepy

ico	shortcode	ico	shortcode
😌	relieved	😔	pensive
😪	sleepy	🤤	drooling_face
😴	sleeping		

Face Unwell

ico	shortcode	ico	shortcode
😷	mask	🤒	face_with_thermometer
🤕	face_with_head_bandage	🤢	nauseated_face
🤮	vomiting_face	🤧	sneezing_face
🥵	hot_face	🥶	cold_face
🥴	woozy_face	😵	dizzy_face
😵‍💫	face_with_spiral_eyes	🤯	exploding_head

Face Hat

ico	shortcode	ico	shortcode
🤠	cowboy_hat_face	🥳	partying_face
🥸	disguised_face		

Face Glasses

ico	shortcode	ico	shortcode
😎	sunglasses	🤓	nerd_face
🧐	monocle_face		

Face Concerned

ico	shortcode	ico	shortcode
😕	confused	😟	worried
🙁	slightly_frowning_face	☹️	frowning_face
😮	open_mouth	😯	hushed
😲	astonished	😳	flushed

ico	shortcode	ico	shortcode
🥺	pleading_face	😦	frowning
😧	anguished	😨	fearful
😰	cold_sweat	😥	disappointed_relieved
😢	cry	😭	sob
😱	scream	😖	confounded
😣	persevere	😞	disappointed
😓	sweat	😩	weary
😫	tired_face	🥱	yawning_face

Face Negative

ico	shortcode	ico	shortcode
😤	triumph	😡	pout rage
😠	angry	🤬	cursing_face
😈	smiling_imp	👿	imp
💀	skull	☠️	skull_and_crossbones

Face Costume

ico	shortcode	ico	shortcode
💩	hankey poop shit	🤡	clown_face
👹	japanese_ogre	👺	japanese_goblin
👻	ghost	👽	alien
👾	space_invader	🤖	robot

Cat Face

ico	shortcode	ico	shortcode
😺	smiley_cat	😸	smile_cat
😹	joy_cat	😻	heart_eyes_cat
😼	smirk_cat	😽	kissing_cat
🙀	scream_cat	😿	crying_cat_face
😾	pouting_cat		

Monkey Face

ico	shortcode	ico	shortcode
🙈	see_no_evil	🙉	hear_no_evil
🙊	speak_no_evil		

Heart

ico	shortcode	ico	shortcode
💌	love_letter	💘	cupid
💝	gift_heart	💖	sparkling_heart
💗	heartpulse	💓	heartbeat
💞	revolving_hearts	💕	two_hearts
💟	heart_decoration	❣️	heavy_heart_exclamation
💔	broken_heart	❤️‍🔥	heart_on_fire
❤️‍🩹	mending_heart	❤️	heart
🧡	orange_heart	💛	yellow_heart
💚	green_heart	💙	blue_heart
💜	purple_heart	🤎	brown_heart
🖤	black_heart	🤍	white_heart

Emotion

ico	shortcode	ico	shortcode
💋	`kiss`	💯	`100`
💢	`anger`	💥	`boom` `collision`
💫	`dizzy`	💦	`sweat_drops`
💨	`dash`	🕳️	`hole`
💬	`speech_balloon`	👁️‍🗨️	`eye_speech_bubble`
🗨️	`left_speech_bubble`	💢	`right_anger_bubble`
💭	`thought_balloon`	💤	`zzz`

People & Body

- Hand Fingers Open
- Hand Fingers Partial
- Hand Single Finger
- Hand Fingers Closed
- Hands
- Hand Prop
- Body Parts
- Person
- Person Gesture
- Person Role
- Person Fantasy
- Person Activity
- Person Sport
- Person Resting
- Family
- Person Symbol

Hand Fingers Open

ico	shortcode	ico	shortcode
👋	`wave`	🤚	`raised_back_of_hand`
🖐	`raised_hand_with_fingers_splayed`	✋	`hand` `raised_hand`
🖖	`vulcan_salute`		

Hand Fingers Partial

ico	shortcode	ico	shortcode
👌	`ok_hand`	🤌	`pinched_fingers`
🤏	`pinching_hand`	✌	`v`
🤞	`crossed_fingers`	🤟	`love_you_gesture`
🤘	`metal`	🤙	`call_me_hand`

Hand Single Finger

ico	shortcode	ico	shortcode
👈	`point_left`	👉	`point_right`
👆	`point_up_2`	🖕	`fu` `middle_finger`
👇	`point_down`	☝	`point_up`

Hand Fingers Closed

ico	shortcode	ico	shortcode
👍	`+1` `thumbsup`	👎	`-1` `thumbsdown`

ico	shortcode	ico	shortcode

✊	`fist`
	`fist_raised`
		👊	`facepunch`
			`fist_oncoming`
			`punch`

🤛	`fist_left`	🤜	`fist_right`

Hands

ico	shortcode	ico	shortcode
👏	`clap`	🙌	`raised_hands`
👐	`open_hands`	🤲	`palms_up_together`
🤝	`handshake`	🙏	`pray`

Hand Prop

ico	shortcode	ico	shortcode
✍️	`writing_hand`	💅	`nail_care`
🤳	`selfie`		

Body Parts

ico	shortcode	ico	shortcode
💪	`muscle`	🦾	`mechanical_arm`
🦿	`mechanical_leg`	🦵	`leg`
🦶	`foot`	👂	`ear`
🦻	`ear_with_hearing_aid`	👃	`nose`
🧠	`brain`	🫀	`anatomical_heart`
🫁	`lungs`	🦷	`tooth`
🦴	`bone`	👀	`eyes`

ico	shortcode	ico	shortcode
👁️	eye	👅	tongue
👄	lips		

Person

ico	shortcode	ico	shortcode
👶	baby	🧒	child
👦	boy	👧	girl
🧑	adult	👱	blond_haired_person
👨	man	🧔	bearded_person
👨‍🦰	man_beard	👩‍🦰	woman_beard
👨‍🦰	red_haired_man	👨‍🦱	curly_haired_man
👨‍🦳	white_haired_man	👨‍🦲	bald_man
👩	woman	👩‍🦰	red_haired_woman
🧑‍🦰	person_red_hair	👩‍🦱	curly_haired_woman
🧑‍🦱	person_curly_hair	👩‍🦳	white_haired_woman
🧑‍🦳	person_white_hair	👩‍🦲	bald_woman
🧑‍🦲	person_bald	👱‍♀️	blond_haired_woman blonde_woman
👱‍♂️	blond_haired_man	🧓	older_adult
👴	older_man	👵	older_woman

Person Gesture

ico	shortcode	ico	shortcode
🙍	frowning_person	🙍‍♂️	frowning_man
🙍‍♀️	frowning_woman	🙎	pouting_face
🙎‍♂️	pouting_man	🙎‍♀️	pouting_woman

ico	shortcode	ico	shortcode
🙅	no_good	🙅‍♂️	ng_man no_good_man
🙅‍♀️	ng_woman no_good_woman	🙆	ok_person
🙆‍♂️	ok_man	🙆‍♀️	ok_woman
💁	information_desk_person tipping_hand_person	💁‍♂️	sassy_man tipping_hand_man
💁‍♀️	sassy_woman tipping_hand_woman	🙋	raising_hand
🙋‍♂️	raising_hand_man	🙋‍♀️	raising_hand_woman
🧏	deaf_person	🧏‍♂️	deaf_man
🧏‍♀️	deaf_woman	🙇	bow
🙇‍♂️	bowing_man	🙇‍♀️	bowing_woman
🤦	facepalm	🤦‍♂️	man_facepalming
🤦‍♀️	woman_facepalming	🤷	shrug
🤷‍♂️	man_shrugging	🤷‍♀️	woman_shrugging

Person Role

ico	shortcode	ico	shortcode
🧑‍⚕️	health_worker	👨‍⚕️	man_health_worker
👩‍⚕️	woman_health_worker	🧑‍🎓	student
👨‍🎓	man_student	👩‍🎓	woman_student
🧑‍🏫	teacher	👨‍🏫	man_teacher
👩‍🏫	woman_teacher	🧑‍⚖️	judge
👨‍⚖️	man_judge	👩‍⚖️	woman_judge
🧑‍🌾	farmer	👨‍🌾	man_farmer
👩‍🌾	woman_farmer	🧑‍🍳	cook

ico	shortcode	ico	shortcode
👨‍🍳	man_cook	👩‍🍳	woman_cook
👨‍🔧	mechanic	👨‍🔧	man_mechanic
👩‍🔧	woman_mechanic	👨‍🏭	factory_worker
👨‍🏭	man_factory_worker	👩‍🏭	woman_factory_worker
🧑‍💼	office_worker	👨‍💼	man_office_worker
👩‍💼	woman_office_worker	👩‍🔬	scientist
👨‍🔬	man_scientist	👩‍🔬	woman_scientist
🧑‍💻	technologist	👨‍💻	man_technologist
👩‍💻	woman_technologist	🧑‍🎤	singer
👨‍🎤	man_singer	👩‍🎤	woman_singer
🧑‍🎨	artist	👨‍🎨	man_artist
👩‍🎨	woman_artist	👨‍✈️	pilot
👨‍✈️	man_pilot	👩‍✈️	woman_pilot
🧑‍🚀	astronaut	👨‍🚀	man_astronaut
👩‍🚀	woman_astronaut	🧑‍🚒	firefighter
👨‍🚒	man_firefighter	👩‍🚒	woman_firefighter
👮	cop police_officer	👮‍♂️	policeman
👮‍♀️	policewoman	🕵️	detective
🕵️‍♂️	male_detective	🕵️‍♀️	female_detective
💂	guard	💂‍♂️	guardsman
💂‍♀️	guardswoman	🥷	ninja
👷	construction_worker	👷‍♂️	construction_worker_man
👷‍♀️	construction_worker_woman	🤴	prince
👸	princess	👳	person_with_turban
👳‍♂️	man_with_turban	👳‍♀️	woman_with_turban
👲	man_with_gua_pi_mao	🧕	woman_with_headscarf

ico	shortcode	ico	shortcode
🤵	person_in_tuxedo	🤵‍♂️	man_in_tuxedo
🤵‍♀️	woman_in_tuxedo	👰	person_with_veil
👰‍♂️	man_with_veil	👰‍♀️	bride_with_veil woman_with_veil
🤰	pregnant_woman	🤱	breast_feeding
👩‍🍼	woman_feeding_baby	👨‍🍼	man_feeding_baby
🧑‍🍼	person_feeding_baby		

Person Fantasy

ico	shortcode	ico	shortcode
👼	angel	🎅	santa
🤶	mrs_claus	🧑‍🎄	mx_claus
🦸	superhero	🦸‍♂️	superhero_man
🦸‍♀️	superhero_woman	🦹	supervillain
🦹‍♂️	supervillain_man	🦹‍♀️	supervillain_woman
🧙	mage	🧙‍♂️	mage_man
🧙‍♀️	mage_woman	🧚	fairy
🧚‍♂️	fairy_man	🧚‍♀️	fairy_woman
🧛	vampire	🧛‍♂️	vampire_man
🧛‍♀️	vampire_woman	🧜	merperson
🧜‍♂️	merman	🧜‍♀️	mermaid
🧝	elf	🧝‍♂️	elf_man
🧝‍♀️	elf_woman	🧞	genie
🧞‍♂️	genie_man	🧞‍♀️	genie_woman
🧟	zombie	🧟‍♂️	zombie_man
🧟‍♀️	zombie_woman		

Person Activity

ico	shortcode	ico	shortcode
💆	massage	💆‍♂️	massage_man
💆‍♀️	massage_woman	💇	haircut
💇‍♂️	haircut_man	💇‍♀️	haircut_woman
🚶	walking	🚶‍♂️	walking_man
🚶‍♀️	walking_woman	🧍	standing_person
🧍‍♂️	standing_man	🧍‍♀️	standing_woman
🧎	kneeling_person	🧎‍♂️	kneeling_man
🧎‍♀️	kneeling_woman	🧑‍🦯	person_with_probing_car
👨‍🦯	man_with_probing_cane	👩‍🦯	woman_with_probing_cane
🧑‍🦼	person_in_motorized_wheelchair	👨‍🦼	man_in_motorized_wheelc
👩‍🦼	woman_in_motorized_wheelchair	🧑‍🦽	person_in_manual_wheelc
👨‍🦽	man_in_manual_wheelchair	👩‍🦽	woman_in_manual_wheelch
🏃	runner running	🏃‍♂️	running_man
🏃‍♀️	running_woman	💃	dancer woman_dancing
🕺	man_dancing	🕴️	business_suit_levitatir
👯	dancers	👯‍♂️	dancing_men
👯‍♀️	dancing_women	🧖	sauna_person
🧖‍♂️	sauna_man	🧖‍♀️	sauna_woman
🧗	climbing	🧗‍♂️	climbing_man
🧗‍♀️	climbing_woman		

Person Sport

ico	shortcode	ico	shortcode
🤺	person_fencing	🏇	horse_racing
⛷️	skier	🏂	snowboarder
🏌️	golfing	🏌️‍♂️	golfing_man
🏌️‍♀️	golfing_woman	🏄	surfer
🏄‍♂️	surfing_man	🏄‍♀️	surfing_woman
🚣	rowboat	🚣‍♂️	rowing_man
🚣‍♀️	rowing_woman	🏊	swimmer
🏊‍♂️	swimming_man	🏊‍♀️	swimming_woman
⛹️	bouncing_ball_person	⛹️‍♂️	basketball_man bouncing_ball_man
⛹️‍♀️	basketball_woman bouncing_ball_woman	🏋️	weight_lifting
🏋️‍♂️	weight_lifting_man	🏋️‍♀️	weight_lifting_woman
🚴	bicyclist	🚴‍♂️	biking_man
🚴‍♀️	biking_woman	🚵	mountain_bicyclist
🚵‍♂️	mountain_biking_man	🚵‍♀️	mountain_biking_woman
🤸	cartwheeling	🤸‍♂️	man_cartwheeling
🤸‍♀️	woman_cartwheeling	🤼	wrestling
🤼‍♂️	men_wrestling	🤼‍♀️	women_wrestling
🤽	water_polo	🤽‍♂️	man_playing_water_polo
🤽‍♀️	woman_playing_water_polo	🤾	handball_person
🤾‍♂️	man_playing_handball	🤾‍♀️	woman_playing_handball
🤹	juggling_person	🤹‍♂️	man_juggling
🤹‍♀️	woman_juggling		

Person Resting

ico	shortcode	ico	shortcode
🧘	lotus_position	🧘‍♂️	lotus_position_man
🧘‍♀️	lotus_position_woman	🛀	bath
🛌	sleeping_bed		

Family

ico	shortcode	ico	shortcode
👫	people_holding_hands	👭	two_women_holding_hands
👫	couple	👬	two_men_holding_hands
💏	couplekiss	💏	couplekiss_man_woman
💏	couplekiss_man_man	💏	couplekiss_woman_woman
💑	couple_with_heart	💑	couple_with_heart_woman_m
💑	couple_with_heart_man_man	💑	couple_with_heart_woman_w
👪	family_man_woman_boy	👪	family_man_woman_girl
👪	family_man_woman_girl_boy	👪	family_man_woman_boy_boy
👪	family_man_woman_girl_girl	👪	family_man_man_boy
👪	family_man_man_girl	👪	family_man_man_girl_boy
👪	family_man_man_boy_boy	👪	family_man_man_girl_girl
👪	family_woman_woman_boy	👪	family_woman_woman_girl
👪	family_woman_woman_girl_boy	👪	family_woman_woman_boy_bo
👪	family_woman_woman_girl_girl	👪	family_man_boy
👪	family_man_boy_boy	👪	family_man_girl
👪	family_man_girl_boy	👪	family_man_girl_girl
👪	family_woman_boy	👪	family_woman_boy_boy
👪	family_woman_girl	👪	family_woman_girl_boy

ico	shortcode	ico	shortcode

Person Symbol

ico	shortcode	ico	shortcode
🗣️	`speaking_head`	👤	`bust_in_silhouette`
👥	`busts_in_silhouette`	🫂	`people_hugging`
👪	`family`	👣	`footprints`

Animals & Nature

Animal Mammal

ico	shortcode	ico	shortcode
🐵	`monkey_face`	🐒	`monkey`
🦍	`gorilla`	🦧	`orangutan`
🐶	`dog`	🐕	`dog2`
🦮	`guide_dog`	🐕‍🦺	`service_dog`
🐩	`poodle`	🐺	`wolf`
🦊	`fox_face`	🦝	`raccoon`

ico	shortcode	ico	shortcode
🐱	cat	🐈	cat2
🐈‍⬛	black_cat	🦁	lion
🐯	tiger	🐅	tiger2
🐆	leopard	🐴	horse
🐎	racehorse	🦄	unicorn
🦓	zebra	🦌	deer
🦬	bison	🐮	cow
🐂	ox	🐃	water_buffalo
🐄	cow2	🐷	pig
🐖	pig2	🐗	boar
🐽	pig_nose	🐏	ram
🐑	sheep	🐐	goat
🐪	dromedary_camel	🐫	camel
🦙	llama	🦒	giraffe
🐘	elephant	🦣	mammoth
🦏	rhinoceros	🦛	hippopotamus
🐭	mouse	🐁	mouse2
🐀	rat	🐹	hamster
🐰	rabbit	🐇	rabbit2
🐿️	chipmunk	🦫	beaver
🦔	hedgehog	🦇	bat
🐻	bear	🐻‍❄️	polar_bear
🐨	koala	🐼	panda_face
🦥	sloth	🦦	otter
🦨	skunk	🦘	kangaroo
🦡	badger	🐾	feet paw_prints

Animal Bird

ico	shortcode	ico	shortcode
🦃	`turkey`	🐔	`chicken`
🐓	`rooster`	🐣	`hatching_chick`
🐤	`baby_chick`	🐥	`hatched_chick`
🐦	`bird`	🐧	`penguin`
🕊️	`dove`	🦅	`eagle`
🦆	`duck`	🦢	`swan`
🦉	`owl`	🦤	`dodo`
🪶	`feather`	🦩	`flamingo`
🦚	`peacock`	🦜	`parrot`

Animal Amphibian

ico	shortcode
🐸	`frog`

Animal Reptile

ico	shortcode	ico	shortcode
🐊	`crocodile`	🐢	`turtle`
🦎	`lizard`	🐍	`snake`
🐲	`dragon_face`	🐉	`dragon`
🦕	`sauropod`	🦖	`t-rex`

Animal Marine

ico	shortcode	ico	shortcode
🐳	whale	🐋	whale2
🐬	dolphin flipper	🦭	seal
🐟	fish	🐠	tropical_fish
🐡	blowfish	🦈	shark
🐙	octopus	🐚	shell

Animal Bug

ico	shortcode	ico	shortcode
🐌	snail	🦋	butterfly
🐛	bug	🐜	ant
🐝	bee honeybee	🪲	beetle
🐞	lady_beetle	🦗	cricket
🪳	cockroach	🕷	spider
🕸	spider_web	🦂	scorpion
🦟	mosquito	🪰	fly
🪱	worm	🦠	microbe

Plant Flower

ico	shortcode	ico	shortcode
💐	bouquet	🌸	cherry_blossom
💮	white_flower	🏵	rosette
🌹	rose	🥀	wilted_flower

ico	shortcode	ico	shortcode
🌺	`hibiscus`	🌻	`sunflower`
🌼	`blossom`	🌷	`tulip`

Plant Other

ico	shortcode	ico	shortcode
🌱	`seedling`	🪴	`potted_plant`
🌲	`evergreen_tree`	🌳	`deciduous_tree`
🌴	`palm_tree`	🌵	`cactus`
🌾	`ear_of_rice`	🌿	`herb`
☘️	`shamrock`	🍀	`four_leaf_clover`
🍁	`maple_leaf`	🍂	`fallen_leaf`
🍃	`leaves`	🍄	`mushroom`

Food & Drink

- Food Fruit
- Food Vegetable
- Food Prepared
- Food Asian
- Food Marine
- Food Sweet
- Drink
- Dishware

Food Fruit

ico	shortcode	ico	shortcode
🍇	`grapes`	🍈	`melon`

ico	shortcode	ico	shortcode
🍉	watermelon	🍊	mandarin orange tangerine
🍋	lemon	🍌	banana
🍍	pineapple	🥭	mango
🍎	apple	🍏	green_apple
🍐	pear	🍑	peach
🍒	cherries	🍓	strawberry
🫐	blueberries	🥝	kiwi_fruit
🍅	tomato	🫒	olive
🥥	coconut		

Food Vegetable

ico	shortcode	ico	shortcode
🥑	avocado	🍆	eggplant
🥔	potato	🥕	carrot
🌽	corn	🌶️	hot_pepper
🫑	bell_pepper	🥒	cucumber
🥬	leafy_green	🥦	broccoli
🧄	garlic	🧅	onion
🥜	peanuts	🌰	chestnut

Food Prepared

ico	shortcode	ico	shortcode
🍞	bread	🥐	croissant
🥖	baguette_bread	🫓	flatbread

ico	shortcode	ico	shortcode
🥨	pretzel	🥯	bagel
🥞	pancakes	🧇	waffle
🧀	cheese	🍖	meat_on_bone
🍗	poultry_leg	🥩	cut_of_meat
🥓	bacon	🍔	hamburger
🍟	fries	🍕	pizza
🌭	hotdog	🥪	sandwich
🌮	taco	🌯	burrito
🫔	tamale	🥙	stuffed_flatbread
🧆	falafel	🥚	egg
🍳	fried_egg	🥘	shallow_pan_of_food
🍲	stew	🫕	fondue
🥣	bowl_with_spoon	🥗	green_salad
🍿	popcorn	🧈	butter
🧂	salt	🥫	canned_food

Food Asian

ico	shortcode	ico	shortcode
🍱	bento	🍘	rice_cracker
🍙	rice_ball	🍚	rice
🍛	curry	🍜	ramen
🍝	spaghetti	🍠	sweet_potato
🍢	oden	🍣	sushi
🍤	fried_shrimp	🍥	fish_cake
🥮	moon_cake	🍡	dango
🥟	dumpling	🥠	fortune_cookie

ico	shortcode	ico	shortcode
🥡	takeout_box		

Food Marine

ico	shortcode	ico	shortcode
🦀	crab	🦞	lobster
🦐	shrimp	🦑	squid
🦪	oyster		

Food Sweet

ico	shortcode	ico	shortcode
🍦	icecream	🍧	shaved_ice
🍨	ice_cream	🍩	doughnut
🍪	cookie	🎂	birthday
🍰	cake	🧁	cupcake
🥧	pie	🍫	chocolate_bar
🍬	candy	🍭	lollipop
🍮	custard	🍯	honey_pot

Drink

ico	shortcode	ico	shortcode
🍼	baby_bottle	🥛	milk_glass
☕	coffee	🫖	teapot
🍵	tea	🍶	sake
🍾	champagne	🍷	wine_glass
🍸	cocktail	🍹	tropical_drink

ico	shortcode	ico	shortcode
🍺	`beer`	🍻	`beers`
🥂	`clinking_glasses`	🥃	`tumbler_glass`
🥤	`cup_with_straw`	🧋	`bubble_tea`
🧃	`beverage_box`	🧉	`mate`
🧊	`ice_cube`		

Dishware

ico	shortcode	ico	shortcode
🥢	`chopsticks`	🍽️	`plate_with_cutlery`
🍴	`fork_and_knife`	🥄	`spoon`
🔪	`hocho` `knife`	🏺	`amphora`

Travel & Places

- Place Map
- Place Geographic
- Place Building
- Place Religious
- Place Other
- Transport Ground
- Transport Water
- Transport Air
- Hotel
- Time
- Sky & Weather

Place Map

ico	shortcode	ico	shortcode
🌍	earth_africa	🌎	earth_americas
🌏	earth_asia	🌐	globe_with_meridians
🗺️	world_map	🗾	japan
🧭	compass		

Place Geographic

ico	shortcode	ico	shortcode
🏔️	mountain_snow	⛰️	mountain
🌋	volcano	🗻	mount_fuji
🏕️	camping	🏖️	beach_umbrella
🏜️	desert	🏝️	desert_island
🏞️	national_park		

Place Building

ico	shortcode	ico	shortcode
🏟️	stadium	🏛️	classical_building
🏗️	building_construction	🧱	bricks
🪨	rock	🪵	wood
🛖	hut	🏘️	houses
🏚️	derelict_house	🏠	house
🏡	house_with_garden	🏢	office
🏣	post_office	🏤	european_post_office
🏥	hospital	🏦	bank
🏨	hotel	🏩	love_hotel

ico	shortcode	ico	shortcode
🏪	convenience_store	🏫	school
🏬	department_store	🏭	factory
🏯	japanese_castle	🏰	european_castle
💒	wedding	🗼	tokyo_tower
🗽	statue_of_liberty		

Place Religious

ico	shortcode	ico	shortcode
⛪	church	🕌	mosque
🛕	hindu_temple	🕍	synagogue
⛩️	shinto_shrine	🕋	kaaba

Place Other

ico	shortcode	ico	shortcode
⛲	fountain	⛺	tent
🌁	foggy	🌃	night_with_stars
🏙️	cityscape	🌄	sunrise_over_mountains
🌅	sunrise	🌆	city_sunset
🌇	city_sunrise	🌉	bridge_at_night
♨️	hotsprings	🎠	carousel_horse
🎡	ferris_wheel	🎢	roller_coaster
💈	barber	🎪	circus_tent

Transport Ground

ico	shortcode	ico	shortcode
🚂	steam_locomotive	🚃	railway_car
🚄	bullettrain_side	🚅	bullettrain_front
🚆	train2	🚇	metro
🚈	light_rail	🚉	station
🚊	tram	🚝	monorail
🚞	mountain_railway	🚋	train
🚌	bus	🚍	oncoming_bus
🚎	trolleybus	🚐	minibus
🚑	ambulance	🚒	fire_engine
🚓	police_car	🚔	oncoming_police_car
🚕	taxi	🚖	oncoming_taxi
🚗	car red_car	🚘	oncoming_automobile
🚙	blue_car	🛻	pickup_truck
🚚	truck	🚛	articulated_lorry
🚜	tractor	🏎	racing_car
🏍	motorcycle	🛵	motor_scooter
🦽	manual_wheelchair	🦼	motorized_wheelchair
🛺	auto_rickshaw	🚲	bike
🛴	kick_scooter	🛹	skateboard
🛼	roller_skate	🚏	busstop
🛣	motorway	🛤	railway_track
🛢	oil_drum	⛽	fuelpump
🚨	rotating_light	🚥	traffic_light
🚦	vertical_traffic_light	🛑	stop_sign

ico	shortcode	ico	shortcode
🚧	`construction`		

Transport Water

ico	shortcode	ico	shortcode
⚓	`anchor`	⛵	`boat` `sailboat`
🛶	`canoe`	🚤	`speedboat`
🛳️	`passenger_ship`	⛴️	`ferry`
🛥️	`motor_boat`	🚢	`ship`

Transport Air

ico	shortcode	ico	shortcode
✈️	`airplane`	🛩️	`small_airplane`
🛫	`flight_departure`	🛬	`flight_arrival`
🪂	`parachute`	💺	`seat`
🚁	`helicopter`	🚟	`suspension_railway`
🚡	`mountain_cableway`	🚠	`aerial_tramway`
🛰️	`artificial_satellite`	🚀	`rocket`
🛸	`flying_saucer`		

Hotel

ico	shortcode	ico	shortcode
🛎️	`bellhop_bell`	🧳	`luggage`

Time

ico	shortcode	ico	shortcode
⌛	hourglass	⏳	hourglass_flowing_sand
⌚	watch	⏰	alarm_clock
⏱️	stopwatch	⏲️	timer_clock
🕰️	mantelpiece_clock	🕛	clock12
🕧	clock1230	🕐	clock1
🕜	clock130	🕑	clock2
🕝	clock230	🕒	clock3
🕞	clock330	🕓	clock4
🕟	clock430	🕔	clock5
🕠	clock530	🕕	clock6
🕡	clock630	🕖	clock7
🕢	clock730	🕗	clock8
🕣	clock830	🕘	clock9
🕤	clock930	🕙	clock10
🕥	clock1030	🕚	clock11
🕦	clock1130		

Sky & Weather

ico	shortcode	ico	shortcode
🌑	new_moon	🌒	waxing_crescent_moon
🌓	first_quarter_moon	🌔	moon waxing_gibbous_moon
🌕	full_moon	🌖	waning_gibbous_moon
🌗	last_quarter_moon	🌘	waning_crescent_moon
🌙	crescent_moon	🌚	new_moon_with_face

ico	shortcode	ico	shortcode
🌛	first_quarter_moon_with_face	🌜	last_quarter_moon_with_fa
🌡️	thermometer	☀️	sunny
🌝	full_moon_with_face	🌞	sun_with_face
🪐	ringed_planet	⭐	star
🌟	star2	🌠	stars
🌌	milky_way	☁️	cloud
⛅	partly_sunny	⛈️	cloud_with_lightning_and_
🌤️	sun_behind_small_cloud	🌥️	sun_behind_large_cloud
🌦️	sun_behind_rain_cloud	🌧️	cloud_with_rain
🌨️	cloud_with_snow	🌩️	cloud_with_lightning
🌪️	tornado	🌫️	fog
🌬️	wind_face	🌀	cyclone
🌈	rainbow	🌂	closed_umbrella
☂️	open_umbrella	☔	umbrella
⛱️	parasol_on_ground	⚡	zap
❄️	snowflake	☃️	snowman_with_snow
⛄	snowman	☄️	comet
🔥	fire	💧	droplet
🌊	ocean		

Activities

- Event
- Award Medal
- Sport
- Game
- Arts & Crafts

Event

ico	shortcode	ico	shortcode
🎃	jack_o_lantern	🎄	christmas_tree
🎆	fireworks	🎇	sparkler
🧨	firecracker	✨	sparkles
🎈	balloon	🎉	tada
🎊	confetti_ball	🎋	tanabata_tree
🎍	bamboo	🎎	dolls
🎏	flags	🎐	wind_chime
🎑	rice_scene	🧧	red_envelope
🎀	ribbon	🎁	gift
🎗️	reminder_ribbon	🎟️	tickets
🎫	ticket		

Award Medal

ico	shortcode	ico	shortcode
🎖️	medal_military	🏆	trophy
🏅	medal_sports	🥇	1st_place_medal
🥈	2nd_place_medal	🥉	3rd_place_medal

Sport

ico	shortcode	ico	shortcode
⚽	soccer	⚾	baseball
🥎	softball	🏀	basketball
🏐	volleyball	🏈	football
🏉	rugby_football	🎾	tennis

ico	shortcode	ico	shortcode
🥏	flying_disc	🎳	bowling
🏏	cricket_game	🏑	field_hockey
🏒	ice_hockey	🥍	lacrosse
🏓	ping_pong	🏸	badminton
🥊	boxing_glove	🥋	martial_arts_uniform
🥅	goal_net	⛳	golf
⛸	ice_skate	🎣	fishing_pole_and_fish
🤿	diving_mask	🎽	running_shirt_with_sash
🎿	ski	🛷	sled
🥌	curling_stone		

Game

ico	shortcode	ico	shortcode
🎯	dart	🪀	yo_yo
🪁	kite	🔫	gun
🎱	8ball	🔮	crystal_ball
🪄	magic_wand	🎮	video_game
🕹	joystick	🎰	slot_machine
🎲	game_die	🧩	jigsaw
🧸	teddy_bear	🪅	pinata
🪆	nesting_dolls	♠	spades
♥	hearts	♦	diamonds
♣	clubs	♟	chess_pawn
🃏	black_joker	🀄	mahjong
🎴	flower_playing_cards		

Arts & Crafts

ico	shortcode	ico	shortcode
🎭	`performing_arts`	🖼️	`framed_picture`
🎨	`art`	🧵	`thread`
🪡	`sewing_needle`	🧶	`yarn`
🪢	`knot`		

Objects

- Clothing
- Sound
- Music
- Musical Instrument
- Phone
- Computer
- Light & Video
- Book Paper
- Money
- Mail
- Writing
- Office
- Lock
- Tool
- Science
- Medical
- Household
- Other Object

Clothing

ico	shortcode	ico	shortcode
👓	eyeglasses	🕶️	dark_sunglasses
🥽	goggles	🥼	lab_coat
🦺	safety_vest	👔	necktie
👕	shirt tshirt	👖	jeans
🧣	scarf	🧤	gloves
🧥	coat	🧦	socks
👗	dress	👘	kimono
🥻	sari	🩱	one_piece_swimsuit
🩲	swim_brief	🩳	shorts
👙	bikini	👚	womans_clothes
👛	purse	👜	handbag
👝	pouch	🛍️	shopping
🎒	school_satchel	🩴	thong_sandal
👞	mans_shoe shoe	👟	athletic_shoe
🥾	hiking_boot	🥿	flat_shoe
👠	high_heel	👡	sandal
🩰	ballet_shoes	👢	boot
👑	crown	👒	womans_hat
🎩	tophat	🎓	mortar_board
🧢	billed_cap	🪖	military_helmet
⛑️	rescue_worker_helmet	📿	prayer_beads
💄	lipstick	💍	ring
💎	gem		

Sound

ico	shortcode	ico	shortcode
🔇	`mute`	🔈	`speaker`
🔉	`sound`	🔊	`loud_sound`
📢	`loudspeaker`	📣	`mega`
📯	`postal_horn`	🔔	`bell`
🔕	`no_bell`		

Music

ico	shortcode	ico	shortcode
🎼	`musical_score`	🎵	`musical_note`
🎶	`notes`	🎙	`studio_microphone`
🎚	`level_slider`	🎛	`control_knobs`
🎤	`microphone`	🎧	`headphones`
📻	`radio`		

Musical Instrument

ico	shortcode	ico	shortcode
🎷	`saxophone`	🪗	`accordion`
🎸	`guitar`	🎹	`musical_keyboard`
🎺	`trumpet`	🎻	`violin`
🪕	`banjo`	🥁	`drum`
🪘	`long_drum`		

Phone

ico	shortcode	ico	shortcode
📱	iphone	📲	calling
☎️	phone telephone	📞	telephone_receiver
📟	pager	📠	fax

Computer

ico	shortcode	ico	shortcode
🔋	battery	🔌	electric_plug
💻	computer	🖥️	desktop_computer
🖨️	printer	⌨️	keyboard
🖱️	computer_mouse	🖲️	trackball
💽	minidisc	💾	floppy_disk
💿	cd	📀	dvd
🧮	abacus		

Light & Video

ico	shortcode	ico	shortcode
🎥	movie_camera	🎞️	film_strip
📽️	film_projector	🎬	clapper
📺	tv	📷	camera
📸	camera_flash	📹	video_camera
📼	vhs	🔍	mag
🔎	mag_right	🕯️	candle
💡	bulb	🔦	flashlight

ico	shortcode	ico	shortcode
🏮	izakaya_lantern lantern	🪔	diya_lamp

Book Paper

ico	shortcode	ico	shortcode
📔	notebook_with_decorative_cover	📕	closed_book
📖	book open_book	📗	green_book
📘	blue_book	📙	orange_book
📚	books	📓	notebook
📒	ledger	📃	page_with_curl
📜	scroll	📄	page_facing_up
📰	newspaper	🗞	newspaper_roll
🗂	bookmark_tabs	🔖	bookmark
🏷	label		

Money

ico	shortcode	ico	shortcode
💰	moneybag	🪙	coin
💴	yen	💵	dollar
💶	euro	💷	pound
💸	money_with_wings	💳	credit_card
🧾	receipt	💹	chart

Mail

ico	shortcode	ico	shortcode
✉️	`envelope`	📧	`e-mail` `email`
📨	`incoming_envelope`	📩	`envelope_with_arrow`
📤	`outbox_tray`	📥	`inbox_tray`
📦	`package`	📫	`mailbox`
📪	`mailbox_closed`	📬	`mailbox_with_mail`
📭	`mailbox_with_no_mail`	📮	`postbox`
🗳️	`ballot_box`		

Writing

ico	shortcode	ico	shortcode
✏️	`pencil2`	✒️	`black_nib`
🖋️	`fountain_pen`	🖊️	`pen`
🖌️	`paintbrush`	🖍️	`crayon`
📝	`memo` `pencil`		

Office

ico	shortcode	ico	shortcode
💼	`briefcase`	📁	`file_folder`
📂	`open_file_folder`	🗂️	`card_index_dividers`
📅	`date`	📆	`calendar`
🗒️	`spiral_notepad`	🗓️	`spiral_calendar`
📇	`card_index`	📈	`chart_with_upwards_trend`

ico	shortcode	ico	shortcode
📉	chart_with_downwards_trend	📊	bar_chart
📋	clipboard	📌	pushpin
📍	round_pushpin	📎	paperclip
🖇	paperclips	📏	straight_ruler
📐	triangular_ruler	✂	scissors
🗃	card_file_box	🗄	file_cabinet
🗑	wastebasket		

Lock

ico	shortcode	ico	shortcode
🔒	lock	🔓	unlock
🔏	lock_with_ink_pen	🔐	closed_lock_with_key
🔑	key	🗝	old_key

Tool

ico	shortcode	ico	shortcode
🔨	hammer	🪓	axe
⛏	pick	⚒	hammer_and_pick
🛠	hammer_and_wrench	🗡	dagger
⚔	crossed_swords	💣	bomb
🪃	boomerang	🏹	bow_and_arrow
🛡	shield	🪚	carpentry_saw
🔧	wrench	🪛	screwdriver
🔩	nut_and_bolt	⚙	gear
🗜	clamp	⚖	balance_scale

ico	shortcode	ico	shortcode
🦯	probing_cane	🔗	link
⛓️	chains	🪝	hook
🧰	toolbox	🧲	magnet
🪜	ladder		

Science

ico	shortcode	ico	shortcode
⚗️	alembic	🧪	test_tube
🧫	petri_dish	🧬	dna
🔬	microscope	🔭	telescope
📡	satellite		

Medical

ico	shortcode	ico	shortcode
💉	syringe	🩸	drop_of_blood
💊	pill	🩹	adhesive_bandage
🩺	stethoscope		

Household

ico	shortcode	ico	shortcode
🚪	door	🛗	elevator
🪞	mirror	🪟	window
🛏️	bed	🛋️	couch_and_lamp
🪑	chair	🚽	toilet
🪠	plunger	🚿	shower

ico	shortcode	ico	shortcode
🛁	bathtub	🪤	mouse_trap
🪒	razor	🧴	lotion_bottle
🧷	safety_pin	🧹	broom
🧺	basket	🧻	roll_of_paper
🪣	bucket	🧼	soap
🪥	toothbrush	🧽	sponge
🧯	fire_extinguisher	🛒	shopping_cart

Other Object

ico	shortcode	ico	shortcode
🚬	smoking	⚰️	coffin
🪦	headstone	⚱️	funeral_urn
🧿	nazar_amulet	🗿	moyai
🪧	placard		

Symbols

- Transport Sign
- Warning
- Arrow
- Religion
- Zodiac
- Av Symbol
- Gender
- Math
- Punctuation
- Currency
- Other Symbol

- Keycap
- Alphanum
- Geometric

Transport Sign

ico	shortcode	ico	shortcode
🏧	`atm`	🚮	`put_litter_in_its_place`
🚰	`potable_water`	♿	`wheelchair`
🚹	`mens`	🚺	`womens`
🚻	`restroom`	🚼	`baby_symbol`
🚾	`wc`	🛂	`passport_control`
🛃	`customs`	🧳	`baggage_claim`
🛄	`left_luggage`		

Warning

ico	shortcode	ico	shortcode
⚠️	`warning`	🚸	`children_crossing`
⛔	`no_entry`	🚫	`no_entry_sign`
🚳	`no_bicycles`	🚭	`no_smoking`
🚯	`do_not_litter`	🚱	`non-potable_water`
🚷	`no_pedestrians`	📵	`no_mobile_phones`
🔞	`underage`	☢️	`radioactive`
☣️	`biohazard`		

Arrow

ico	shortcode	ico	shortcode
⬆️	arrow_up	↗️	arrow_upper_right
➡️	arrow_right	↘️	arrow_lower_right
⬇️	arrow_down	↙️	arrow_lower_left
⬅️	arrow_left	↖️	arrow_upper_left
↕️	arrow_up_down	↔️	left_right_arrow
↩️	leftwards_arrow_with_hook	↪️	arrow_right_hook
⤴️	arrow_heading_up	⤵️	arrow_heading_down
🔃	arrows_clockwise	🔄	arrows_counterclockwise
🔙	back	🔚	end
🔛	on	🔜	soon
🔝	top		

Religion

ico	shortcode	ico	shortcode
🛐	place_of_worship	⚛️	atom_symbol
🕉️	om	✡️	star_of_david
☸️	wheel_of_dharma	☯️	yin_yang
✝️	latin_cross	☦️	orthodox_cross
☪️	star_and_crescent	☮️	peace_symbol
🕎	menorah	🔯	six_pointed_star

Zodiac

ico	shortcode	ico	shortcode
♈	aries	♉	taurus

ico	shortcode	ico	shortcode
♊	gemini	♋	cancer
♌	leo	♍	virgo
♎	libra	♏	scorpius
♐	sagittarius	♑	capricorn
♒	aquarius	♓	pisces
⛎	ophiuchus		

Av Symbol

ico	shortcode	ico	shortcode
🔀	twisted_rightwards_arrows	🔁	repeat
🔂	repeat_one	▶️	arrow_forward
⏩	fast_forward	⏭️	next_track_button
⏯️	play_or_pause_button	◀️	arrow_backward
⏪	rewind	⏮️	previous_track_button
🔼	arrow_up_small	⏫	arrow_double_up
🔽	arrow_down_small	⏬	arrow_double_down
⏸️	pause_button	⏹️	stop_button
⏺️	record_button	⏏️	eject_button
🎦	cinema	🔅	low_brightness
🔆	high_brightness	📶	signal_strength
📳	vibration_mode	📴	mobile_phone_off

Gender

ico	shortcode	ico	shortcode
♀️	female_sign	♂️	male_sign

ico	shortcode	ico	shortcode
⚧	transgender_symbol		

Math

ico	shortcode	ico	shortcode
✖	heavy_multiplication_x	➕	heavy_plus_sign
➖	heavy_minus_sign	➗	heavy_division_sign
♾	infinity		

Punctuation

ico	shortcode	ico	shortcode
‼	bangbang	⁉	interrobang
❓	question	❔	grey_question
❕	grey_exclamation	❗	exclamation heavy_exclamation_mark
〰	wavy_dash		

Currency

ico	shortcode	ico	shortcode
💱	currency_exchange	💲	heavy_dollar_sign

Other Symbol

ico	shortcode	ico	shortcode
⚕	medical_symbol	♻	recycle
⚜	fleur_de_lis	🔱	trident

ico	shortcode	ico	shortcode
📛	name_badge	🔰	beginner
⭕	o	✅	white_check_mark
☑️	ballot_box_with_check	✔️	heavy_check_mark
❌	x	❎	negative_squared_cross_mark
➰	curly_loop	➿	loop
〽️	part_alternation_mark	✳️	eight_spoked_asterisk
✴️	eight_pointed_black_star	❇️	sparkle
©️	copyright	®️	registered
™️	tm		

Keycap

ico	shortcode	ico	shortcode
#️⃣	hash	*️⃣	asterisk
0️⃣	zero	1️⃣	one
2️⃣	two	3️⃣	three
4️⃣	four	5️⃣	five
6️⃣	six	7️⃣	seven
8️⃣	eight	9️⃣	nine
🔟	keycap_ten		

Alphanum

ico	shortcode	ico	shortcode
🔠	capital_abcd	🔡	abcd
🔢	1234	🔣	symbols
🔤	abc	🅰️	a

ico	shortcode	ico	shortcode
🆎	ab	🅱️	b
🆑	cl	🆒	cool
🆓	free	ℹ️	information_source
🆔	id	Ⓜ️	m
🆕	new	🆖	ng
🅾️	o2	🆗	ok
🅿️	parking	🆘	sos
🆙	up	🆚	vs
🈁	koko	🈂️	sa
🈷️	u6708	🈶	u6709
🈯	u6307	🉐	ideograph_advantage
🈹	u5272	🈚	u7121
🈲	u7981	🉑	accept
🈸	u7533	🈴	u5408
🈳	u7a7a	㊗️	congratulations
㊙️	secret	🈺	u55b6
🈵	u6e80		

Geometric

ico	shortcode	ico	shortcode
🔴	red_circle	🟠	orange_circle
🟡	yellow_circle	🟢	green_circle
🔵	large_blue_circle	🟣	purple_circle
🟤	brown_circle	⚫	black_circle
⚪	white_circle	🟥	red_square
🟧	orange_square	🟨	yellow_square

ico	shortcode	ico	shortcode
🟩	green_square	🟦	blue_square
🟪	purple_square	🟫	brown_square
⬛	black_large_square	⬜	white_large_square
◼️	black_medium_square	◻️	white_medium_square
◾	black_medium_small_square	◽	white_medium_small_squ
▪️	black_small_square	▫️	white_small_square
🔶	large_orange_diamond	🔷	large_blue_diamond
🔸	small_orange_diamond	🔹	small_blue_diamond
🔺	small_red_triangle	🔻	small_red_triangle_dow
💠	diamond_shape_with_a_dot_inside	🔘	radio_button
⬛	white_square_button	🔲	black_square_button

Flags

- Flag
- Country Flag
- Subdivision Flag

Flag

ico	shortcode	ico	shortcode
🏁	checkered_flag	🚩	triangular_flag_on_post
🎌	crossed_flags	🏴	black_flag
🏳️	white_flag	🏳️‍🌈	rainbow_flag
🏳️‍⚧️	transgender_flag	🏴‍☠️	pirate_flag

Country Flag

ico	shortcode	ico	shortc
	ascension_island		andorra
	united_arab_emirates		afghanistan
	antigua_barbuda		anguilla
	albania		armenia
	angola		antarctica
	argentina		american_samoa
	austria		australia
	aruba		aland_islands
	azerbaijan		bosnia_herzegovi
	barbados		bangladesh
	belgium		burkina_faso
	bulgaria		bahrain
	burundi		benin
	st_barthelemy		bermuda
	brunei		bolivia
	caribbean_netherlands		brazil
	bahamas		bhutan
	bouvet_island		botswana
	belarus		belize
	canada		cocos_islands
	congo_kinshasa		central_african_
	congo_brazzaville		switzerland
	cote_divoire		cook_islands
	chile		cameroon
	cn		colombia

ico	shortcode	ico	shortc
🇨🇵	clipperton_island	🇨🇷	costa_rica
🇨🇺	cuba	🇨🇻	cape_verde
🇨🇼	curacao	🇨🇽	christmas_island
🇨🇾	cyprus	🇨🇿	czech_republic
🇩🇪	de	🇩🇬	diego_garcia
🇩🇯	djibouti	🇩🇰	denmark
🇩🇲	dominica	🇩🇴	dominican_republ
🇩🇿	algeria	🇪🇦	ceuta_melilla
🇪🇨	ecuador	🇪🇪	estonia
🇪🇬	egypt	🇪🇭	western_sahara
🇪🇷	eritrea	🇪🇸	es
🇪🇹	ethiopia	🇪🇺	eu european_union
🇫🇮	finland	🇫🇯	fiji
🇫🇰	falkland_islands	🇫🇲	micronesia
🇫🇴	faroe_islands	🇫🇷	fr
🇬🇦	gabon	🇬🇧	gb uk
🇬🇩	grenada	🇬🇪	georgia
🇬🇫	french_guiana	🇬🇬	guernsey
🇬🇭	ghana	🇬🇮	gibraltar
🇬🇱	greenland	🇬🇲	gambia
🇬🇳	guinea	🇬🇵	guadeloupe
🇬🇶	equatorial_guinea	🇬🇷	greece
🇬🇸	south_georgia_south_sandwich_islands	🇬🇹	guatemala
🇬🇺	guam	🇬🇼	guinea_bissau
🇬🇾	guyana	🇭🇰	hong_kong

ico	shortcode	ico	shortc
	heard_mcdonald_islands		honduras
	croatia		haiti
	hungary		canary_islands
	indonesia		ireland
	israel		isle_of_man
	india		british_indian_oc
	iraq		iran
	iceland		it
	jersey		jamaica
	jordan		jp
	kenya		kyrgyzstan
	cambodia		kiribati
	comoros		st_kitts_nevis
	north_korea		kr
	kuwait		cayman_islands
	kazakhstan		laos
	lebanon		st_lucia
	liechtenstein		sri_lanka
	liberia		lesotho
	lithuania		luxembourg
	latvia		libya
	morocco		monaco
	moldova		montenegro
	st_martin		madagascar
	marshall_islands		macedonia
	mali		myanmar
	mongolia		macau

ico	shortcode	ico	shortc
	northern_mariana_islands		martinique
	mauritania		montserrat
	malta		mauritius
	maldives		malawi
	mexico		malaysia
	mozambique		namibia
	new_caledonia		niger
	norfolk_island		nigeria
	nicaragua		netherlands
	norway		nepal
	nauru		niue
	new_zealand		oman
	panama		peru
	french_polynesia		papua_new_guinea
	philippines		pakistan
	poland		st_pierre_miquelo
	pitcairn_islands		puerto_rico
	palestinian_territories		portugal
	palau		paraguay
	qatar		reunion
	romania		serbia
	ru		rwanda
	saudi_arabia		solomon_islands
	seychelles		sudan
	sweden		singapore
	st_helena		slovenia
	svalbard_jan_mayen		slovakia

ico	shortcode	ico	shortc
	sierra_leone		san_marino
	senegal		somalia
	suriname		south_sudan
	sao_tome_principe		el_salvador
	sint_maarten		syria
	swaziland		tristan_da_cunha
	turks_caicos_islands		chad
	french_southern_territories		togo
	thailand		tajikistan
	tokelau		timor_leste
	turkmenistan		tunisia
	tonga		tr
	trinidad_tobago		tuvalu
	taiwan		tanzania
	ukraine		uganda
	us_outlying_islands		united_nations
	us		uruguay
	uzbekistan		vatican_city
	st_vincent_grenadines		venezuela
	british_virgin_islands		us_virgin_islands
	vietnam		vanuatu
	wallis_futuna		samoa
	kosovo		yemen
	mayotte		south_africa
	zambia		zimbabwe

Subdivision Flag

ico	shortcode	ico	shortcode
	`england`		`scotland`
	`wales`		

typst.

typst demo.

```
t
```

```
// demo0
#import "@preview/cetz:0.3.4": canvas, draw
#import draw: line, content, rect, circle, bezier, group

#set page(width: auto, height: auto, margin: 15pt)

#canvas({
  // Core constants and reusable values
  let plot_width = 24
  let timeline_width = 16
  let time_tick_spacing = 1.0
  let step_width = time_tick_spacing
  let step_gap = 0.1
  let box_width_factor = 0.9
  let rect_height = 0.3
  let border_radius = 0.05
  let gap = 0.15
  let strategy_y_positions = (11, 6.5, 2.0) // Further compressed y-
positions

  // Colors - more pastel version for better text readability
  // Grays
  let dark_gray = rgb("#5D6B7A")
  let cpu_color = rgb("#8899AA")
  let gpu_color = rgb("#6A7A8A")
  let section_bg = rgb("#F7FAFC")
  // Define a consistent CPU light gray color for all strategies
  let cpu_light_gray = cpu_color.lighten(70%)

  // Blues with pastel tones
  let light_blue = rgb("#BBDEFB")
  let mid_blue = rgb("#90CAF9")
  let dark_blue = rgb("#64B5F6")

  // Greens with pastel tones
  let light_green = rgb("#C8E6C9")
  let mid_green = rgb("#A5D6A7")
  let dark_green = rgb("#81C784")

  // Oranges with pastel tones
  let light_orange = rgb("#FFE0B2")
  let mid_orange = rgb("#FFCC80")
  let dark_orange = rgb("#FFB74D")
```

```
  // Reds with pastel tones
  let light_red = rgb("#FFCDD2")
  let mid_red = rgb("#EF9A9A")
  let dark_red = rgb("#E57373")

  // Helper functions
  let draw_sim_block(x_pos, y_pos, width, color, label, task_label: "",
opacity: 100%) = {
    rect(
      (x_pos, y_pos - rect_height / 2),
      (x_pos + width, y_pos + rect_height / 2),
      fill: color.transparentize(100% - opacity),
      stroke: color.darken(10%),
      radius: border_radius,
      name: "block-" + label,
    )
    if task_label != "" {
      content((x_pos + width / 2, y_pos), text(size: 8pt)[#task_label],
name: "label-" + label)
    }
  }

  let draw_structure_rect(x_pos, y_pos, width, color, label, struct_name)
= {
    rect(
      (x_pos, y_pos),
      (x_pos + 0.95 * width, y_pos + 0.1),
      fill: color.transparentize(20%),
      stroke: none,
      name: "block-" + struct_name,
    )
    content((x_pos + width / 2, y_pos + 0.15), text(size: 8pt)[#label],
name: "label-" + struct_name, anchor: "south")
  }

  let draw_util_bar(x_pos, y_pos, percentage, label, is_bad: false) = {
    // Bar showing CPU or GPU utilization - consolidated function
    let width = 2.8
    let height = 0.4

    // Background bar
    rect(
      (x_pos, y_pos - height / 2),
      (x_pos + width, y_pos + height / 2),
      fill: rgb("#E2E8F0"),
```

```
      stroke: 0.5pt,
      radius: 0.1,
      name: "bar-bg-" + label,
    )

    // Filled portion with appropriate color
    let fill_width = width * percentage / 100

    // Determine color based on percentage and is_bad flag
    let fill_color = if is_bad {
      light_red.darken(20%) // Use darker red for explicitly marked "bad"
utilization
    } else if percentage < 30 {
      light_red
    } else if percentage < 70 {
      light_orange
    } else {
      light_green
    }

    rect(
      (x_pos, y_pos - height / 2),
      (x_pos + fill_width, y_pos + height / 2),
      fill: fill_color,
      stroke: none,
      radius: (north-west: 0.1, south-west: 0.1),
      name: "bar-fill-" + label,
    )

    // Label
    content(
      (rel: (0.03, 0), to: "bar-bg-" + label),
      text(size: 8pt, weight: "bold")[#label #percentage%],
      name: "bar-label-" + label,
      anchor: "center",
    )
  }

  // Title, subtitle and legend
  let title_y = 15.0
  content(
    (plot_width / 2, title_y),
    text(weight: "bold", size: 16pt)[GPU Batching Strategies for Atomistic
Simulations],
    name: "main-title",
```

```
  )
  content(
    (rel: (0, -1), to: "main-title"),
    text(size: 12pt)[Comparison of Unbatched vs. BinningAutoBatcher vs.
InFlightAutoBatcher],
    name: "subtitle",
  )


  // Add section backgrounds and scenario setups
  let strategies = (
    (strategy_y_positions.at(0), "Unbatched\nSimulations", (80, 5)),
    (strategy_y_positions.at(1), "Binning\nAutoBatcher", (40, 60)),
    (strategy_y_positions.at(2), "InFlight\nAutoBatcher", (60, 90)),
  )

  for (idx, (y_pos, label, (cpu_util, gpu_util))) in
strategies.enumerate() {
    // Background
    rect(
      (0.5, y_pos - 2.0),
      (plot_width - 0.5, y_pos + 2.0),
      fill: section_bg,
      stroke: none,
      radius: border_radius * 3,
      name: "section-bg-" + str(idx),
    )

    // Scenario label
    content((2, y_pos + 1.0), text(weight: "bold", size: 12pt)[#label],
name: "scenario-label-" + str(idx))

    // Utilization meters
    draw_util_bar(.8, y_pos - 0.1, cpu_util, "CPU Utilization")

    // GPU utilization - mark as bad for unbatched simulations (idx == 0)
    draw_util_bar(.8, y_pos - 1, gpu_util, "GPU Utilization", is_bad: idx
== 0)

    // CPU/GPU labels
    content(
      (4.6, y_pos + 1.3),
      text(fill: cpu_color, size: 10pt, weight: "bold")[CPU],
      anchor: "east",
      name: "cpu-label-" + str(idx),
```

```
    )
    content(
      (4.6, y_pos - 0.5),
      text(fill: gpu_color, size: 10pt, weight: "bold")[GPU],
      anchor: "east",
      name: "gpu-label-" + str(idx),
    )

    // Timeline axis with ticks
    line(
      (4.8, y_pos - 1.5),
      (4.8 + timeline_width, y_pos - 1.5),
      stroke: 0.8pt,
      mark: (end: "stealth", fill: black, scale: 0.5),
      name: "timeline-" + str(idx),
    )

    for t in range(1, 17) {
      let x_pos = 4 + t * time_tick_spacing
      if x_pos <= 4 + timeline_width {
        line(
          (x_pos, y_pos - 1.5 - 0.05),
          (x_pos, y_pos - 1.5 + 0.05),
          stroke: 0.8pt,
          name: "tick-" + str(idx) + "-" + str(t),
        )
        content(
          (rel: (0, -0.2), to: "tick-" + str(idx) + "-" + str(t)),
          text(size: 8pt)[t=#t],
          anchor: "north",
          name: "tick-label-" + str(idx) + "-" + str(t),
        )
      }
    }

    // CPU/GPU separator
    let separator_y = if idx == 0 { y_pos + 0.6 } else { y_pos + 0.95 }
    line(
      (4, separator_y),
      (4 + timeline_width, separator_y),
      stroke: (dash: "dotted", paint: dark_gray.lighten(30%)),
      name: "separator-" + str(idx),
    )
  }
```

```
  // 1. Unbatched Simulations
  let unbatched_cpu_y = strategy_y_positions.at(0) + 1.4
  let unbatched_gpu_y = strategy_y_positions.at(0) - 0.5

  // Structure rectangles with consistent pattern
  let unbatched_structures = (
    (5.0, 2.0, mid_blue, "Structure 1"),
    (7.0, 2.3, mid_green, "Structure 2"),
    (9.3, 1.8, mid_orange, "Structure 3"),
    (11.1, 2.1, mid_red, "Structure 4"),
    (13.2, 1.8, dark_green, "Structure 5"),
    (15.0, 2.0, dark_green.darken(10%), "Structure 6"),
    (17.0, 1.6, dark_green.darken(15%), "Structure 7"),
    (18.6, 1.5, dark_green.darken(20%), "Structure 8"),
  )

  for (idx, (x_pos, width, color, label)) in
unbatched_structures.enumerate() {
    draw_structure_rect(x_pos, unbatched_cpu_y, width, color, label,
"unbatched-" + str(idx + 1) + "-cpu")
  }

  content((21.8, unbatched_cpu_y - 1.9), text(size: 10pt, weight: "bold")
[... continues], anchor: "center")

  // CPU operation blocks in a loop
  for x in range(22) {
    let x_offset = 5.25 + x * 0.7
    if x_offset < 20 {
      draw_sim_block(
        x_offset,
        unbatched_cpu_y - 0.4,
        0.4,
        cpu_light_gray,
        "unbatched-cpu-op-" + str(x),
        task_label: "",
        opacity: 90%,
      )
    }
  }

  // GPU blocks with structure IDs and positions
  let gpu_blocks = (
    (1, dark_blue, (5.5, 6.5)),
    (2, dark_green, (7.5, 8.5)),
```

```
    (3, dark_orange, (9.8, 10.6)),
    (4, dark_red, (11.5, 12.5)),
    (5, dark_green, (13.8, 14.6, 15.4)),
    (6, dark_green.darken(10%), (16.2, 16.8, 17.4, 18.0, 18.6)),
    (7, dark_green.darken(15%), (19.2, 19.8)),
  )

  for (struct_num, color, positions) in gpu_blocks {
    for (idx, pos) in positions.enumerate() {
      draw_sim_block(
        pos,
        unbatched_gpu_y,
        0.3 * box_width_factor,
        color,
        "unbatched-" + str(struct_num) + "-gpu-" + str(idx + 1),
        task_label: "S" + str(struct_num),
      )
    }
  }

  // 2. BinningAutoBatcher
  let binning_cpu_y = strategy_y_positions.at(1) + 1.3
  let binning_gpu_y = strategy_y_positions.at(1) - 0.85

  // CPU processing blocks - simplified to only show prep batch operations
  let cpu_blocks = (
    (5.0, "Prep batch"),
    (10.4, "Prep batch"),
  )

  for (idx, (x_pos, label)) in cpu_blocks.enumerate() {
    draw_sim_block(x_pos, binning_cpu_y, 1.3, cpu_light_gray, "binning-op-
" + str(idx), task_label: label)
  }


  // Batch visualization setup
  let s_colors = (
    dark_blue,
    dark_green,
    dark_orange,
    dark_red,
    dark_blue.darken(10%),
  )
```

```
  let s_labels = ("S1", "S2", "S3", "S4", "S5")

  // Bin parameters and active patterns
  let bins = (
    (
      // Bin 1: 6 steps with completion patterns
      0,
      false,
      (
        (1, 1, 1, 1, 1), // Step 0: all active
        (1, 1, 1, 1, 1), // Step 1: all active
        (0, 1, 1, 1, 1), // Step 2: structure 1 completes earlier
        (0, 0, 1, 1, 1), // Step 3: structures 1,2 complete
        (0, 0, 0, 1, 0), // Step 4: structures 1,2,3,5 complete
        (0, 0, 0, 1, 0), // Step 5: only 4 remains
      ),
    ),
    (
      // Bin 2: 6 steps with different completion pattern
      6,
      true,
      (
        (1, 1, 1, 1, 1), // Step 0: all active
        (1, 1, 1, 1, 1), // Step 1: structure 3 completes early
        (0, 1, 0, 1, 1), // Step 2: structures 1,3 complete
        (0, 1, 0, 0, 1), // Step 3: only 4,5 remain
        (0, 1, 0, 0, 0), // Step 4: only 5 remains
        (0, 1, 0, 0, 0), // Step 5: all complete
      ),
    ),
  )

  // Draw both bins with common function
  for (start_step, is_second_bin, patterns) in bins {
    for step in range(patterns.len()) {
      let x_pos = 5.0 + (start_step + step) * (step_width - step_gap)
      let box_width = (step_width - step_gap) * box_width_factor
      let box_x_start = x_pos + ((step_width - step_gap) - box_width) / 2

      // Draw each structure slot
      for i in range(5) {
        let block_y = binning_gpu_y - 0.3 + i * (rect_height + gap)
        let binning_block_name = "binning-block-" + str(start_step) + "-"
+ str(step) + "-" + str(i)
```

```
        if patterns.at(step).at(i) == 1 {
          let color = s_colors.at(i)
          let label = if is_second_bin { "S" + str(i + 6) } else {
s_labels.at(i) }

          rect(
            (box_x_start, block_y - rect_height / 2),
            (box_x_start + box_width, block_y + rect_height / 2),
            fill: color,
            stroke: color.darken(20%),
            radius: border_radius,
            name: binning_block_name,
          )

          content(
            (box_x_start + box_width / 2, block_y),
            text(size: 7pt)[#label],
            anchor: "center",
            name: binning_block_name + "-label",
          )
        } else {
          // Empty placeholder
          rect(
            (box_x_start, block_y - rect_height / 2),
            (box_x_start + box_width, block_y + rect_height / 2),
            fill: light_red.transparentize(90%),
            stroke: (dash: "dotted", paint:
light_red.transparentize(30%)),
            radius: border_radius,
            name: binning_block_name + "-empty",
          )
        }
      }

      // Add completion label for the last step
      if step == patterns.len() - 1 {
        content(
          (box_x_start - 0.25, binning_gpu_y - 0.45),
          text(size: 8pt, fill: dark_gray, style: "italic")[Batch #if
is_second_bin [2] else [1] Complete],
          anchor: "center",
          name: "batch-" + str(if is_second_bin { 2 } else { 1 }) + "-
complete-label",
        )
      }
```

```
    }
  }

  for (x_pos, percentage) in ((5.6, 100), (8.0, 60), (9.8, 30), (11, 100),
(13.5, 60), (15.5, 20)) {
    content(
      (x_pos, binning_gpu_y + 2.7),
      text(size: 8pt, fill: dark_gray)[#percentage% GPU],
      anchor: "center",
    )
  }

  // Explanatory arrows and labels - removing the isolated red arrow
  content(
    (10.5, binning_gpu_y - 1.5),
    text(size: 7pt, fill: dark_red, style: "italic")[Must wait for batch
to complete, resulting in underutilized GPU],
    anchor: "center",
    name: "must-wait",
  )

  let batch_end_x = 5.0 + 6 * (step_width - step_gap)
  let timeline_y = strategy_y_positions.at(1) - 1.5

  line(
    "must-wait.north",
    (batch_end_x, timeline_y),
    stroke: (dash: "dotted", paint: dark_red),
    mark: (end: "stealth", fill: dark_red, scale: 0.4, offset: 0.05),
    name: "must-wait-arrow",
  )

  // 3. In-flightAutoBatcher
  let inflight_cpu_y = strategy_y_positions.at(2) + 1.3
  let inflight_gpu_y = strategy_y_positions.at(2) - 0.9

  // Structure colors and labels
  let all_colors = (
    dark_blue,
    dark_green,
    dark_orange,
    dark_red,
    dark_blue.darken(10%),
    dark_green.darken(10%),
    dark_orange.darken(10%),
```

```
    dark_red.darken(10%),
    dark_blue.darken(20%),
    dark_green.darken(20%),
  )
  let all_labels = ("S1", "S2", "S3", "S4", "S5", "S6", "S7", "S8", "S9",
"S10")

  // Structure placement by time step - fix structure continuity
  let structures_by_step = (
    (0, 1, 2, 3, 4), // t=1: S1-S5
    (0, 1, 2, 3, 4), // t=2: S1-S5 (S1 continues for longer)
    (0, 6, 2, 3, 4), // t=3: S1 continues, S6 replaces S2, S3-S5 continue
    (5, 6, 7, 3, 4), // t=4: S5, S6-S8, S4 continue (S1 completed, S5
swapped in)
    (5, 6, 7, 8, 4), // t=5: S5-S9, S4 (S3 completed, S8 swapped in)
    (5, 6, 7, 8, 9), // t=6: S5-S10 (S4 completed, S9 swapped in)
    (5, 6, 7, 8, 9), // t=7: still fully filled
    (5, 6, 7, -1, -1), // t=8: final step, partially filled (about 50%) -
using S6 instead of isolated S4
  )

  // Place CPU operations equidistant, one for each time step
  // Create the same number of prep boxes as structure steps (8 steps)

  // Draw CPU operations - one per time step
  for step in range(structures_by_step.len()) {
    let x_pos = 5.0 + step * (step_width - step_gap)
    let box_width = (step_width - step_gap) * box_width_factor * 0.7 //
Make boxes 10% wider (70% instead of 60%)

    // Left align by using 10% of remaining space as left margin instead
of centering
    let box_x_start = x_pos + ((step_width - step_gap) - box_width) * 0.1

    // Use the same generic gray for all prep blocks
    draw_sim_block(
      box_x_start,
      inflight_cpu_y,
      box_width,
      cpu_light_gray,
      "inflight-prep-" + str(step),
      task_label: "Prep",
    )
  }
```

```
  // Draw all structures by step
  for step in range(structures_by_step.len()) {
    let x_pos = 5.0 + step * (step_width - step_gap)
    let box_width = (step_width - step_gap) * box_width_factor
    let box_x_start = x_pos + ((step_width - step_gap) - box_width) / 2

    // Create a batch name for this step
    let batch_name = "inflight-batch-" + str(step)

    // Draw each structure position
    for pos in range(5) {
      let struct_idx = structures_by_step.at(step).at(pos)
      let block_y = inflight_gpu_y - 0.3 + pos * (rect_height + gap)
      let block_name = "inflight-block-" + str(step) + "-" + str(pos)

      if struct_idx != -1 {
        let color = all_colors.at(struct_idx)
        let label = all_labels.at(struct_idx)

        // Structure box
        rect(
          (box_x_start, block_y - rect_height / 2),
          (box_x_start + box_width, block_y + rect_height / 2),
          fill: color,
          stroke: color.darken(20%),
          radius: border_radius,
          name: block_name,
        )

        content(
          (box_x_start + box_width / 2, block_y),
          text(size: 7pt)[#label],
          name: block_name + "-label",
          anchor: "center",
        )

        // Swap-in indicator
        if step > 0 {
          let prev_structures = structures_by_step.at(step - 1)
          let prev_block_name = "inflight-block-" + str(step - 1) + "-" +
str(pos)

          if pos < prev_structures.len() and prev_structures.at(pos) !=
struct_idx {
            // Simple diagonal line for swap-in indicator
```

```
            line(
              (rel: (0, 0), to: block_name + ".south-west"), // End at the
block edge
              (rel: (-0.25, -0.3), to: block_name + ".south-west"), //
Start from outside
              stroke: (dash: "dotted", paint: dark_green),
              mark: (start: "stealth", fill: dark_green, scale: 0.3),
              name: block_name + "-swap-indicator",
            )
          }
        }
      } else {
        // Empty placeholders with dotted lines to indicate unused GPU
cycles
        rect(
          (box_x_start, block_y - rect_height / 2),
          (box_x_start + box_width, block_y + rect_height / 2),
          fill: light_red.transparentize(90%),
          stroke: (dash: "dotted", paint: light_red.transparentize(30%)),
          radius: border_radius,
          name: block_name + "-empty",
        )
      }
    }

    // Mark the last batch specially
    if step == structures_by_step.len() - 1 {
      // Store a reference to the last batch for later reference
      let final_block_name = "inflight-block-" + str(step) + "-0"

      // Instead of using absolute positioning, use relative positioning
      // based on the named elements we just created
      content(
        (rel: (box_width + 1.7, 0.2), to: final_block_name),
        text(size: 8pt, fill: dark_gray, style: "italic")[Final batch
partially filled],
        anchor: "center",
        name: "final-batch-label",
      )
    }
  }

  // First label with frame
  content(
    (plot_width / 2, -0.8),
```

```
    text(
      size: 9pt,
      weight: "bold",
      fill: dark_green,
    )[In-flight batching achieves highest GPU utilization and maximizes
predictions per unit time],
    frame: "rect",
    fill: light_green.transparentize(70%),
    stroke: dark_green,
    padding: 3pt,
    radius: border_radius,
  )

  // Summary notes
  content(
    (plot_width / 2, -3),
    box(width: 50em)[
      *Unbatched:* Each simulation runs sequentially with most
calculations on CPU and minimal GPU utilization\
      *Binning:* Fixed-size batches improve GPU utilization but can't
adapt to varying simulation completion times\
      *In-flight:* Dynamic reallocation eliminates GPU idle time by
immediately adding new structures when others complete. Color changes
indicate in-flight structure replacement.
    ],
    frame: "rect",
    fill: rgb("#F7FAFC"),
    stroke: 0.5pt,
    padding: (10pt, 10pt, 0pt),
    radius: border_radius,
  )
})
```